interpretable and scalable MACHINE OPERATIONS



GCP DECISION TREE CLASSIFIER FOR CARBON CAPTURE & STORAGE OR CARBON UTILIZATION PRIORITIES IN ALBERTA

Tj ティー ジェー 978-1-0698848-1-7

Table of Contents

Introduction 4
Machine Learning Project Lifecycle 6

Scoping
- Business Need 8
- Problem Discovery 8
- The Goal 8
- Author Motivation 9
- Software Engineering 10
- Mechanical Engineering 11
- Degrees of Automation 12
- Deployment Strategies 14
- Reflection 16

Data and Modeling
- Emission Data 18
- Data Exploratory 20
  - Data Preview 22
  - Data Type, Duplicates, Null values 24
- Data ELT : Extract, Load and Transformation 26
- Data Transformation
  - Transformation Road Map 28
  - Transformation into df_clean dataframe 30
  - Transformation into df_ab dataframe 32
  - Transformation into facility dataframe 34
  - Transformation into above dataframe 36
  - Training Data Development : X and y 38
- Reflection 40

Delivery
- Introduction to Decision Tree Classifier 42
- Interpreting a Simple Decision Tree Classifier 44
- Interpreting a Scalable Decision Tree Classifier 46
- Classification with Original Dataset 48
  - Confusion Matrix 50

Classification Report and Label Distribution 52
Decision Tree Chart 54
Sankey Chart 56
Delivery Improvement 58
Classification with Balanced Dataset 60
Classification Report and Label Distribution 62
Confusion Matrix 64
Decision Tree Chart 66
Reflection 68

Deployment
Scalability 70
Cloud Engineering 72
DevOps 74
MLOps 76
Introduction to GCP and other cloud platforms 78
Gemini Enterprise Agent Platform 80
Saving and Sharing Machine Learning Model 82
Simple Deployment Procedure 84
Buckets 86
Endpoints 88
Model Registry 90
Monitoring and Observability 92
Security and Compliance 94
GCP Account and Billing 96
Credit Card Verification 98
Three Months Trial 100
Example of GCP Cost 102
Reflection 104

Summary
Dealing with Decarbonization 106
Final Reflections 108
Index 110
References 118

Introduction

As part of my academic project in the technical track scholarship I received from the Alberta Machine Intelligence Institute (amii), I chose to document my reflections and learning journey in a book. Currently working in energy sector supporting oil refineries, petrochemicals, hydrogen and ammonia production in Machine Operation role, I've seen firsthand how AI skills can transform traditional industries. This book brings together the practical techniques I've developed for building interpretable machine-learning (ML) classifiers and designing scalable machine-learning operations (MLOps) that can be applied in real industrial environments.

Amii has played a pivotal role in expanding AI training opportunities for Canadian energy workers. Through the Government of Canada's investment of more than $9 million in the AI Pathways: Energizing Canada's Low-Carbon Workforce initiative, nearly 5,000 workers will gain foundational and applied AI skills. This effort is part of the Sustainable Jobs Training Fund and is supported by an additional $19.7 million in federal funding under the Canadian Sovereign AI Compute Strategy. Together, these investments aim to accelerate AI research, expand compute capacity, and help Canadian businesses bring AI-enabled products and services to market—strengthening productivity and competitiveness across the economy.

This project applies data analytics and machine learning to a central question in Alberta's industrial decarbonization strategy: which facilities and sectors should be prioritized for Carbon Capture and Storage (CCS) or Carbon Utilization (CU) deployment? Rather than relying on costly, site-specific engineering assessments, the project demonstrates how publicly available emissions data can provide a scalable, repeatable, and transparent first-pass screening tool for policymakers, operators, and investors.

Alberta's industrial base is among the most emissions-intensive in Canada, carrying a disproportionate share of national greenhouse gas output. In 2022, the province accounted for about **38% of Canada's total** emissions—270 megatonnes of CO_2 equivalent—despite representing only 11% of the national population.



By 2023, emissions stood at **263.4 million tonnes CO_2e**, with oil and gas production responsible for **59% of the provincial total**, making it the dominant source sector. Alberta's per-capita emissions reached 59.8 tonnes CO_2e, more than triple the national average of 18.2 tonnes.

Reaching net-zero by 2050 requires more than incremental efficiency improvements; it demands large-scale deployment of carbon-capture technologies. Carbon Capture and Storage (CCS) captures CO_2 at industrial sources and stores it in geological formations. Carbon Utilization (CU) uses captured CO_2 or mixed greenhouse gas streams as feedstock in industrial processes, converting an emissions liability into an economic asset.

Both approaches are supported through Alberta's Technology Innovation and Emissions Reduction **(TIER) regulation**, the province's industrial carbon-pricing and emissions-trading system introduced in January 2020.

TIER applies to large industrial facilities that emitted **100,000 tonnes or more** of CO_2 equivalent in 2016 or any subsequent year.

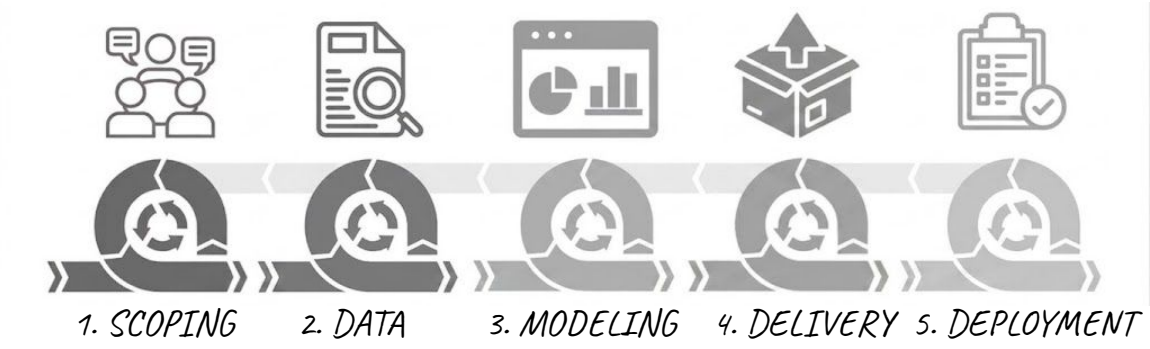
These facilities must reduce emissions below regulated benchmarks or contribute to a compliance fund. From a CCS/CU perspective, they are also the most attractive deployment candidates, producing concentrated, high-volume emissions that make carbon capture economically viable.

In 2023, TIER-covered entities emitted **164.7 megatonnes** of CO_2e, underscoring the scale of regulated industrial activity. This project uses the 100,000-tonne threshold as the baseline filter for facility inclusion.

Traditional site assessments are costly, slow, and dependent on engineering data that is not always publicly available. A data-driven screening approach using publicly reported emissions enables policymakers, industry groups, and investors to identify high-priority CCS/CU candidates and compare facilities at scale before committing to full feasibility studies.

Machine Learning Project Lifecycle (MLPL)

Using the MLPL framework helps break complex, real-world problems into structured, manageable steps. It provides a clear sequence of decisions that guides a project from definition to execution, improving focus, consistency, and alignment between technical work and business needs.



SCOPING

Scoping translates a broad business need into a clear, solvable technical problem. Rather than jumping directly into modeling, this stage defines success metrics, clarifies constraints, and aligns expectations early. Teams work with stakeholders to refine the problem, assess feasibility, and understand organizational priorities. This phase blends business understanding with problem discovery to ensure the project addresses a real need rather than a vague idea. The goal is to confirm that machine learning is appropriate, determine what "success" looks like, and frame the challenge so the technical team can design a solution that delivers measurable value. Effective scoping prevents wasted effort by ensuring everyone agrees on objectives before any data or modeling work begins.

DATA

Once the project is defined, attention shifts to acquiring and preparing the data. This stage involves gathering information from internal systems, external sources, or sensors, then cleaning, transforming, and validating it so the model learns from meaningful patterns rather than noise. Teams conduct exploratory data analysis to understand structure, quality, and relationships, followed by feature engineering to convert raw inputs into useful variables. Key activities include handling missing values, addressing outliers, ensuring consistency across datasets, and checking for bias. The goal is to curate a representative, high-quality dataset that reflects the true characteristics of the problem domain. Strong data preparation establishes the foundation for reliable modeling and reduces downstream errors.

MODELING

In the modeling stage, data scientists select and train algorithms to identify patterns within the prepared dataset. The process is iterative and experimental, requiring multiple attempts because initial models rarely perform well. Teams test different algorithms, adjust hyperparameters, and use cross-validation to avoid overfitting. Key activities include model selection, training, tuning, and evaluating performance against the metrics defined during Scoping. This phase focuses on building a model that generalizes well to new, unseen data rather than memorizing historical examples. The ultimate goal is to produce a reliable, interpretable, and scalable model that meets the project's technical and business requirements.

DELIVERY

Before a model is approved, it must be validated against the success criteria established during Scoping. Delivery ensures the model solves the real business problem, not just the technical one. Teams test performance on hold-out datasets, check for fairness issues, and perform sanity checks to confirm predictions make sense in real-world conditions. Results are presented to stakeholders to verify that expectations are met. Once the model passes these checks, it is ready for deployment with confidence that it adds value and minimizes risk.

DEPLOYMENT

Deployment ensures a model that performs well in the lab can operate reliably in production. Real-world environments introduce challenges such as data drift, latency constraints, integration issues, and the need for continuous monitoring. This stage involves selecting a deployment strategy, integrating the model into existing systems, and establishing monitoring processes to track performance and detect failures. The goal is to maintain consistent, trustworthy behavior once the model is live.

Dale (Lane 2020) used Scratch with ML API like machinelearningforkids.co.uk to simplify above phases

MLforKids	MLPL Phases	Why
Train	Data & Modeling	Building the model from data
Learn & Test	Delivery	Validating, packaging, preparing for production
Make	Deployment	Using the model in a real application
(None)	Scoping	MLforKids assumes the goal was already defined

Example of scratch program about pattern for Grade 4



Business Needs

Decision makers want to know which facilities and sectors should be first in line for Carbon Capture and Storage (CCS) or Carbon Utilization (CU). Instead of relying on expensive, case-by-case engineering reviews, they have decided to use a machine learning model to do the initial screening work.

Problem Discovery

The core question is whether basic facility information—its sector and emissions scale—can predict if it is a viable CCS or CU candidate. Traditional site assessments are slow, costly, and rely on engineering data that is not always publicly available. A data-driven approach using reported emissions enables hundreds of facilities to be evaluated simultaneously, something expert judgment cannot achieve at scale. This makes the task a classification problem. Because decision-makers need simple, transparent logic, an interpretable model is essential. A Decision Tree fits well: it handles sector data, produces clear rule-based outputs, and aligns with the threshold-driven structure used in the TIER framework.

> Dino/Francesco (Esposito 2020) outlined three core foundations of shallow learning—distinct from deep learning. These include simple prediction using linear regression, complex prediction using decision trees for non-linear relationships, and better decision-making through ensemble methods that combine multiple models to improve accuracy and robustness.
> As one of the oldest method in machine learning, decision tree with its simple training and prediction promises its accuracy in many domain and should be taken as one of the basic benchmark before any more complex learning algorithm is tried (Alpaydin 2021)

The Goal

The goal is to train an interpretable Decision Tree to classify TIER-regulated facilities as CCS Candidates or Potential CU Candidates using NAICS industry codes and annual emissions. The model supports fast, scalable screening without full engineering assessments. Beyond facility-level predictions, the analysis highlights which NAICS industries in Alberta contain the highest concentration of CCS/CU priority sites, giving decision makers a sector-level view of where CCUS deployment efforts should be focused.

Author's Motivation

While carbon management serves as the core case study of this text, the underlying philosophy of this work is driven by a personal passion for data-driven precision and continuous improvement.

At my current role in a large kaizen-driven company, the culture of continuous improvement has deeply shaped how I approach problems. Working directly with machine operations has shown me the impact of automation and precision in manufacturing. To push those capabilities further, I've been building advanced machine-learning skills—especially in Python, matrix computation, and data visualization—to bridge hands-on engineering with modern data-driven methods.

Building this advanced skill set is not only personally transformative but universally applicable. The exact machine learning and MLOps principles explored in this book are highly relevant across countless sectors. Within manufacturing, these tools lay the groundwork for a future where traditional operations meet next-generation tech. While this project applies data science to climate infrastructure, the ultimate goal is to translate these capabilities back to the production floor when the right resources and tools become affordable—such as combining mass-preference data collection sensors and physical AI robotic arms with the cloud architecture learned here.



Skilled Trades
Tribal Knowledges



mass-preference data
collection sensors

By fusing a hands-on manufacturing background with advanced cloud computing via Google Cloud's Gemini Agentic Platform, this work seeks to bridge the gap between traditional machining and modern intelligent automation. This book represents a personal step toward expanding technical capabilities, honoring the spirit of Kaizen, and creating highly efficient, data-informed workflows for the future of industry.

There are more Machine Learning pillars beyond interpretability, including predictive performance, robustness, and data quality. MLOps also includes more pillars beyond scalability, such as observability, reproducibility, interoperability, and improvability (Kehrer, Kriser 2024). This book focuses specifically on interpretable, scalable and a little bit on observability of machine operations.

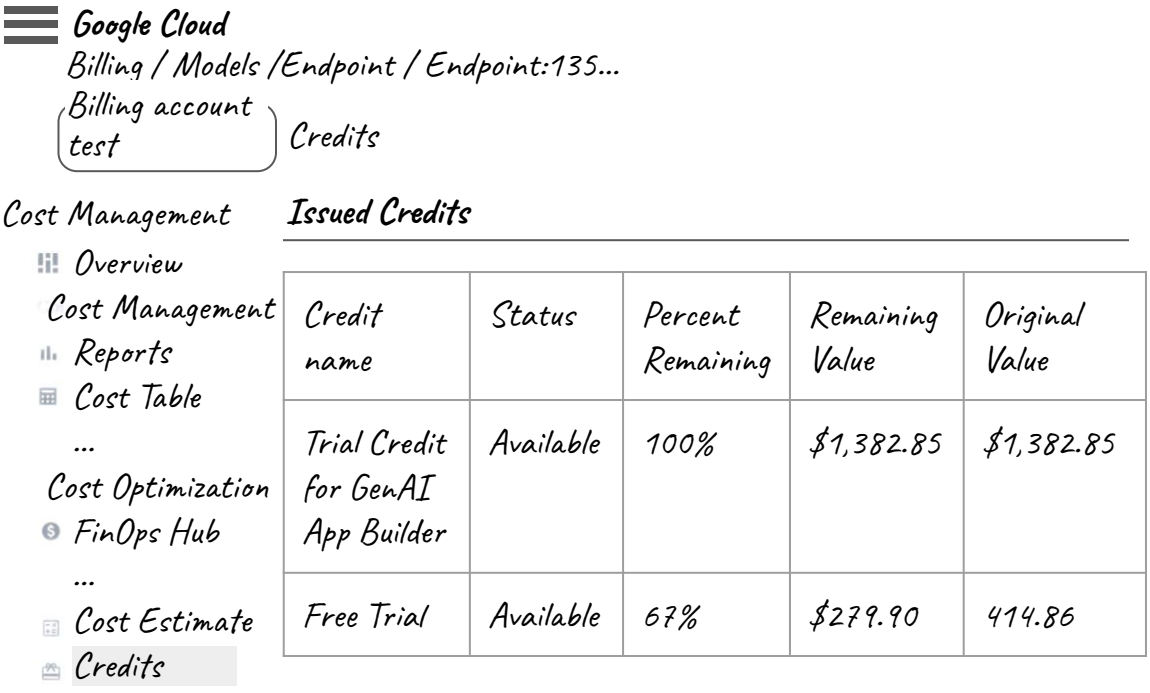
Software Engineering

From experimentation to production, it's not just training a model but designing a complete prediction system, which is a software engineering scope like below

Prediction Mode: Decide between Synchronous (real-time predictions) or Asynchronous (Batch) (high-volume, non-urgent processing). Since emissions data is typically reported and analyzed annually, I don't need real-time decision making.

Deployment Environment: Prediction systems can be deployed in the cloud, at the edge, or in-browser. For this project, deploying to the cloud (GCP) provides scalability and ease of use. In this project I will deploy the finished model to the Google Cloud for scalability instead of on-prem deployment.

Resource Limit: Constraints such as CPU, GPU, and memory may influence model choice. Since decision trees are lightweight, standard cloud resources are sufficient. Other configuration options are available but always be mindful of the cost of the services chosen.



Credit name	Status	Percent Remaining	Remaining Value	Original Value
Trial Credit for GenAI App Builder	Available	100%	$1,382.85	$1,382.85
Free Trial	Available	67%	$279.90	414.86

Security & Privacy: I would deploy endpoint that should be protected from unauthorized access. Even emissions data can be sensitive in industrial contexts, so basic security measures are important.

Mechanical Engineering

In addition to software-engineering considerations, the scoping phase of a CCS/CU project must incorporate the mechanical-engineering constraints that ultimately determine whether capture equipment can be integrated into an existing facility. Drawing on my experience at ABB, several factors deserve explicit attention. From a thermodynamics perspective (heat integration, energy_penalty), CO_2 capture introduces a substantial energy penalty (Assael 2011; Brannen 2025), meaning a site must have adequate thermal headroom, recoverable waste heat, and viable heat-integration pathways. Heat-transfer limitations also shape feasibility: cooling-water availability, ambient conditions, and existing heat-exchanger capacity determine whether capture units can be added without major redesign.

Fluid-mechanics constraints further influence suitability. Flue-gas flow rate, allowable pressure drop, ducting modifications, and CO_2 compression requirements all affect whether the system can be integrated without disrupting upstream processes. Transport considerations — including pipeline hydraulics (friction losses), phase behavior, and compression staging (Rabie 2024) — define whether CO_2 can be moved reliably from the capture unit to storage or utilization infrastructure. Material compatibility and safety must also be evaluated, since CO_2 impurities can accelerate corrosion, alter phase behavior, require more stringent containment strategies.

These engineering constraints can be translated into structured features for a machine-learning model, allowing the model to reflect physical feasibility rather than relying solely on emissions data or industry type. Incorporating such constraints produces classifications that are not only statistically sound but also operationally realistic.

Capture Technology	AI Technique	Energy Savings	Capture Efficiency	Scalability Potential	Cost Reduction	Case Study	Limitations
Amine Based Absorption	Neural Networks	10–15%	85–90%	Moderate(retrofitting challenges)	$5–10/ tonne	Boundary Dam	Data scarcity, sensor costs
Amine Based Absorption	Deep RL	15–20%	90–95%	Low (high computation costs)	$8–12/ tonne	TCM	Long training times, GPU requirements
Membrane Based Separation	SVM	8–10%	80–85%	High (need advanced material)	$3–7/ tonne	Stanford University	Membrane synthesis cost
Membrane Based Separation	DL (Inverse Design)	10–12%	85–90%	Moderate (fouling issues)	$5–8/ tonne	MIT	High synthesis costs, data requirements
Hybrid Capture Systems	Ensemble ML Models	12–18%	88–92%	Moderate (integration complexity)	$6–10/ tonne	Petra Nova	Legacy system integration
Direct Air Capture (DAC)	Neural Networks	10–15%	70–80%	Low (early stage tech)	$10–20/ tonne	Climeworks Pilot	High energy demand, dataset limited

Source: https://sciresjournals.com/ijstra/sites/default/files/IJSTRA-2025-0043.pdf

Degrees on Automation

Decision-making in this domain requires careful consideration. My model should not operate fully autonomously; it must support — not replace — human experts such as environmental engineers and policymakers. Drawing from a decade of visiting industrial plants that emit significant amounts of carbon dioxide, I cannot place full confidence in model-generated predictions based solely on emission numbers.

Dimension	Mechanical Pulp Mill
Carbon Stream Characteristics	High-volume, steady, centralized CO_2 streams (often 12–15% concentration). Predictable combustion-based emissions.
Process Integration Complexity	One or two large stacks make CCS retrofits straightforward.
Economics and ROI	Long-lived assets (40–60 years). Higher capital budgets. CCS investment more justifiable.
Regulatory Pressure	Subject to carbon-intensity regulations; qualify for BECCS incentives.
CO_2 Utilization Pathways	Can supply CO_2 for PCC production or industrial clusters. Biomass-derived CO_2 is attractive for CU.
Site-Specific Constraints	Large industrial land parcels with space for capture units, compressors, pipelines.
Future Industry Outlook	Stable or growing demand for packaging/tissue products. Long-term viability supports CCS payback.
Overall CCS/CU Feasibility	Technically and economically more feasible.

Each industrial site is fundamentally unique. The feasibility of adopting carbon capture and storage (CCS) or carbon utilization (CU) varies widely and depends on economic viability, site-specific return-on-investment considerations – especially for businesses facing uncertain futures – regulatory pressures, and many other contextual factors. Fully automated decisions in this space would carry substantial risk. For example, emissions from a mechanical pulp differ from beef-cattle slaughtering facility. It is not possible to generalize or assume that suitability of CCS or CU in one industry can be transferred to another.

Beef Cattle Slaughterhouse

Low-concentration (<5%), diffuse, scattered emissions from refrigeration, boilers, rendering, wastewater. Highly variable.

Many small emission points; no single major flue source. Integration is technically difficult.

Thin margins, volatile markets, shorter facility lifespan. CCS capital cost rarely viable.

Regulatory focus is on food safety, wastewater, and animal welfare—not CO_2.

No meaningful CU pathways; CO_2 is low-purity and biologically contaminated.

Often constrained urban/peri-urban sites with limited expansion room.

Uncertain future due to plant-based alternatives, market volatility, and regulatory shifts.

Low feasibility due to technical, economic, and regulatory barriers.

Human judgment must remain central in this process. At a minimum, any predictive model that spans multiple industries must be developed with extreme caution — and ideally, industries should be modeled separately. When data from fundamentally different sectors are combined, the likelihood of producing unreliable or misleading predictions increases significantly.

Even within the same industry, sites can differ dramatically in their ability to adopt carbon capture and storage (CCS) or carbon utilization. Competitive advantages, proprietary processes, operational maturity, and site-specific economic conditions all influence feasibility. These nuances cannot be captured by a generalized model without expert interpretation.

For these reasons, human expertise — particularly from environmental engineers, industry specialists, and policy analysts — must remain firmly in the loop to ensure that model outputs are contextualized, validated, and responsibly applied.

Deployment Strategies

Adopting Agentic (Vertex AI) as recommended for academic purpose for this project can be risky if the cost could be a concern. Running traditional MLOps using cheaper option such as native kubernetes orchestration and docker container also can be complex as it requires massive skills.

Question	GCP (Scalable and Established Cloud Compute/Storage)	Build Docker + Kubernetes Infrastructure from the beginning (limited scalability on prem or cloud)
How can I evaluate my model in a live environment while minimizing risk?	Provides built-in shadow deployment, canary testing, automated rollback, and monitoring. Minimal infrastructure risk because the platform manages scaling and isolation.	Requires manual setup of routing, monitoring, autoscaling, and rollback. Safe evaluation is possible but depends heavily on skilled DevOps engineers. Higher risk of misconfiguration.
Is there a way to compare my model's predictions with existing processes before taking action on them?	Yes – predictions can be logged in parallel with existing workflows. Vertex AI centralized logging, versioning, and evaluation, making comparison straightforward.	Also possible, but requires building custom logging, storage, and comparison pipelines. More engineering effort and higher operational overhead.
What deployment approach would give me confidence before full adoption?	A staged rollout using built-in tools: shadow mode → human-review mode → limited canary traffic → gradual scaling. Automated cost controls and rollback increase confidence.	Progressive rollout is feasible but requires manual configuration of service meshes, autoscaling, and rollback. Confidence depends on the maturity and expertise of the DevOps team.

Reflection on Scoping

The Big Picture

Training a machine-learning model to classify industrial sites as suitable for Carbon Capture and Storage (CCS) or Carbon Utilization (CU) requires far more than historical emissions and industry type. While these variables help identify where carbon is produced, they say little about whether a site can realistically support CCS or CU deployment. This limitation reflects a broader truth: CCS and CU are not simply emission-reduction technologies—they are infrastructure-dependent, geology-dependent, and policy-dependent systems. A model trained only on emissions cannot capture these structural constraints, and therefore cannot produce meaningful classifications.

To improve predictive power, the dataset must incorporate the same multidimensional factors that governments and developers consider when evaluating CCS/CU projects. First, geological suitability is essential. CCS requires deep saline aquifers or depleted hydrocarbon reservoirs with adequate porosity, permeability, and caprock integrity. Without subsurface storage potential within a feasible transport radius, CCS is impossible regardless of emission volume. CU, by contrast, may be more viable near industrial clusters that can use CO_2 as feedstock. Incorporating geological maps, storage capacity estimates, and proximity to known basins would dramatically improve model realism.

Second, infrastructure proximity is a decisive factor. CCS depends on pipelines, compression facilities, and access to large, stable energy supplies. CU depends on co-location with industries such as cement, chemicals, synthetic fuels, or greenhouses. Including features such as distance to existing CO_2 pipelines, natural gas networks, rail corridors, and industrial clusters would allow the model to reflect real-world deployment constraints.

☑ CCS
☐ CU = $f($ emission data, NAICS code, Σ years, geological suitability, infrastructure proximity, regulatory and policy context, economic and operational characteristics, strategic value $)$

Third, regulatory and policy context strongly influences feasibility. Jurisdictions with supportive CCS incentives, carbon pricing, or permitting frameworks are far more likely to host viable projects. Encoding regional policy variables—carbon price, tax credits, regulatory timelines, and provincial CCS strategies—would help the model distinguish between technically possible and politically supported sites.

Fourth, economic and operational characteristics matter. Capture cost varies with flue-gas concentration, plant age, thermal efficiency, and load factor. CU pathways depend on CO_2 purity, energy intensity, and market demand for CO_2-derived products. Adding variables such as capture cost curves, plant efficiency, fuel type, and CO_2 concentration would allow the model to differentiate between high-cost and low-cost capture opportunities.

Finally, including strategic value features—such as potential for knowledge transfer, regional workforce expertise, or alignment with national decarbonization priorities—reflects why projects like Boundary Dam remain important despite high costs (Ervine 2018). These factors help identify sites that may be prioritized for demonstration value rather than pure economics.

How Realistic Is the Project

This project is academically valuable because it integrates Python development, predictive modeling, and cloud deployment to build a simple ML / ML Ops for education purpose. Yet emissions data alone cannot meaningfully classify CCS or CU suitability. Real-world assessments require geological, economic, regulatory, and engineering factors that vary widely across regions and industries, far beyond simple emission profiles.

Is There a Part That Seems Especially Challenging

While I aim to build cloud-scalable skills, GCP which is recommended by the school presents challenges. Its interface is clean, but timely support—especially for billing issues—can be difficult to obtain. In contrast, AWS and Azure for example typically provide more responsive technical and commercial assistance, making them more reliable for rapid troubleshooting.

Emission Data

The data used in this project comes from the Greenhouse Gas Reporting Program (GHGRP), administered by Environment and Climate Change Canada (ECCC). This program provides publicly available emissions data for all Canadian industrial facilities that exceed the federal reporting threshold. For this project, one of the GHGRP datasets was modified to create a focused subset suitable for analysis.

Specifically, I used the PDGES-GHGRP-GHG Emissions (GES-2004-Present) dataset from GHGRP repository.



Beyond its use in this machine learning project, I have previously relied on similar emissions data (NPRI) in a practical industrial context. While working at Mersen during the second decade of the millenium, I used these emissions records to identify potential customers during field visits across Manitoba, Northern Ontario, and Newfoundland and Labrador, where I conducted surveys on direct current and alternating current electrical machines' contact.



For this project, the selected subset of the emissions dataset serves as the raw input for all analysis. To streamline the workflow and avoid repeated file uploads—especially when working in environments like Google Colab where sessions can time out—I stored a copy of the dataset in GitHub. This allows the extract-load-transform process to run locally and consistently. Herewith a sample python program to save above data into pd dataframe.

```
import pandas as pd

url = "https://raw.githubusercontent.com/FT-cnc/amii/main/GHGRP_Dataset.csv"
df = pd.read_csv(url)
```

Let's see what are the size of this data by using **df.shape** which going to show **(18772, 17)**

Which means there are **18772 rows** of data from **17 columns**

To view the column names, we use df.columns, which produces an output like the one below. The circled numbers and bold labels correspond to brief explanations provided on the next page.

```
Index(['GHGRP ID No. / No d'identification du PDGES',                    (1)
       'Reference Year / Année de référence',                            (2)
       'Facility Name / Nom de l'installation',                          (3)
       'Facility City or District or Municipality / Ville ou District ou
                                          Municipalité de l'installation',(4)
       'Facility Province or Territory / Province ou territoire de l'installation',(5)
       'Facility NPRI ID / Numéro d'identification de l'INRP',           (6)
       'Facility NAICS Code / Code SCIAN de l'installation',             (7)
       'English Facility NAICS Code Description / Description du code SCIAN
                                          de l'installation en anglais',(8)
       'French Facility NAICS Code Description / Description du code SCIAN
                                          de l'installation en français',(9)
       'Reporting Company Legal Name / Dénomination sociale de la société
                                          déclarante',                   (10)
       'Reporting Company Trade Name / Nom commercial de la société
                                          déclarante',                   (11)
       'CO2 (tonnes)',                                                   (12)
       'CH4 (tonnes)',                                                   (13)
       'CH4 (tonnes CO2e / tonnes éq. CO2)',                             (14) → CH4CO2e
       'N2O (tonnes)',                                                   (15)
       'N2O (tonnes CO2e / tonnes éq. CO2)',                             (16) → N2OCO2e
       'Total Emissions (tonnes CO2e) / Émissions totales (tonnes éq. CO2)'],(17) → TotalCO2e
      dtype='object')
```

Data Exploratory

We would cover 4 essential first-pass checks through data exploratory. In a real ML workflow, after data extraction it is usually followed by broader assessment such as,

Statistical Profiling – Compute summary statistics (mean, median, std, min/max, percentiles) to understand distribution shapes and detect anomalies (outliers).

Outlier Detection – Identify unusual values with z-scores, IQR (interquartile range), domain-specific thresholds. Outliers signal data-quality issue or important rare event.

Distribution Analysis – Plot histograms, KDEs (Kernel Density Estimation) to see each feature behaves and determine if transformations (log, scaling) are needed.

Correlation Analysis – Examine relationships between variables using correlation matrices or scatterplots. Useful for spotting multicollinearity or redundant features.

Class Imbalance Check – inspect target distribution prior to classify if SMOTE (synthetic minority over-sampling technique) or random oversampling needed.

Domain Validity Checks – Ensure values make sense in context (e.g., negative emissions, impossible temperatures, unrealistic flow rates). This is often more important than statistical checks.

Data Consistency Checks – Look for mismatched units, inconsistent naming, duplicated categories, or formatting differences across files.

Time-Series Integrity – If the data is temporal, verify ordering, missing timestamps, irregular intervals, and seasonality patterns.

Feature Cardinality Review – Inspect categorical columns for too many unique values, which may require grouping or encoding strategies.

Initial Target-Feature Relationship – Quick plots or simple models (baseline regression, decision tree) to see whether the data contains signal.

Data Preview – Scanning rows and columns provides an immediate understanding of the dataset's structure, patterns, and potential issues. It helps surface obvious anomalies such as unexpected values, formatting inconsistencies, or missing fields before deeper analysis begins. This step builds intuition about the data. In this particular case, I am already familiar with many of the industrial sites in Manitoba, Ontario, and Newfoundland and Labrador. Although this background is outside the scope of the book, it is worth noting that the dataset may include facilities that no longer exist, such as the Kingston cogen plant.

Data-Type Inspection – Machine-learning workflows depend on correct data types. Misclassified fields (for example, numeric values stored as strings or dates stored as plain text) disrupt feature engineering, break statistical operations, and cause pipeline errors. Verifying data types early prevents cascading failures later in the process.

Null-Value Identification – Missing data affects model accuracy, bias, and stability. Understanding where nulls occur allows me to choose appropriate strategies such as imputation, removal, or domain-specific corrections. Ignoring nulls can distort distributions or even cause model crashes.

Duplicate Detection – Duplicate records artificially inflate sample size, bias model learning, and reduce generalization. Identifying and resolving duplicates ensures each observation contributes unique information and prevents overfitting to repeated patterns.

Exploratory data analysis has also been central throughout my career. Before moving to Canada, I applied EDA extensively in purchasing role, later in manufacturing, and eventually into a relocation project responsible for moving and expanding factories at Schneider Electric. EDA supported multiple operational decisions: the Lotus Notes purchasing-approval workflow helped me prioritize negotiation efforts, while SAP ERP historical orders and inventory data allowed me to design relocation plans and point-of-use warehouse layouts for new factories. In each role, EDA provided the factual foundation needed to streamline processes, reduce risk, and make informed engineering and accounting decisions.

Data Preview

1	2	3	4	5	6	7	8	9
GHGRP /PDGES	Year	Facility Name	City	Prov	NPRI	NAICS	English Description	French Description
G10001	2023	Division Alma	Alma	QC	983	322111	Mechanical Pulp Mills	Usines de Pâte mécanique
G10003	2023	Foothills Pipeline, Alberta	AIRDRIE	AB	0	486210	Pipeline Transportation of Natural Gas	Transport du gaz naturel par gazoduc
G10004	2023	Kingston CoGen	Bath	ON	5765	221112	Fossil-Fuel Electric Power Generation	Production d'électricité à partir de combustible
G10006	2023	Redwater Fertilizer Operations	Sturgeon County	AB	2134	325313	Chemical Fertilizer(except Potash) Mfg.	Fabrication d' Engrais chimiques, sauf la potasse
G10007	2023	Alberta Envirofuels	Edmonton	AB	3974	325110	Petrochemical Manufacturing	Fabrication de produits pétrochimiques
G10008	2023	Alliance Pipeline Ltd. - AB Pipeline System	CALGARY	AB	0	486210	Pipeline Transportation of Natural Gas	Transport du gaz naturel par gazoduc
G10009	2023	Alberta-Pacific Forest Industries Inc.	COUNTY OF ATHABASCA	AB	1	322112	Chemical Pulp Mills	Usines de pâte chimique
G10010	2023	Rio Tinto Alcan Inc.	Kitimat	BC	2788	331313	Primary Production of Alumina and Aluminum	Production Primaire d'alumine et d'aluminium
G10011	2023	Algoma Steel Inc	Sault Ste. Marie	ON	1070	331110	Iron and Steel Mills and Ferro-Alloy Manufacturing	Sidérurgie
G10012	2023	Aluminerie Alouette inc.	Sept-Îles	QC	4778	331313	Primary Production of Alumina and Aluminum	Production Primaire d'alumine et d'aluminium

10	11	12	13	14	15	16	17
Legal Name	Trade Name	CO2	CH4	CH4CO2e	N2O	N2OCO2e	TotalCO2e
Produits Forestiers Résolu	Resolute FP Canada Inc.	35709	580	16261	0	164	52135
Foothills Pipe Lines Ltd.	Foothills Pipe Lines Ltd.	243466	441	12364	6	1668	257499
Kingston CoGen Limited Partnership	Kingston CoGen	934	0	5	0	6	946
Nutrien (Canada) Holdings ULC	Nutrien	1015330	2071	58013	24	6592	1079936
Keyera Corp	Keyera Corp	343296	15	438	6	1746	345481
Alliance Pipeline Ltd. Partnership Alberta-Pacific	Alliance Pipeline	522104	628	17589	10	2873	542567
Forest Industries Inc.	Alberta-Pacific Forest Industries Inc.	156230	199	5574	29	7726	169586
Rio Tinto Alcan Inc	Rio Tinto	712384	2	73	1	284	809153
Algoma Steel Inc.	Algoma Steel Inc.	4276836	398	11170	15	4159	4292166
Aluminerie Alouette inc.	Aluminerie Alouette inc.	1057955	1	46	2	625	1108309

Data Type, Duplicates, Null Values

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18772 entries, 0 to 18771
Data columns (total 17 columns):
 #   Column                                                  Non-Null Count  Dtype
---  ------                                                  --------------  -----
 0   GHGRP ID No. / No d'identification du PDGES             18772 non-null  object
 1   Reference Year / Année de référence                     18772 non-null  int64
 2   Facility Name / Nom de l'installation                   18772 non-null  object
 3   Facility City or District or Municipality /
     Ville ou District ou Municipalité de l'installation     16906 non-null  object
 4   Facility Province or Territory /
     Province ou territoire de l'installation                18771 non-null  object
 5   Facility NPRI ID / Numéro d'identification de l'INRP    18658 non-null  float64
 6   Facility NAICS Code / Code SCIAN de l'installation      18772 non-null  int64
 7   English Facility NAICS Code Description /
     Description du code SCIAN de l'installation en anglais  18772 non-null  object
 8   French Facility NAICS Code Description /
     Description du code SCIAN de l'installation en français 18772 non-null  object
 9   Reporting Company Legal Name /
     Dénomination sociale de la société déclarante           18772 non-null  object
 10  Reporting Company Trade Name /
     Nom commercial de la société déclarante                 10476 non-null  object
 11  CO2 (tonnes)                                            18772 non-null  float64
 12  CH4 (tonnes)                                            18608 non-null  float64
 13  CH4 (tonnes CO2e / tonnes éq. CO2)                      18772 non-null  float64
 14  N2O (tonnes)                                            18509 non-null  float64
 15  N2O (tonnes CO2e / tonnes éq. CO2)                      18772 non-null  float64
 16  Total Emissions (tonnes CO2e) /
     Émissions totales (tonnes éq. CO2)                      18772 non-null  float64
dtypes: float64(7), int64(2), object(8)
```

float64 data type may need conversion to ease the data representation.

df.duplicated().sum()

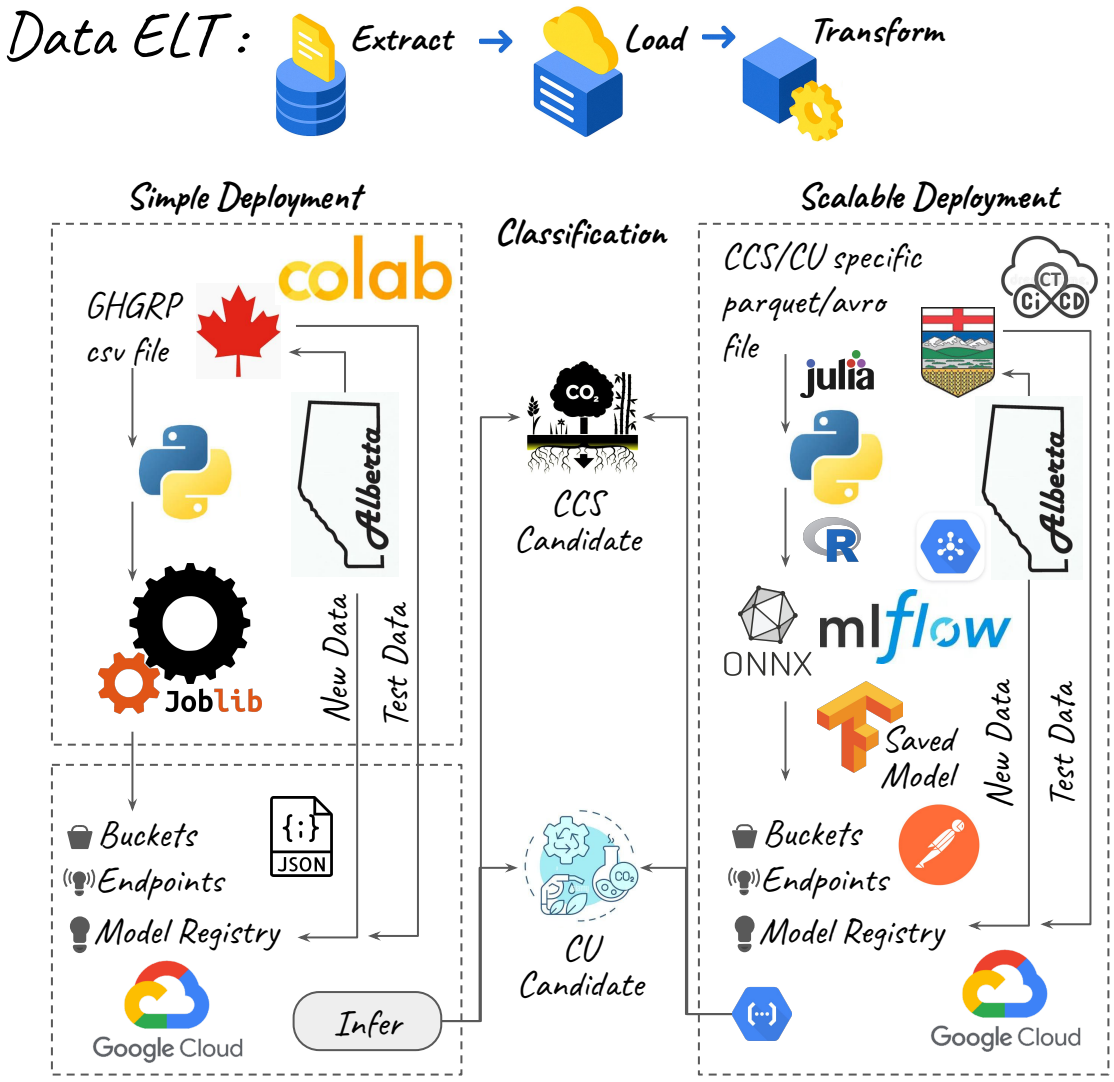
```
np.int64(0)
```

it means no data duplicates

```
(df.isnull().mean() * 100).round(2)
```

GHGRP ID No. / No d'identification du PDGES	0.00
Reference Year / Année de référence	0.00
Facility Name / Nom de l'installation	0.00
Facility City or District or Municipality / Ville ou District ou Municipalité de l'installation	9.94
Facility Province or Territory / Province ou territoire de l'installation	0.01
Facility NPRI ID / Numéro d'identification de l'INRP	0.61
Facility NAICS Code / Code SCIAN de l'installation	0.00
English Facility NAICS Code Description / Description du code SCIAN de l'installation en anglais	0.00
French Facility NAICS Code Description / Description du code SCIAN de l'installation en français	0.00
Reporting Company Legal Name / Dénomination sociale de la société déclarante	0.00
Reporting Company Trade Name / Nom commercial de la société déclarante	44.19
CO2 (tonnes)	0.00
CH4 (tonnes)	0.87
CH4 (tonnes CO2e / tonnes éq. CO2)	0.00
N2O (tonnes)	1.40
N2O (tonnes CO2e / tonnes éq. CO2)	0.00
Total Emissions (tonnes CO2e) / Émissions totales (tonnes éq. CO2)	0.00

These "five columns "with null values won't affect my modeling as they are not taken into account for classification model.



The diagrams show two analytical scenarios: one using a simple emissions-only data, and another use rich metadata that includes emission, carbon-stream characteristic, and operational attribute. This expanded structure leads more accurate classification of potential CCS or carbon-utilization sites by reflecting industrial process complexity rather than relying solely on emissions. Beyond Python, R is an increasingly adopted programming option (Nwanganga, Chapple 2020), and JavaScript with TensorFlow.js enables testing complex models in a browser (Suryadevara 2021). For decision-tree classifier, scikit-learn served through Node.js with ONNX Runtime or a Python microservice remains more practical than TensorFlow.js.

My experience with data transformation extends far beyond this project. I began developing these skills in 2008 working in mechanical engineering at General Electric. During that period, I helped implement a cloud-based SPC system that started at a single facility and quickly expanded across multiple sites and the database grew.

File format: In this book, we would pick a simple dataset in csv format with no time dependencies. As the file continuously grows bigger the csv file format may no longer sufficient. Parquet or Avro file format would be more likely that could bring more start-ups to collect this "presumably" highly sensitive data for classification purpose.

Programming Language: Python would stay strong in machine learning area, followed by Julia and R. From Data Engineering or DevOps point of view GO, Rust, Groovy could be also popular.



Example of GO project

Model file : Joblib is suitable for small models, especially in simple Python workflows. As model complexity increases, ONNX, MLflow, and TensorFlow SavedModel become preferable and support for cross-framework usage, versioning, scalable deployment.

Inference : In basic setups, model inference is often performed by sending a JSON file directly to the Model Registry. As pipelines become more complex and automated, REST APIs become the preferred mechanism for invoking the model. Under a CI/CD architecture, Cloud Functions and Pub/Sub can coordinate the arrival of new data, trigger inference, and publish CCS/CU classification results automatically.

We would not see critical needs for ELT now but when raw data getting complex, an enterprise grade tool like Informatica or at least GoAnyWhere MFT could be helpful.

Tool	GCP Equivalent	Best Fit
GoAnywhere MFT	Cloud Storage + Transfer Service + Composer	Secure file movement + automation
Informatica ETL/ELT	Cloud Data Fusion	Visual ETL, enterprise governance
Informatica Cloud Data Integration	Dataflow	Serverless ELT, streaming + batch
Spark-based Informatica workloads	Dataproc	Heavy transformations, big data
Informatica workflow orchestration	Cloud Composer	Airflow-based orchestration

Transformation Road Map





STEP 1 : **Extract** raw data, **Load** into df with 17 columns prior to **Transformation**

GHGRP ID No. / No d'identification du PDGES (1)

Reference Year / Année de reference (2)

Facility Name / Nom de l'installation (3)

Facility City or District or Municipality /
Ville ou District ou Municipalité de l'installation (4)

Facility Province or Territory / Province ou territoire de l'installation (5)

Facility NPRI ID / Numéro d'identification de l'INRP (6)

Facility NAICS Code / Code SCIAN de l'installation (7)

English Facility NAICS Code Description /
Description du code SCIAN de l'installation en anglaise (8)

French Facility NAICS Code Description /
Description du code SCIAN de l'installation en français (9)

Reporting Company Legal Name /
Dénomination sociale de la société déclarante (10)

Reporting Company Trade Name /
Nom commercial de la société déclarante (11)

CO2 (tonnes) (12)

CH4 (tonnes) (13)

CH4 (tonnes CO2e / tonnes éq. CO2) (14)

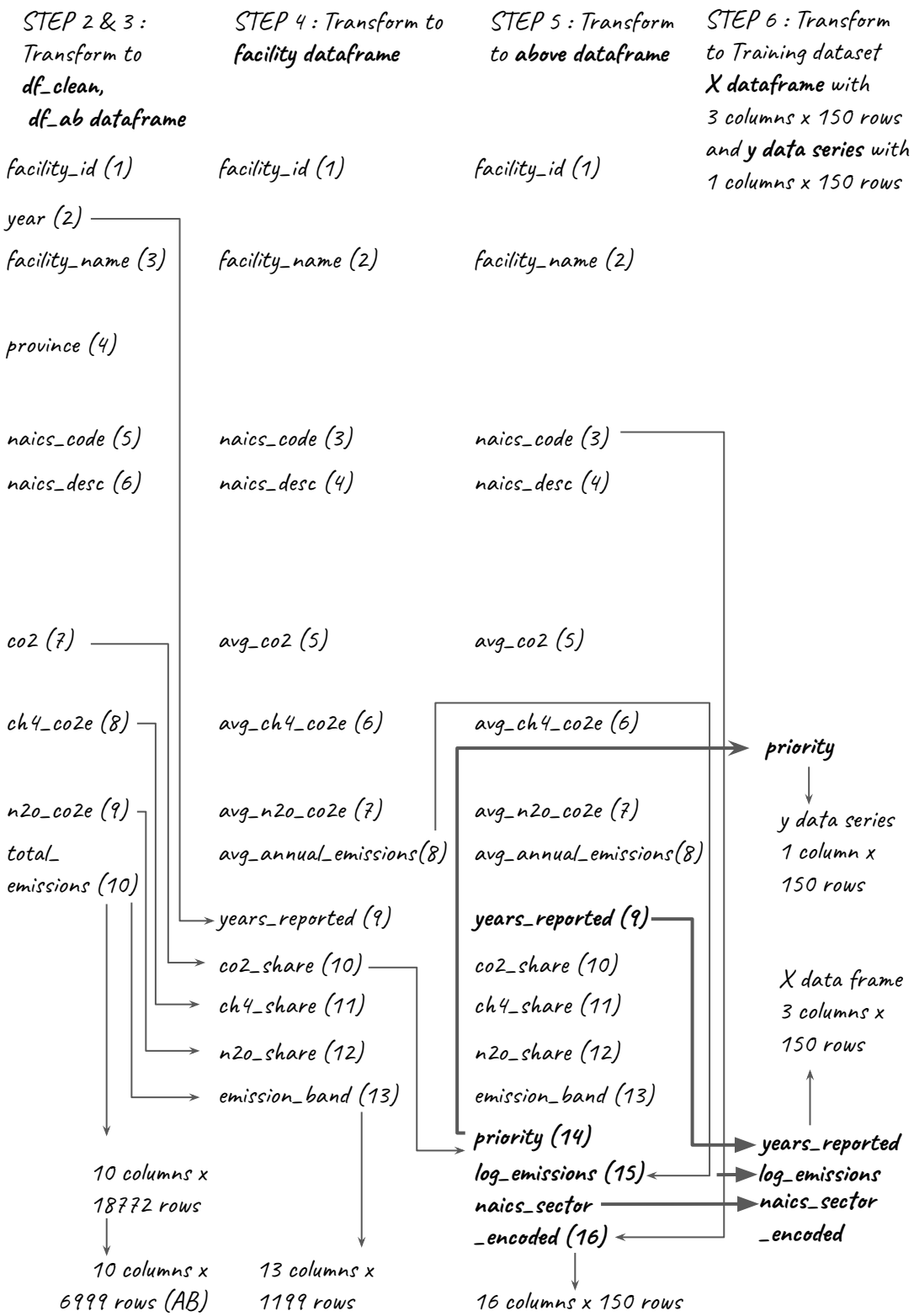
N2O (tonnes) (15)

N2O (tonnes CO2e / tonnes éq. CO2) (16)

Total Emissions (tonnes CO2e) / Émissions totales (tonnes éq. CO2) (17)

Before proceeding with modeling, we first need to narrow, clean, and transform the dataset. Raw Pandas dataframe is transformed into the X feature matrix—containing columns years_reported, log_emissions, naics_sector_encoded and the Y target series representing priority. These prepared structures form the inputs for subsequent modeling.

17 columns x 18772 rows


STEP 2 & 3 : Transform to df_clean, df_ab dataframe
STEP 4 : Transform to facility dataframe
STEP 5 : Transform to above dataframe
STEP 6 : Transform to Training dataset X dataframe with 3 columns x 150 rows and y data series with 1 columns x 150 rows
facility_id (1)
year (2)
facility_name (3)
province (4)
naics_code (5)
naics_desc (6)
co2 (7)
ch4_co2e (8)
n2o_co2e (9)
total_ emissions (10)
10 columns x 18772 rows
10 columns x 6999 rows (AB)
facility_id (1)
facility_name (2)
naics_code (3)
naics_desc (4)
avg_co2 (5)
avg_ch4_co2e (6)
avg_n2o_co2e (7)
avg_annual_emissions(8)
years_reported (9)
co2_share (10)
ch4_share (11)
n2o_share (12)
emission_band (13)
13 columns x 1199 rows
facility_id (1)
facility_name (2)
naics_code (3)
naics_desc (4)
avg_co2 (5)
avg_ch4_co2e (6)
avg_n2o_co2e (7)
avg_annual_emissions(8)
years_reported (9)
co2_share (10)
ch4_share (11)
n2o_share (12)
emission_band (13)
priority (14)
log_emissions (15)
naics_sector _encoded (16)
16 columns x 150 rows
priority
y data series 1 column x 150 rows
X data frame 3 columns x 150 rows
years_reported
log_emissions
naics_sector _encoded

Transformation into “df_clean“ dataframe



```
cols_to_keep = [
    "GHGRP ID No. / No d'identification du PDGES",
    "Reference Year / Année de référence",
    "Facility Name / Nom de l'installation",
    "Facility Province or Territory / Province ou territoire de l'installation",
    "Facility NAICS Code / Code SCIAN de l'installation",
    "English Facility NAICS Code Description /
        Description du code SCIAN de l'installation en anglais",
    "CO2 (tonnes)",
    "CH4 (tonnes CO2e / tonnes éq. CO2)",
    "N2O (tonnes CO2e / tonnes éq. CO2)",
    "Total Emissions (tonnes CO2e) / Émissions totales (tonnes éq. CO2)"
]
```

	facility_id	year	facility_name	province	naics_code	naics_desc
0	G10001	2023	Division Alma	Quebec	322111	Mechanical Pulp Mills
1	G10003	2023	Foothills Pipeline, Alberta	Alberta	486210	Pipeline Transportation of Natural Gas
2	G10004	2023	Kingston CoGen	Ontario	221112	Fossil-Fuel Electric Power Generation
3	G10006	2023	Redwater Fertilizer Operations	Alberta	325313	Chemical Fertilizer (except Potash) Manufacturing
4	G10007	2023	Alberta Envirofuels	Alberta	325110	Petrochemical Manufacturing
5	G10008	2023	Alliance Pipeline Ltd. - AB Pipeline System	Alberta	486210	Pipeline Transportation of Natural Gas
6	G10009	2023	Alberta-Pacific Forest Industries Inc.	Alberta	322112	Chemical Pulp Mills
7	G10010	2023	Rio Tinto Alcan Inc.	British Columbia	331313	Primary Production of Alumina and Aluminum
8	G10011	2023	Algoma Steel Inc	Ontario	331110	Iron and Steel Mills and Ferro-Alloy Manufacturer
9	G10012	2023	Aluminerie Alouette inc.	Quebec	331313	Primary Production of Alumina and Aluminum

```
df_clean = df[cols_to_keep].rename(columns={
    "GHGRP ID No. / No d'identification du PDGES": "facility_id",
    "Reference Year / Année de référence": "year",
    "Facility Name / Nom de l'installation": "facility_name",
    "Facility Province or Territory / Province ou territoire de l'installation": "province",
    "Facility NAICS Code / Code SCIAN de l'installation": "naics_code",
    "English Facility NAICS Code Description /
          Description du code SCIAN de l'installation en anglais": "naics_desc",
    "CO2 (tonnes)": "co2",
    "CH4 (tonnes CO2e / tonnes éq. CO2)": "ch4_co2e",
    "N2O (tonnes CO2e / tonnes éq. CO2)": "n2o_co2e",
    "Total Emissions (tonnes CO2e) / Émissions totales (tonnes éq. CO2)": "total_emissions"
  })

df_clean.head(10)
```

co2	ch4_co2e	n2o_co2e	total_emissions
35709	16261	164	52135
243466	12364	1668	257499
934	5	6	946
1015330	58013	6592	1079936
343296	438	1746	345481
522104	17589	2873	542567
156230	5574	7726	169586
712384	73	284	809153
4276836	11170	4159	4292166
1057955	46	625	1108309



Data Cleaning and Feature Engineering take 80 -90 % of Data Scientist time (Bowles 2020)

Transformation into “df_ab“ dataframe



```
df_ab = df_clean[df_clean["province"].str.lower() == "alberta"].copy()

df_ab.head(10)
```

to transform the last four columns to be more readable



	facility_id	year	facility_name	province	naics_code	naics_desc
1	G10003	2023	Foothills Pipeline, Alberta	Alberta	486210	Pipeline Transportation of Natural Gas
3	G10006	2023	Redwater Fertilizer Operations	Alberta	325313	Chemical Fertilizer (except Potash) Manufacturing
4	G10007	2023	Alberta Envirofuels	Alberta	325110	Petrochemical Manufacturing
5	G10008	2023	Alliance Pipeline Ltd. - AB Pipeline System	Alberta	486210	Pipeline Transportation of Natural Gas
6	G10009	2023	Alberta-Pacific Forest Industries Inc.	Alberta	322112	Chemical Pulp Mills
12	G10015	2023	ATCO Pipelines - Transmission System	Alberta	486210	Pipeline Transportation of Natural Gas
14	G10018	2023	Battle River Generating Station	Alberta	221112	Fossil-Fuel Electric Power Generation
23	G10029	2023	Brazeau Gas Plant	Alberta	211110	Oil and gas extraction (except oil sands)
24	G10030	2023	Brazeau River Gas Plant	Alberta	211110	Oil and gas extraction (except oil sands)
31	G10040	2023	Calgary Energy Centre	Alberta	221112	Fossil-Fuel Electric Power Generation

```
df_ab["year"] = pd.to_numeric(df_ab["year"], errors="coerce").astype("Int64")
df_ab["naics_code"] = pd.to_numeric(df_ab["naics_code"], errors="coerce")
                      .astype("Int64")
df_ab["co2"] = pd.to_numeric(df_ab["co2"].round(), errors="coerce").astype("Int64")
df_ab["ch4_co2e"] = pd.to_numeric(df_ab["ch4_co2e"].round(), errors="coerce")
                      .astype("Int64")
df_ab["n2o_co2e"] = pd.to_numeric(df_ab["n2o_co2e"].round(), errors="coerce")
                     .astype("Int64")
df_ab["total_emissions"] = pd.to_numeric(df_ab["total_emissions"].round(),
                      errors="coerce").astype("Int64")
```

co2	ch4_co2e	n2o_co2e	total_emissions	years_reported
243467	12364	1668	257499	20
1015331	58014	6592	1079937	20
343297	439	1746	345481	20
522105	17590	2873	542568	20
156231	5574	7726	169586	20
66028	36406	889	103348	20
903843	520	1279	905641	8
121925	7683	1824	131432	20
44528	1982	355	46865	5
834143	1633	4179	839955	20

Transformation into "facility" dataframe



```
facility=df_ab.groupby(["facility_id"], as_index=False).agg(
    facility_name=("facility_name", "first"),
    naics_code=("naics_code", "first"),
    naics_desc=("naics_desc", "first"),
    avg_co2=("co2", "mean"),
    avg_ch4_co2e=("ch4_co2e", "mean"),
    avg_n2o_co2e=("n2o_co2e", "mean"),
    avg_annual_emissions=("total_emissions", "mean"),
    years_reported=("year", "nunique")
)

facility.head(10)
```









facility_id	facility_name	naics_code	naics_desc	years_reported	avg_co2
G10003	Foothills Pipeline, Alberta	486210	Pipeline Transportation of Natural Gas	20	267942.25
G10006	Redwater Fertilizer Operations	325313	Chemical Fertilizer (except Potash) Manufacturing	20	1085053
G10007	Alberta Envirofuels Alliance Pipeline Ltd. -	325110	Petrochemical Manufacturing	20	296631.55
G10008	AB Pipeline System	486210	Pipeline Transportation of Natural Gas	20	524796.7
G10009	Alberta-Pacific Forest Industries Inc.	322112	Chemical Pulp Mills	20	106562.9
G10015	ATCO Pipelines - Transmission System	486210	Pipeline Transportation of Natural Gas	20	79927.45
G10017	Balzac Gas Plant	211113	Conventional Oil and Gas Extraction	8	190415
G10018	Battle River Generating Station	221112	Fossil-Fuel Electric Power Generation	20	3475388.05
G10023	Bonnie Glen Gas Plant	211113	Conventional Oil and Gas Extraction	5	247918.8
G10029	Brazeau Gas Plant	211110	Oil and gas extraction (except oil sands)	20	102309.7

```
TIER_THRESHOLD = 100_000

facility["emission_band"] = facility["avg_annual_emissions"].apply(
    lambda x: "Above Threshold" if x >= TIER_THRESHOLD else "Below Threshold"
)

facility["n2o_share"]   = facility["avg_n2o_co2e"] / facility["avg_annual_emissions"]
facility["ch4_share"] = facility["avg_ch4_co2e"] / facility["avg_annual_emissions"]
facility["co2_share"] = facility["avg_co2"] / facility["avg_annual_emissions"]
```

avg_ch4_co2e	avg_n2o_co2e	avg_annual_emissions	co2_share	ch4_share	n2o_share	emission_band
18222.2	2443.4	288607.9	0.928395	0.063138	0.008466	Above Threshold
61057.4	10397.65	1156675.75	0.938079	0.052787	0.008989	Above Threshold
856.55	1201.8	298728.85	0.992979	0.002867	0.004023	Above Threshold
32116.5	3482.35	561143.1	0.935228	0.057234	0.006206	Above Threshold
3292.65	14331.65	124230	0.857787	0.026504	0.115364	Above Threshold
129179.25	2789.4	213999.05	0.373494	0.603644	0.013035	Above Threshold
15988.25	1233.125	207852.25	0.916107	0.076921	0.005933	Above Threshold
1075.2	13128	3489591.25	0.99593	0.000308	0.003762	Above Threshold
32823.4	3145.8	283887.6	0.873299	0.115621	0.011081	Above Threshold
40837.5	1947.1	145094	0.705127	0.281455	0.01342	Above Threshold

Transformation into “above“ dataframe



```
CO2_THRESHOLD = 0.85
# Work only with above-threshold facilities (TIER-regulated) for the model
above = facility[facility["emission_band"] == "Above Threshold"].copy()

above["priority"] = above["co2_share"].apply(
 lambda x:   "CCS Candidate"  if x >= CO2_THRESHOLD else "Potential CU Candidate"
)
```

$$\text{priority} = f(\text{CO}_2\text{_share})$$

facility_id	priority	co2_share	facility_name	ch4_share	n2o_share	years_reported	naics_desc
G10003	CCS Candidate	93%	Foothills Pipeline, Alberta	6%	1%	20	Pipeline ...
G10006	CCS Candidate	94%	Redwater Fertilizer Operations	5%	1%	20	Fertilizer ...
G10007	CCS Candidate	99%	Alberta Envirofuels Alliance Pipeline Ltd. -	<1%	<1%	20	Petro ...
G10008	CCS Candidate	94%	AB Pipeline System	6%	<1%	20	Pipeline ...
G10009	CCS Candidate	85%	Alberta-Pacific Forest Industries Inc.	3%	12%	20	Pulp ...
G10015	Potential CU Candidate	37%	ATCO Pipelines - Transmission System	62%	1%	20	Pipeline ...
G10017	CCS Candidate	92%	Balzac Gas Plant	8%	<1%	8	Oil & Gas ...
G10018	CCS Candidate	99%	Battle River Generating Station	<0.1%	<1%	20	Fossil Power ...
G10023	CCS Candidate	87%	Bonnie Glen Gas Plant	12%	1%	5	Oil & Gas ...
G10029	Potential CU Candidate	71%	Brazeau Gas Plant	28%	1%	20	Oil & Gas ...

```
import numpy as np
above["log_emissions"] =
np.log1p(above["avg_annual_emissions"])
```






```
from sklearn.preprocessing
        import LabelEncoder

le = LabelEncoder()
above["naics_sector_encoded"] =
le.fit_transform(above["naics_code"])
```

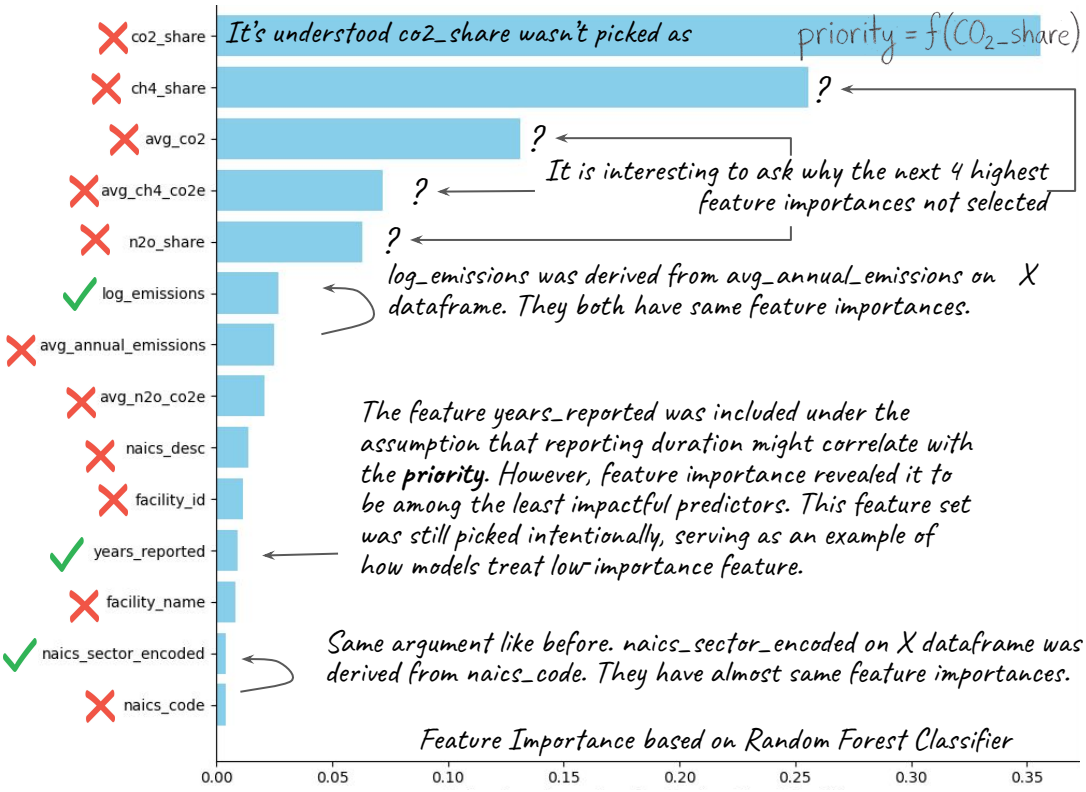
log_emissions	emission_band	avg_annual_emissions	naics_sector_encoded	avg_co2	avg_ch4_co2e	avg_n2o_co2e	naics_code
12.572828	Above	288607.9	24	267942	18222	2443	486210
13.961062	Above	1156675.75	19	1085053	61057	10398	325313
12.607295	Above	298728.85	14	296631	857	1202	325110
13.237733	Above	561143.1	24	524797	32117	3482	486210
11.729898	Above	124230	10	106563	3293	14332	322112
12.273732	Above	213999.05	24	79927	129179	2789	486210
12.244588	Above	207852.25	1	190415	15988	1233	211113
15.065295	Above	3489591.25	7	3475388	1075	13128	221112
12.556337	Above	283887.6	1	247919	32823	3146	211113
11.885144	Above	145094	0	102310	40838	1947	211110

Training Data Development : X and y

X feature matrix that contains 3 features, 150 observations & y target series (150 corresponding labels) were developed by transforming "above" dataframe (16 × 150)

```
X = above[["log_emissions", "naics_sector_encoded", "years_reported"]]
y = above["priority"]
```

Herewith probable arguments why features taken/not taken as part of feature engineering



Aspect	Decision Tree Classifier	Random Forest Classifier
Interpretability	⭐⭐ Very high — easy to visualize and explain; each split is human-readable.	⭐ Low — many trees averaged together; individual trees are interpretable, ensemble is not.
Overfitting Likelihood	High — single trees often memorize training data, especially when deep.	Low — ensemble averaging reduces variance and prevents overfitting.
Feature Importance Stability	Low stability — importance can change drastically with small data variations.	High stability — importance is averaged across many trees, making it more reliable.

```
#FEATURE IMPORTANCE
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt

# Separate features and target
X = above.drop(columns=["priority","emission_band"])
y = above["priority"]

# Encode categorical columns
cat_cols = X.select_dtypes(include=["object"]).columns
encoders = {}

for col in cat_cols:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col].astype(str))
    encoders[col] = le

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    max_depth=None
)

model.fit(X_train, y_train)

importances = model.feature_importances_
feature_names = X.columns

# Create a sorted dataframe
fi_df = pd.DataFrame({
    "feature": feature_names,
    "importance": importances
}).sort_values("importance", ascending=True)

plt.figure(figsize=(10, 8))
plt.barh(fi_df["feature"], fi_df["importance"], color="skyblue")
plt.xlabel("Feature Importance based on Random Forest Classifier")
plt.tight_layout()
plt.show()
```

emission_band was dropped as all value in above dataframe are equal to Above

Random Forest taken for better feature importance stability although less interpretable (See the table on the left !)

Sample Python Code to Visualize Feature Importance

Reflection on Data and Modeling

As I experimented with the model, what settings or features would I consider changing to try to get better results?

When experimenting with the model, there are several settings and features I would consider adjusting to improve performance. A single Decision Tree on its own is rarely the strongest approach because it tends to overfit and is highly sensitive to small changes in the data. An ensemble method such as Random Forest generally performs better — it reduces variance, stabilizes predictions, and captures more complex interactions, even though it becomes less interpretable.

Beyond the choice of algorithm, I would also revisit the feature set. Rather than selecting features based on intuition or what seems interesting, I would prioritize features with high feature importance or strong domain relevance. More years_reported does not automatically improve the model; for example, a long reporting history may simply reflect a facility that is shutting down or transitioning, which could mislead the classifier. Feature enrichment should be driven by what actually improves predictive power.

It's also important to recognize that determining whether an industrial site is suitable for CCS or CU is a multidisciplinary problem. A mechanical engineer's perspective — grounded in process behavior, emissions chemistry, and equipment constraints — differs significantly from a software engineer's perspective, which focuses on statistical patterns and model performance. The GHGRP dataset is useful for academic exploration, but in real applications the limiting factor is often data availability, not model complexity. Industrial emissions data can be sensitive, difficult to obtain, and inconsistent across sectors.

The diversity of industries in Alberta adds another layer of complexity. Some modern facilities are designed from the outset with carbon management in mind, which fundamentally changes their emissions profile and suitability for CCS or CU. Examples include ENGIE and Equinor's Blue Hydrogen project in Ghent, Belgium, and Imperial Oil's Renewable Diesel facility in Strathcona near Edmonton. These cases illustrate that suitability is not just a function of historical emissions — it is tied to long-term design philosophy, fuel strategy, and technology choices.





Did I discover anything messy, missing, or otherwise problematic about the data?
Yes – several issues appeared during the data transformation, mainly around how thresholds were defined. The cutoff that excludes facilities with average annual emissions below 100,000 tonnes is overly reductive and risks misclassifying sites. It seems driven more by financial or political prioritization of high-emitting facilities than by a full assessment of CCS/CU technical feasibility. Emissions volume alone doesn't indicate whether a site can realistically adopt carbon capture.

The same problem shows up in the above dataframe, where priority is based solely on a CO_2-share threshold of 0.85. While simple, a CO_2-only rule ignores how industrial emissions evolve. Gas-powered data centers, for example, may lower CO_2 as they shift to hydrogen or hybrid fuels, but often increase NOx due to hotter combustion, and SOx can persist if gas isn't fully desulfurized. Their emissions profile becomes more complex, not cleaner.

This means a site labeled "high-priority" or "CCS-ready" today may not remain so – especially as hyperscale data centers like Meta's planned Alberta facility expand and gas-turbine systems with higher exhaust temperatures become more common. The current labeling approach risks oversimplifying facilities whose technical realities are anything but simple.

If so, how did I fix it so my model could use the information?
I would make the model usable by moving away from rigid thresholds and applying the broader systems-level framework (pages 16–17) and management approach (pages 26–27). Instead of forcing simple rules, I focused on the technical, financial, and operational inputs the model actually needs. The key step was building the surrounding data ecosystem, how to secure sponsors for funding, and partnerships so CCS/CU datasets can be collected and standardized. With those inputs, the model becomes interpretable, scalable, and better aligned with real CCS/CU decision-making.

If I had to explain how this model makes its 'decisions' in simple terms, how would I describe it?
A decision tree makes its decision by asking a series of simple yes/no questions—about emissions, reporting history, industry type, and other question – and then following the answers down a branching path until it reaches the most likely classification.

Introduction to Decision Tree Classifier

Decision Tree Classifier is one of the clearest, most intuitive ways to explain how a machine learning model makes decisions. It works by breaking a complex problem into a series of simple, human-readable questions, arranged in a branching structure that looks and feels like a flowchart.

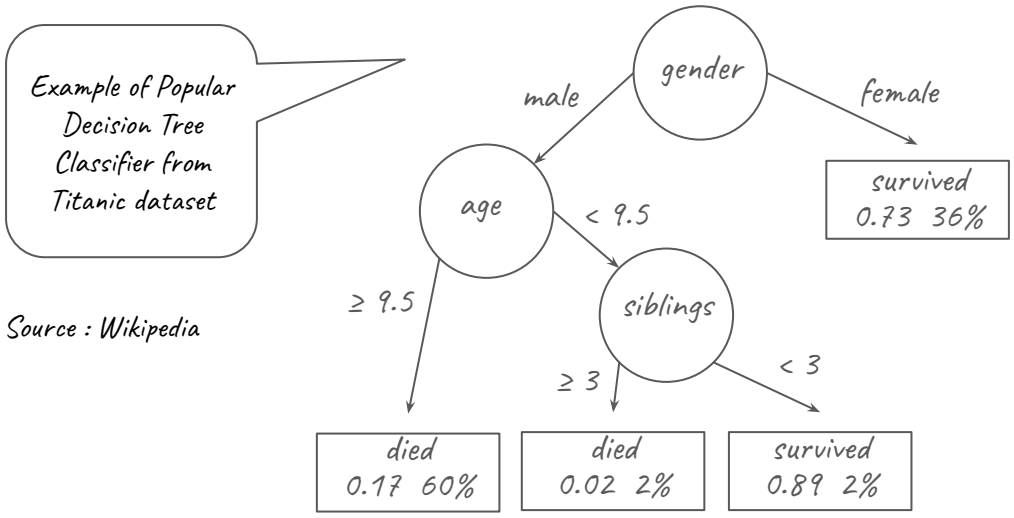
A Decision Tree Classifier is a machine learning model that predicts a category—such as "approve or deny," "low risk or high risk," or "species A or species B"—by following a sequence of simple yes/no questions. Each question checks one feature of the data (like age, income, emissions level, or temperature), and each answer leads to another question, until the model reaches a final decision at a "leaf" of the tree.

It's called a tree because the model branches out from a single starting point into multiple possible paths, just like the branches of a real tree.

Why people like decision trees ?

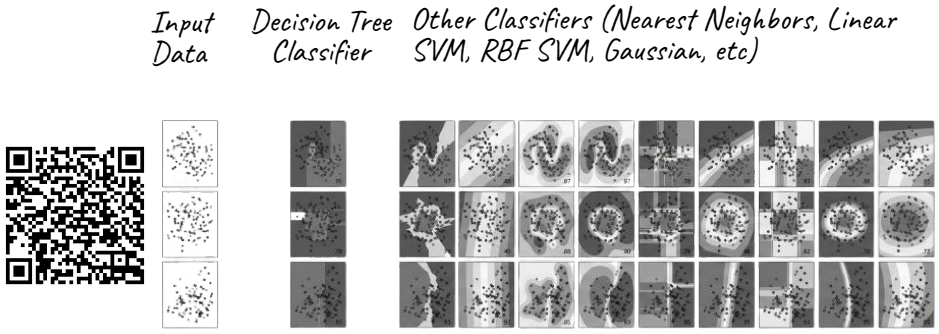
- They are easy to understand: I can trace exactly how the model reached a decision.
- They are transparent: every split is a clear rule.
- They are visual: the entire model can be drawn as a diagram.
- They are practical: they work well with both numeric and categorical data.

This makes decision trees one of the most accessible machine learning methods for the public, policymakers, and industry stakeholders who need clarity—not just accuracy.

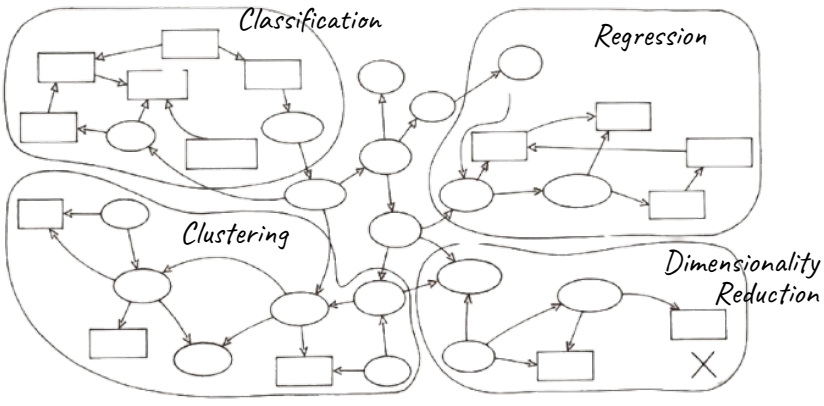


Source : Wikipedia

Within the broader scope of machine learning methods, classification represents just one analytical approach. For applications such as CCS or CU classification, a Decision Tree Classifier is one of several viable algorithms. Other commonly used classifiers include Nearest Neighbors, Linear and RBF Support Vector Machines, Gaussian Process models, Random Forests, Neural Networks, AdaBoost, Naive Bayes, and Quadratic Discriminant Analysis, each providing distinct modeling advantages depending on the characteristics of the dataset.



In a broader model selection discussion, choosing an alternative algorithm to improve accuracy often depends more on sample size and data type than on interpretability. A Decision Tree is flexible enough to handle both small and large datasets, as well as numeric and categorical features, which allows it to operate across multiple tasks—classification, regression, and even feature_selection. Because of this versatility, decision trees don't fit neatly into a single category within a model selection roadmap.



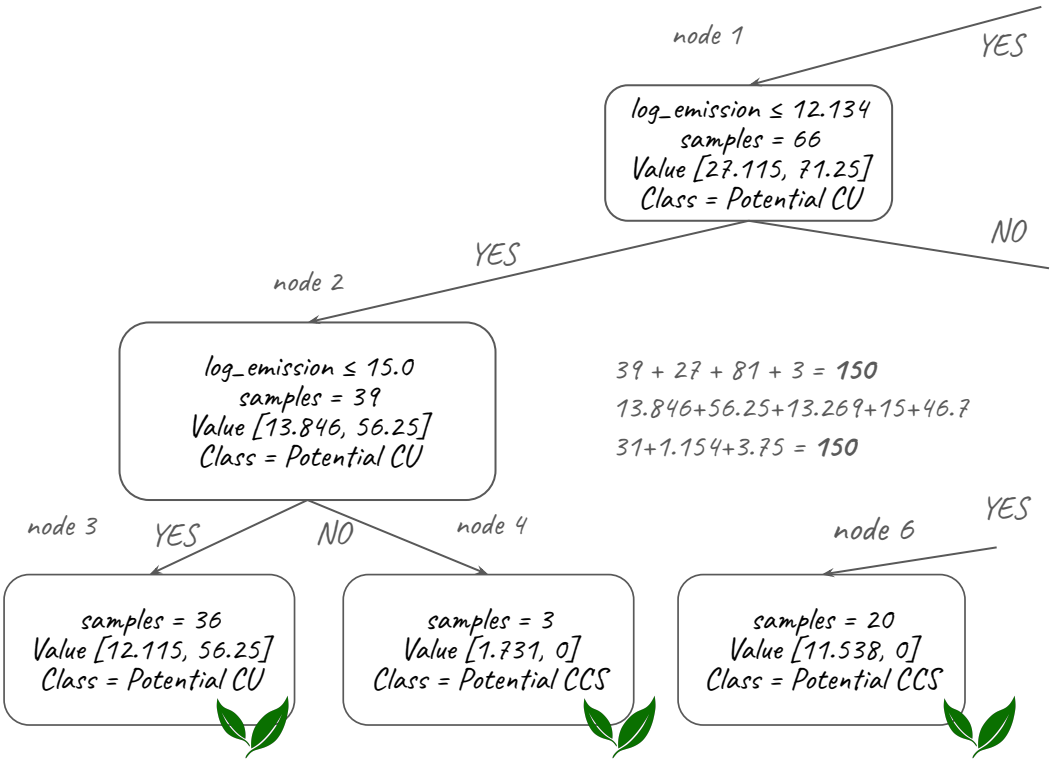
Dimensionality Reduction is reducing the number of random variables under consideration by obtaining a set of principle variables (or components). Principle components are set of values of linearly uncorrelated variables (Paper 2020)

Interpreting a Simple Decision Tree Classifier

While artificial intelligence is often perceived as a "black box"—a highly complex, mysterious digital brain that outputs decisions without explanation—not all machine learning models lack transparency. The Decision Tree Classifier stands out as one of the most intuitive and interpretable models available.

To demonstrate this ease of interpretation, we utilize the dataset detailed on pages 38 and 50, which consists of **150** total samples (130 classified as Potential CCS and 20 classified as Potential CU).

- The decision tree below was developed with a maximum depth of three (max_depth = 3). Its inner workings are remarkably straightforward:
- Sample Consistency: At any given depth of the tree, as well as across the final layer of leaf nodes, the total number of samples consistently sums to **150**.
- Class Determination (The value attribute): The value shown in each node represents the weighted distribution of the samples at that stage (which also sums to **150** across each level).

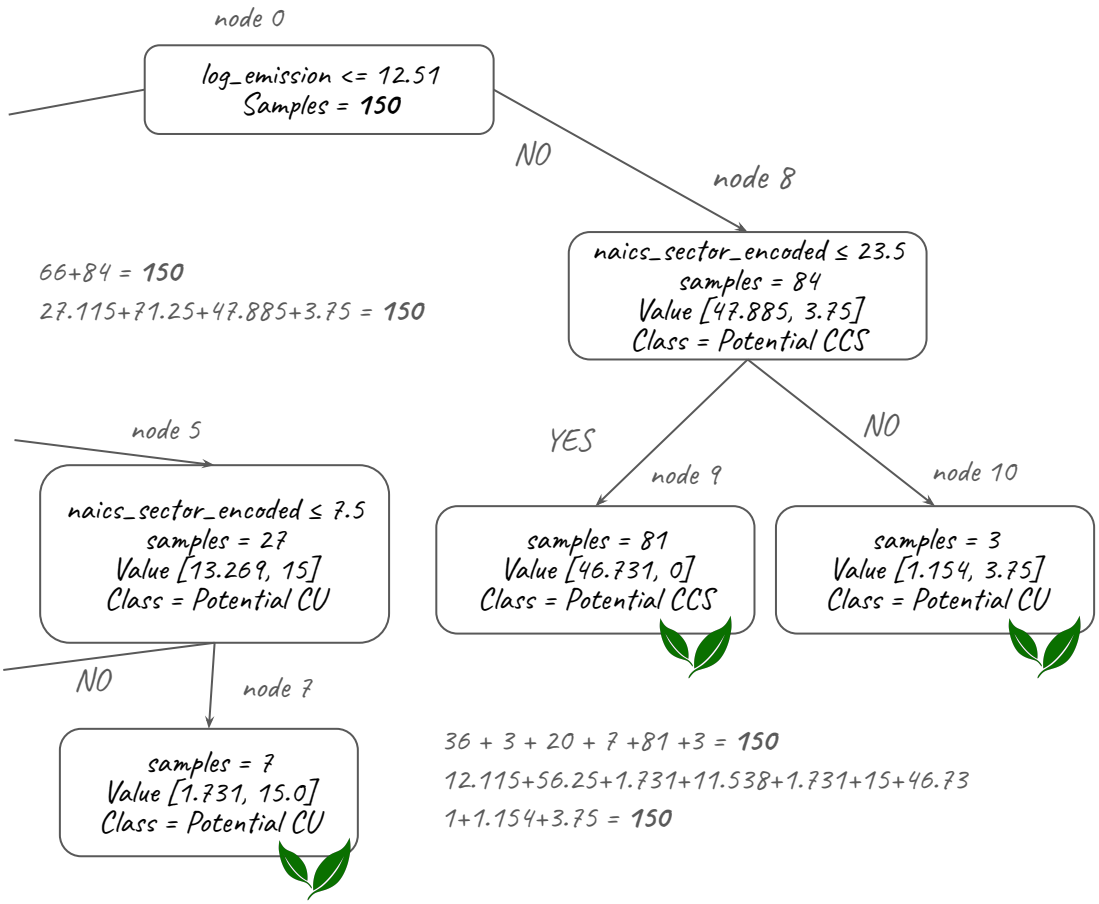


If the first element in the value array is larger than the second, the node's assigned class is Potential CCS.

If the second element is larger, the node's assigned class is Potential CU.

Key Drivers: As this visual model demonstrates, the entire complex decision-making process is ultimately driven by just two key variables: log_emission and naics_sector_encoded (in another version of decision tree with improved accuracy, years_reported also part of the key drivers).

Each node number shown below corresponds to the indexing in the accompanying table (see page 46), which was generated at the same time as the model. Every non-leaf node includes a clear Yes/No branching criterion, making the structure of the model straightforward to interpret. Leaf nodes, by contrast, simply terminate the path without further splits.



Interpreting a Scalable Decision Tree Classifier

Pairing a visual Decision Tree Classifier chart with a Decision Tree Classifier table (often generated as a text-based ruleset or a structured DataFrame) is incredibly powerful.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
tree1 = tree.tree_
tree_table = pd.DataFrame({
    "node_id": range(tree1.node_count),
    "feature": [tree1.feature[i] for i in range(tree1.node_count)],
    "threshold": [tree1.threshold[i] for i in range(tree1.node_count)],
    "samples": tree1.n_node_samples,
    "value": tree1.value.tolist(),
    "weighted_n_node_samples": tree1.weighted_n_node_samples,
})
tree_table["value"] = tree_table.apply( lambda row: [round(x*100*
row["weighted_n_node_samples"],2)/100 for x in row["value"][0]], axis=1)
feature_names = ["log_emissions", "naics_sector_encoded", "years_reported"]
tree_table["feature"] = tree_table["feature"].apply( lambda x:
        feature_names[x] if x != -2 else "leaf"
)
tree_table
```

Refer to page 52 for model **tree**

node_id	feature	threshold	samples	value	weighted_n_node_samples
0	log_emissions	12.509931	150	[75.0, 75.0]	150
1	log_emissions	12.133864	66	[27.1154, 71.25]	98.365385
2	naics_sector_encoded	15	39	[13.8462, 56.25]	70.096154
3	leaf	-2	36	[12.1154, 56.25]	68.365385
4	leaf	-2	3	[1.7308001, 0.0]	1.730769
5	naics_sector_encoded	7.5	27	[13.2692001, 15.0]	28.269231
6	leaf	-2	20	[11.53849999, 0.0]	11.538462
7	leaf	-2	7	[1.73080001, 15.0]	16.730769
8	naics_sector_encoded	23.5	84	[47.8846, 3.75]	51.634615
9	leaf	-2	81	[46.7308, 0.0]	46.730769
10	leaf	-2	3	[1.1538, 3.75]	4.903846

Feature	The Visual Chart	The Tabular Ruleset (Decision Tree Table)
Best Used For	Stakeholder presentations, quick visual debugging, and explaining the "most important" top-level splits to non-technical audiences.	Production deployment, automated model auditing, programmatic querying, and version control.
MLOps & Production Scalability	Extremely Poor. Image-based files cannot be easily parsed by deployment pipelines, version-controlled for logical changes, or run in low-latency environments.	Excellent. Can be exported to .csv, .json, or .sql, allowing it to run instantly as native code on any database, server, or edge device without needing ML libraries.
Version Control & CI/CD	Manual/Visual. Requires a human to manually compare two image files to spot differences in splits or thresholds.	Fully Automated. Easily tracked in Git. I can run a git diff to see exactly which decision boundaries or sample distributions changed after a model retrain.
Automated Testing & Guardrails	Not Possible. Automation scripts cannot inspect a JPEG to ensure it complies with safety rules (e.g., checking for bias or restricted features).	Seamless. MLOps pipelines can run automated unit tests directly on the table to block deployment if a rule violates compliance guidelines.
Cognitive Load	Low (for small trees). Highly intuitive to follow a single branch with my eyes.	Medium to High. Requires reading structured rows, but remains highly searchable and filterable.
Data Extraction	Manual. I have to manually transcribe the threshold values and node statistics from the visual boxes.	Programmatic. Directly queryable via code (e.g., Pandas, SQL) to instantly isolate high-confidence or weak-performing rules.

Classification with Original Dataset

Beyond class distribution, there are several critical issues I should anticipate, and almost all of them are a fundamental part of exploratory data analysis (EDA).

1. Things to Anticipate (And How to Handle Them)

   A. The "Accuracy Paradox" (Metric Decay)

   - What to anticipate: Standard accuracy is a lying metric when data is imbalanced.
   - The fix: I must look at other metrics.
     - Recall (Sensitivity): Out of all the real "Potential CU" cases, how many did the model actually find?
     - Precision: When the model predicts "Potential CU", how often is it actually correct?
     - F1-Score: The harmonic mean of Precision and Recall. This is my target metric for imbalanced data.

   B. Severe Tree Bias (Inadequate Splits)

   - What to anticipate: Because the algorithm tries to maximize "purity" (Gini impurity or Entropy), a standard Decision Tree might simply ignore the minority class entirely if a split doesn't yield a massive mathematical benefit. It will build a tree that predicts the majority class everywhere.
   - The fix: Use class weights during model training. In scikit-learn's DecisionTreeClassifier, I can set class_weight='balanced'. This tells the tree to penalize mistakes on the minority class ("Potential CU") much more heavily, forcing it to find splits that accommodate them.

   C. Evaluation Overoptimism (Data Leakage in Validation)

   - What to anticipate: If I use a standard random train/test split, I might accidentally end up with zero "Potential CU" samples in my test set, making it impossible to evaluate my model.
   - The fix: Always use Stratified Splitting (e.g., StratifiedKFold or train_test_split(..., stratify=y)). This guarantees that both my training and testing sets maintain the exact 130:20 ratio.

2. EDA is not about histograms; it is about identifying risks in data before model sees it.
When dealing with this specific dataset, EDA should include the following steps:

Check for Feature Correlation & Separation (Scatter Plots / Box Plots)

- What to do: Plot my key features (log_emission vs. naics_sector_encoded) on a scatter plot, coloring the points by their class (CCS vs. CU).
- The EDA Goal: Check if the 20 "Potential CU" samples are tightly clustered together or scattered randomly among the CCS samples. If they are clustered, even a simple tree will easily find them. If they are completely mixed in, a simple tree will struggle heavily.

Look for High Cardinality in Encoded Features

- What to do: Check how many unique categories exist in naics_sector_encoded.
- The EDA Goal: Decision trees love splitting on high-cardinality features (features with many unique values). If naics_sector has 50 different codes, the tree might overfit by creating hyper-specific branches just to isolate single minority samples.

Analyze Outliers in the Minority Class

- What to do: Look closely at the 20 "Potential CU" samples. Are any of them extreme outliers in terms of log_emission?
- The EDA Goal: Because the minority class is so small (only 20 samples), a single outlier can wildly distort how the decision tree decides to split that feature. I need to know if those 20 samples are representative or highly noisy.

The Perfect EDA-to-Modeling Roadmap

[EDA Phase]
- 1. Class Distribution Check (Identify the 130:20 imbalance)
- 2. Feature Separation Analysis (Box plots of log_emission per class)
- 3. Outlier Detection (Ensure the 20 CU samples aren't anomalies)

[Modeling Phase]
- 4. Stratified Split (Keep the 130:20 ratio in train and test)
- 5. Train Tree with class_weight='balanced'
- 6. Evaluate using Precision, Recall, and F1-Score (Ignore global Accuracy)

Confusion Matrix with Original Dataset

```
from sklearn.model_selection import StratifiedKFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import confusion_matrix
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

cv   = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
tree = DecisionTreeClassifier(
    max_depth        = 3,
    min_samples_leaf  = 2,
    min_samples_split = 6,
    class_weight      = "balanced",
    random_state      = 42
)

# Get cross-validated predictions for evaluation on full dataset
y_pred_cv = cross_val_predict(tree, X, y, cv=cv)


# Get Confusion Matrix
cm = confusion_matrix(y, y_pred_cv, labels=["CCS Candidate", "Potential CU Candidate"])

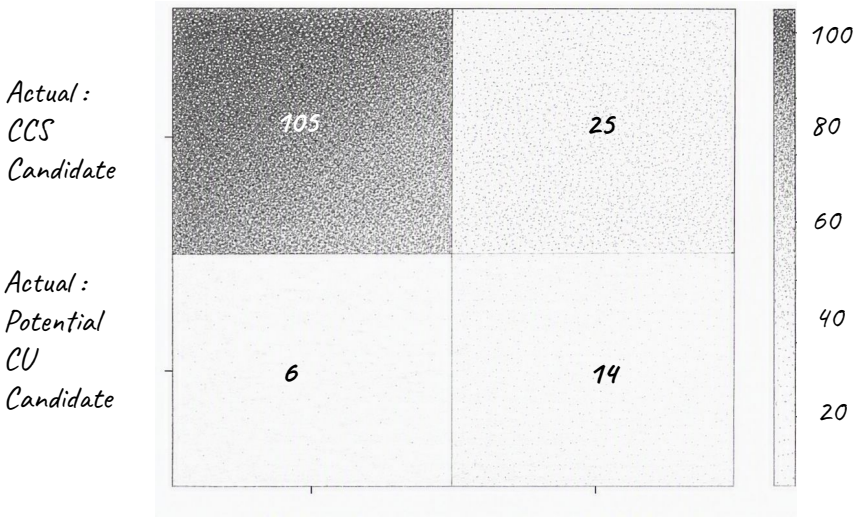
labels = ["CCS Candidate", "Potential CU Candidate"]

# Convert numpy confusion matrix to a labeled DataFrame
df_cm = pd.DataFrame(cm, index=[f"Actual: {l}" for l in labels],
                         columns=[f"Predicted: {l}" for l in labels])

print(df_cm)

# Heatmap visualization
plt.figure(figsize=(6,4))
sns.heatmap(df_cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```

Confusion Matrix



What does it mean?
@ CCS Candidate Prediction Performance
Actual CCS Candidates: 130
Correctly Predicted: 105
Not Predicted Correctly: 25
The model captured 80.8% of actual CCS candidates, with 19.2% missed.

@ Potential CU Candidate Prediction Performance
Actual Potential CU Candidates: 20
Correctly Predicted: 14
Not Predicted Correctly: 6
The model identified 70% of actual Potential CU candidates, while 30% were not captured.

Original Classification Report, Label Distribution

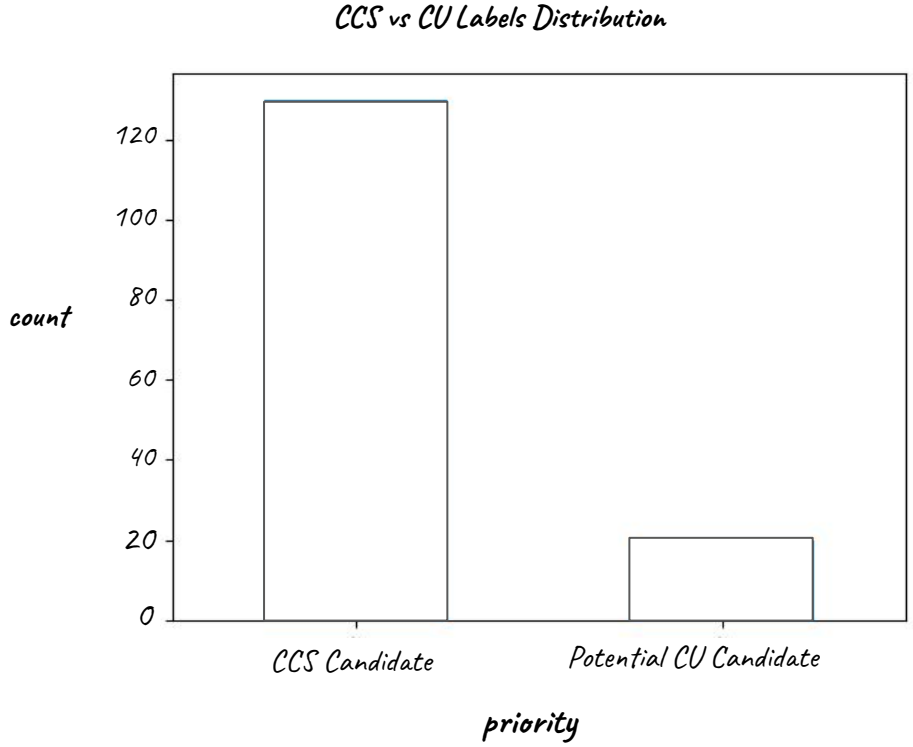
```
from sklearn.metrics import classification_report

# Print a comprehensive classification report
print("=== Classification Report ===")
print(classification_report(y, y_pred_cv, labels=["CCS Candidate", "CU
Candidate"]))
```

```
=== Classification Report ===
                precision    recall  f1-score   support
CCS Candidate       0.95      0.81      0.87       130
CU Candidate        0.36      0.70      0.47        20
accuracy                              0.79       150
macro avg           0.65      0.75      0.67       150
weighted avg        0.87      0.79      0.82       150
```

```
import matplotlib.pyplot as plt

y.value_counts().plot(kind='bar')
plt.title("CCS vs CU Labels Distribution")
plt.ylabel("Count")
plt.show()
```



Imagine a machine learning model of a spam filter protecting email inbox:

Precision as "Quality Control"
When the filter flags an email as spam, how often is it actually right?
High Precision means no mistake. My important emails won't end up in junk.

Recall (Sensitivity) as "The Safety Net"
Out of all actual spam emails sent, what percentage did filter catch?
High Recall means a clean inbox. Very little junk slips through to distract me.

F1-Score as "The Balancer"
A percentage value how well filter balances Precision and Recall.
High F1-score means I successfully block spam without blocking actual emails.

Herewith the summary of what precision, recall and F-score would do

Metric	Simple Question	The Value
Precision	When we flag a target, is it a false alarm?	Saves time by preventing useless follow-ups.
Recall	Did we miss any actual targets?	Saves lost of opportunity by ensuring nothing slips through.
F1-Score	How healthy is our overall compromise?	Optimizes systems by finding the perfect operational sweet spot.

Decision Tree Chart with Original Dataset

In Decision Tree, Gini value (Gini Impurity) measures how mixed up / messy the data is.

Think of it as a "Messiness Score" using a bowl of candy (Chocolates vs. Gummies):

Gini = 0 (Perfect Purity): The bowl contains only Chocolates. There is zero mix-up.

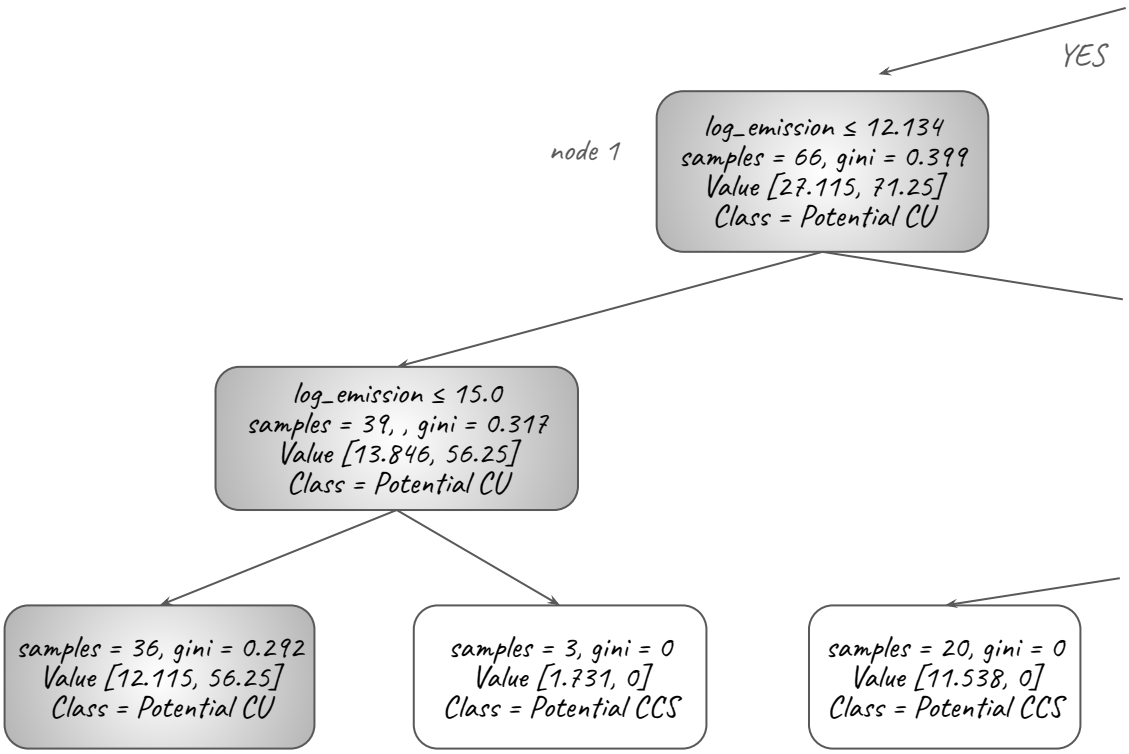
Gini = 0.5 (Maximum Mess): The bowl has perfect 50/50 split. It's complete mixed up.

How the Tree Uses It:
The model acts as an organizer. It splits my data into new groups, always choosing the questions that lower the Gini score as close to 0 as possible.

$$Gini = 1 - \sum_{i=1}^{n} pi^2$$

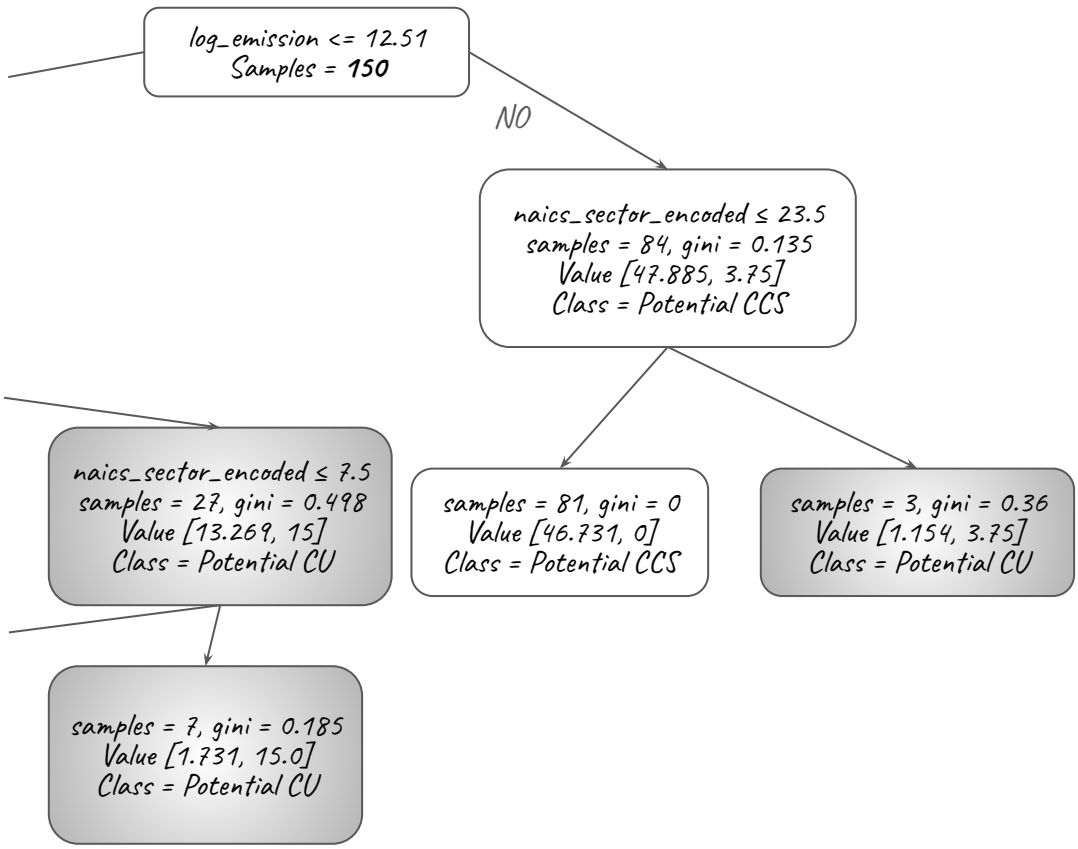
Example of a gini calculation for node 1

$$gini = 1 - \left(\left(\frac{27.115}{27.115 + 71.25}\right)^2 + \left(\frac{71.25}{27.115 + 71.25}\right)^2\right) = 0.339$$



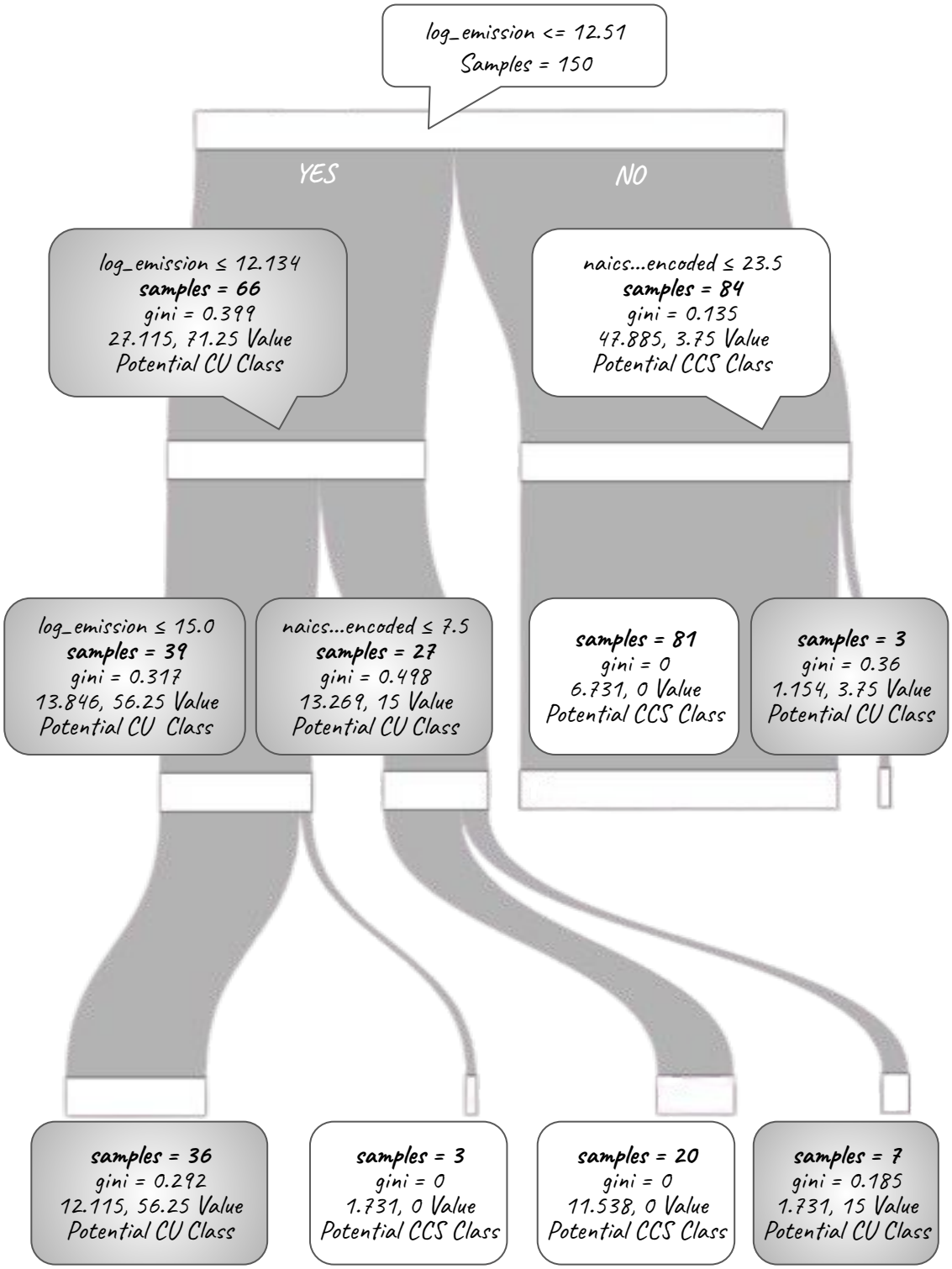
```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

tree.fit(X, y)
plt.figure(figsize=(18, 8))
plot_tree(
    tree,
    feature_names = list(X.columns),
    class_names   = list(tree.classes_),
    filled        = True,
    rounded       = True,
    fontsize      = 7
)
plt.tight_layout()
plt.show()
```



Decision Tree + Sankey Chart with Original Dataset

To make interpreting the results easier, a Sankey diagram like the one below provides a clearer view of the flow.



```
import plotly.graph_objects as go

# Define flows (source → target → value)
sources = [     0, 0,       # 150 → 66, 84
                  1, 1,       # 66 → 39, 27
                  2, 2,       # 84 → 81, 3
                  3, 3,       # 39 → 36, 3
                  4, 4        # 27 → 20, 7 ]
targets = [       1, 2,
                  3, 4,
                  5, 6,
                  7, 8,
                  9, 10    ]
values =  [       66, 84,
                  39, 27,
                  81, 3,
                  36, 3,
                  20, 7    ]

fig = go.Figure(data=[go.Sankey(
   node=dict(
      pad=20, thickness=20,
      color=["rgba(0,0,0,0)"] * 11  # fully transparent nodes
   ),
   link=dict(
      source=sources, target=targets, value=values,
      color="rgba(0,0,0,0.3)"  # faint neutral flow color
   )
)])

fig.update_layout(title_text="", font_size=1)
fig.show()
```

When introducing a Sankey chart to help interpret the flow of decisions in a classifier, I would like to relate it to my past experience in supply chain operations. At Trias Sentosa, I worked extensively with material flow mapping, tracing how jumbo rolls of plastic packaging moved through each stage of the process—slitting into smaller widths, metallizing when required, and finally distributing the finished products to customers. That same principles helped me map a decision tree routes data through different conditions to reach a final classification.

Delivery Improvement

Boosting a Decision Tree goes beyond class-weighting or resampling. The main gains come from better features, tighter constraints, and smarter structure.

1. Tree Pruning
   Limit overgrowth by tuning:

   max_depth – prevents overly specific rules.

   min_samples_split – requires more data before splitting.

   min_samples_leaf – smooths predictions in sparse regions.

2. Feature Engineering
   Trees miss complex math. Add domain-driven features (e.g., emission_density = emission / area) to expose relationships directly.

3. High-Cardinality Handling
   Many unique categories cause overfitting. Group rare values or apply target encoding to simplify splits.

4. Continuous Binning
   Noisy or extreme continuous variables create unstable boundaries. Convert them into meaningful bins (Low/Medium/High).

5. Cost-Complexity Pruning
   Use ccp_alpha to prune branches that add little impurity reduction after growing a full tree.

6. Remove Redundant Features
   Highly correlated variables dilute importance. Drop duplicates to force cleaner, more decisive splits.

7. Feature Selection
   Too many noisy features distract the tree. Use mutual information, chi-square, or preliminary importances to keep only useful columns.

8. Monotonic Constraints
   If domain logic dictates a consistent direction (e.g., higher emissions → higher CCS likelihood), enforce monotonicity to avoid erratic rules.

9. Split Criterion Tuning
   Try gini vs entropy/log_loss; some datasets align better with one metric.

10. Upgrade to Ensembles
    If a single tree plateaus, move to Random Forests or Gradient Boosting/XGBoost for major gains in accuracy and stability.

These ten techniques are general remedies for improving a Decision Tree classifier. Luca and John (Massaron, Mueller 2026) may have a more general remedies for machine learning model in general which some overlaps with above. For now, we might focus on Tree Pruning using max_depth & upgrading to Ensemble Method such as Random Forest. As the model develops, it quickly becomes clear that imbalanced data requires its own treatment. The imbalance problem may lead to biased models that give the priority to the class that holds the majority of the population and perform poorly when it comes to accurately predicting the class that holds the minority of the population (Pulari, Murugesh, Vasudevan, Akhsay 2025). Balancing the classes (says the 11th technique on top of 10 techniques above) – through methods like Random Oversampling or SMOTE—often improves accuracy and reduces complexity the tree needs to separate minority patterns. Herewith is a common imbalance pattern:

Class	F1-score	Interpretation
Majority Class	High (0.95–1.00)	Model sees many examples → learns well
Minority Class	Lower (0.70–0.90)	Model sees few examples → struggles

Classification with Balanced Dataset

Let's say we pick ROS and Random Forest as our model, we could see the model performance and its decision tree chart on page 64 - 67.

Method	What it does	Type of new data	How samples are created	Risk / downside
Random Oversampling (ROS)	Balances classes by duplicating minority samples	No synthetic features (just copies)	Randomly resamples minority rows with replacement	Overfitting due to repeated identical samples
SMOTE	Balances classes by creating new minority samples	Yes — synthetic samples	Interpolates between minority neighbors to create new points	Can create unrealistic samples if minority space is noisy

In EDA, we put effort to remove duplicates. When ROS implemented we actually put the problem back. What could be wrong with Decision Tree Classifier if duplicate occurs?

Decision Trees split based on impurity reduction (Gini or Entropy). Duplicates simply reinforce the minority class patterns so the tree considers them during splitting.

They do not:

- confuse the split logic
- distort impurity calculations
- create invalid decision boundaries
- cause instability in the tree structure
- Trees treat duplicates as "more evidence" for a pattern.

The downside is overfitting — the tree may memorize the duplicated minority samples.
This is why ROS is often paired with Tree Pruning (max_depth), Ensemble Methods.

On page 64 - 65, I would bring three scenarios of random forest

- *First scenario (regular fonts) simply brings all balanced dataset of 260 rows*
  - *The model potentially show very accurate prediction given max_depth=9 and just one decision tree (n_estimators=1)*

  - *rf_model.fit(X_balanced, y_balanced)*
    accuracy_full = accuracy_score(y_balanced, rf_model.predict(X_balanced))

- *Second scenario (italic fonts) tries to split 20% test data from those 260 rows*
  - *The model potentially show less accurate prediction due to very small dataset (150 rows). I have*
    26 NAICS categories
    20 years_reported categories
    2 classes (CCS vs CU)
    This means the model must learn:
    A decision boundary across 26×20 combinations using only 150 samples.
    That is far too little data for any tree-based model to generalize well.
    This leads to: unstable splits, overfitting, weak generalization, high variance sensitivity to random_state, trees that memorize instead of learning patterns.

  - *rf_model.fit(X_train, y_train)*
    accuracy_full = accuracy_score(y_test, rf_model.predict(X_test))

- *Third scenario (**bold fonts**) attempted to use back original data of 150 rows.*
  - *Keeping the same random forest like above but using the original dataset would drop the accuracy due to imbalance data. Having the same max_depth, imbalance dataset with Random Forest may require n_estimators > 18*

  - *rf_model.fit(**X, y**)*
    *accuracy_full = accuracy_score(**y**, rf_model.predict(**X**))*

Label Distribution with Balanced Dataset

```
import pandas as pd
from sklearn.utils import resample

# -----------------------------
# 1. BALANCE THE DATASET
# -----------------------------

# Separate majority and minority classes
df_majority = above[above['priority'] == 'CCS Candidate']
df_minority = above[above['priority'] == 'Potential CU Candidate']

# Oversample minority class to match majority count
df_minority_oversampled = resample(
    df_minority,
    replace=True,               # sample with replacement
    n_samples=len(df_majority),   # match majority class count
    random_state=42
)

# Combine majority class with oversampled minority class
above_balanced = pd.concat([df_majority, df_minority_oversampled])

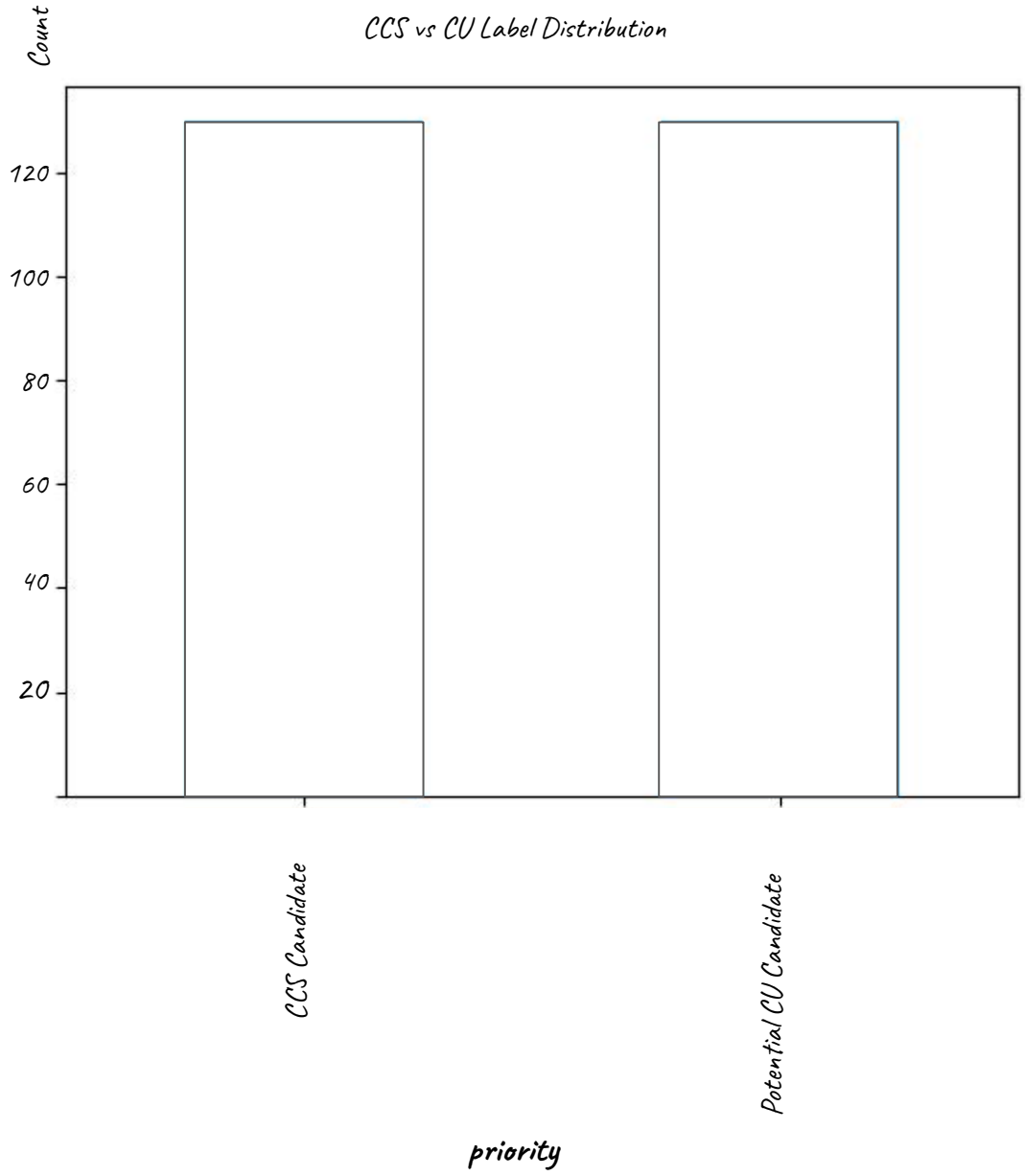
# Shuffle the resulting dataframe
above_balanced = above_balanced.sample(frac=1, random_state=42).reset_index(drop=True)

# -----------------------------
# 2. BUILD X_balanced AND y_balanced
# -----------------------------

X_balanced = above_balanced[["log_emissions", "naics_sector_encoded", "years_reported"]]
y_balanced = above_balanced["priority"]

#This makes imbalance obvious at a glance.
import matplotlib.pyplot as plt

y_balanced.value_counts().plot(kind='bar')
plt.title("CCS vs CU Labels Distribution")
plt.ylabel("Count")
plt.show()
```


CCS vs CU Label Distribution
Count
120
100
80
60
40
20
CCS Candidate
Potential CU Candidate
priority

Confusion Matrix with Balanced/Imbalanced Data

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

# 1. Prepare balanced dataset
X_balanced = above_balanced[["log_emissions", "naics_sector_encoded", "years_reported"]]
y_balanced = above_balanced["priority"]

# 2. Train Random Forest
rf_model = RandomForestClassifier(
    n_estimators=1, # 1 tree with balanced data, 19 or more with original data
    max_depth=9,    # minimum depth for 100% accuracy (same for original data)
    random_state=42
)
rf_model.fit(X_balanced, y_balanced)

# 3. Predict on ALL 260 balanced observations
y_pred_full = rf_model.predict(X_balanced)

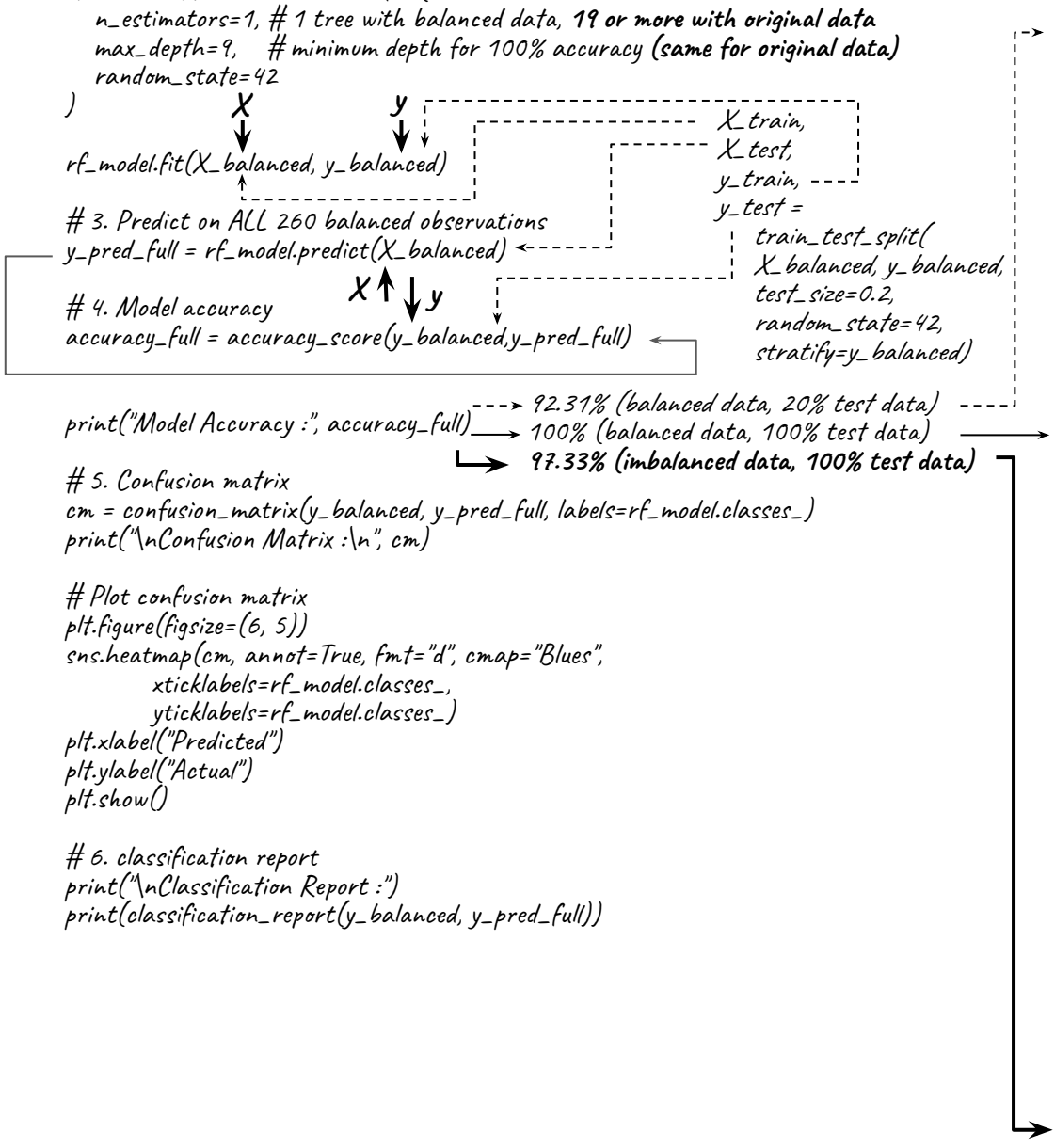
# 4. Model accuracy
accuracy_full = accuracy_score(y_balanced,y_pred_full)

print("Model Accuracy :", accuracy_full)

# 5. Confusion matrix
cm = confusion_matrix(y_balanced, y_pred_full, labels=rf_model.classes_)
print("\nConfusion Matrix :\n", cm)

# Plot confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=rf_model.classes_,
            yticklabels=rf_model.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

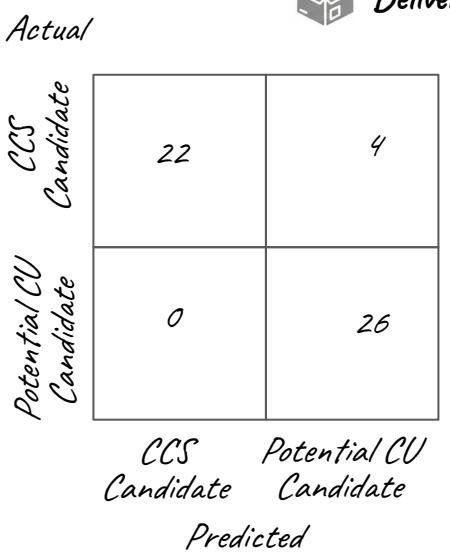
# 6. classification report
print("\nClassification Report :")
print(classification_report(y_balanced, y_pred_full))
```



Model Accuracy : 92.31%
Confusion Matrix :
[[22 4]
[0 26]]

Classification Report :

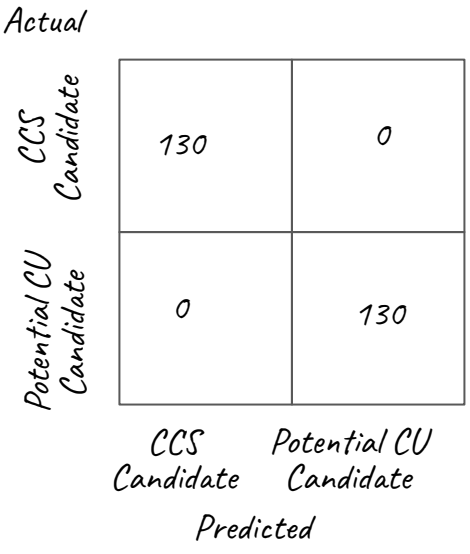
	precision	recall	f1-score	support
CCS Candidate	1.00	0.85	0.92	26
Potential CU Candidate	0.87	1.00	0.93	26
accuracy			0.92	52
macro avg	0.93	0.92	0.92	52
weighted avg	0.93	0.92	0.92	52



Model Accuracy : 100%
Confusion Matrix :
[[130 0]
[0 130]]

Classification Report :

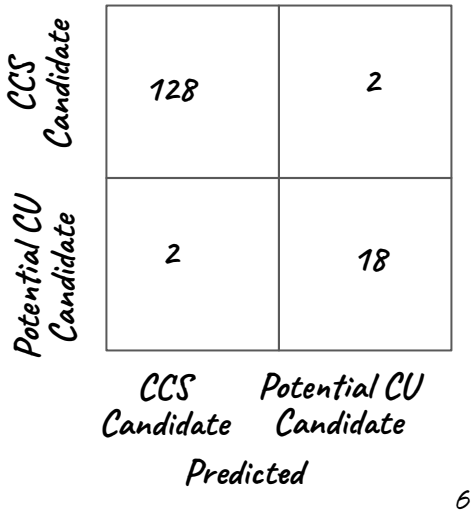
	precision	recall	f1-score	support
CCS Candidate	1.00	1.00	1.00	130
Potential CU Candidate	1.00	1.00	1.00	130
accuracy			1.00	260
macro avg	1.00	1.00	1.00	260
weighted avg	1.00	1.00	1.00	260



Model Accuracy : 97.33%
Confusion Matrix :
[[22 4]
[0 26]]

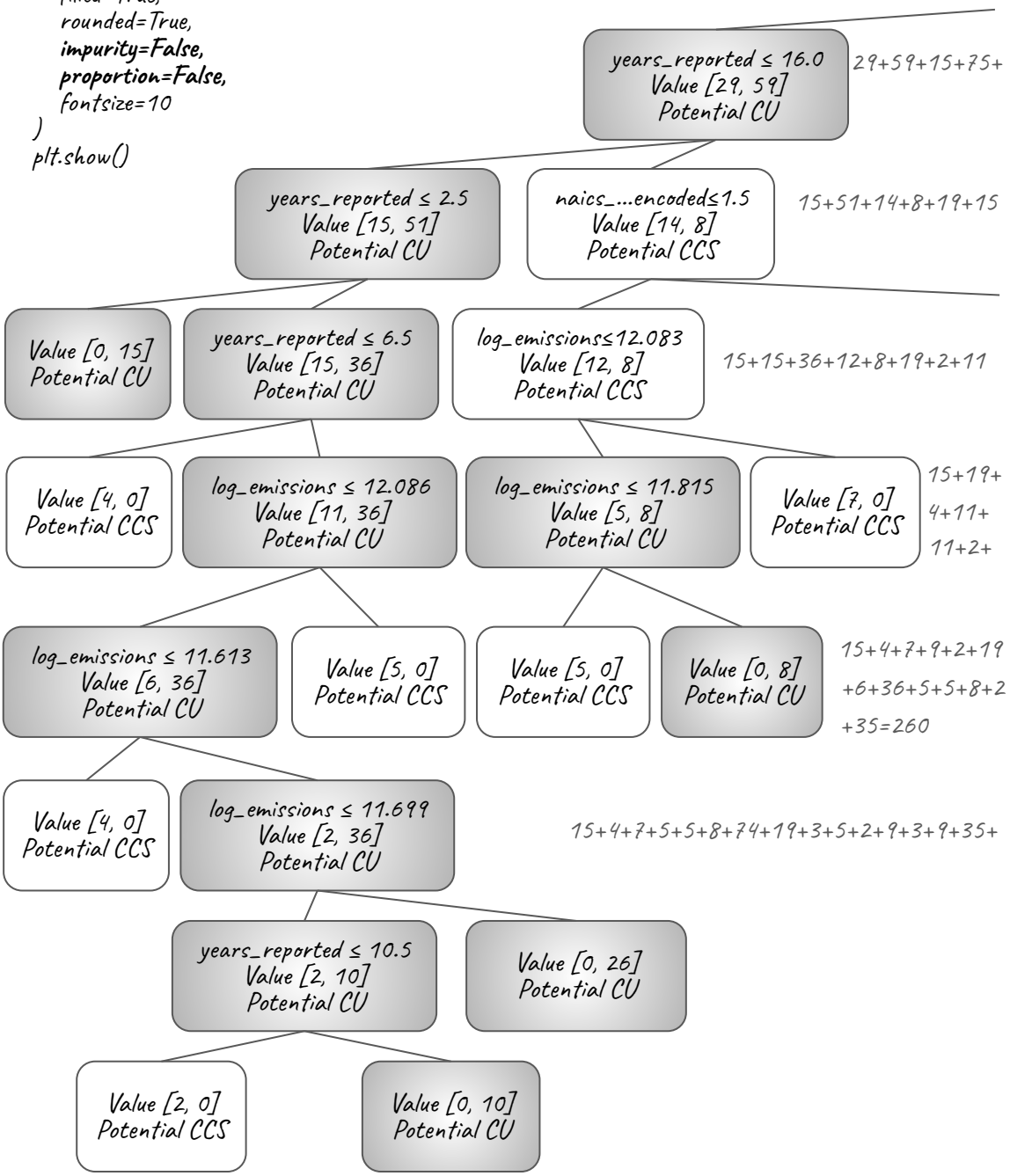
Classification Report :

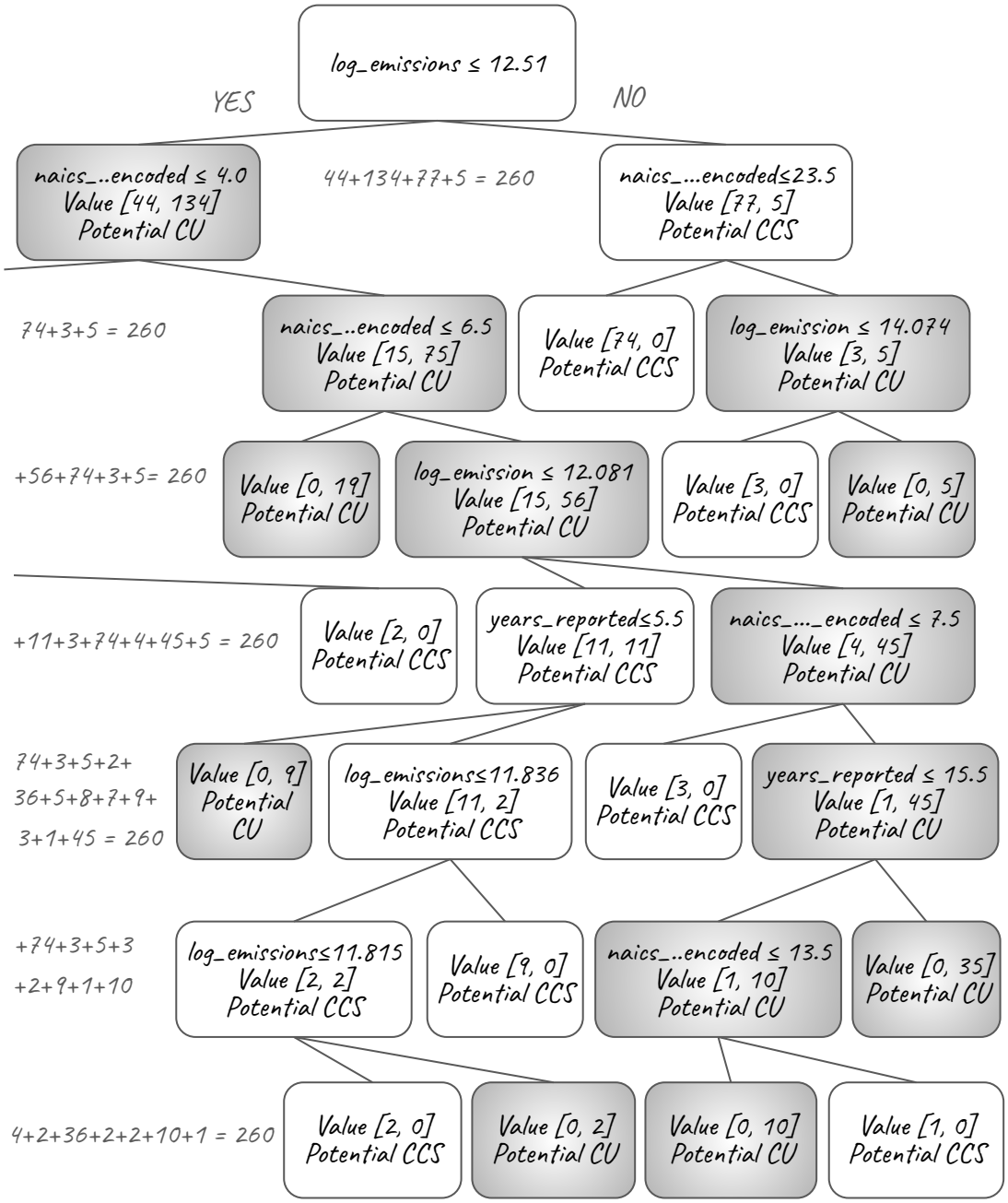
	precision	recall	f1-score	support
CCS Candidate	**0.98**	**0.98**	**0.98**	**130**
Potential CU Candidate	**0.90**	**0.90**	**0.90**	**20**
accuracy			**0.97**	**150**
macro avg	**0.94**	**0.94**	**0.94**	**150**
weighted avg	**0.97**	**0.97**	**0.97**	**150**



Decision Tree with Balanced Dataset

```
tree0 = rf_model.estimators_[0]
plt.figure(figsize=(22, 12))
plot_tree(
    tree0,
    feature_names=X_balanced.columns,
    class_names=rf_model.classes_,
    filled=True,
    rounded=True,
    impurity=False,
    proportion=False,
    fontsize=10
)
plt.show()
```





15+4+7+5+5+8+4+74+19+3+5+2+9+3+9+35+2+2+10+1+2+10+26 = 260

15+4+7+5+5+8+4+74+19+3+5+2+9+3+9+35+2+2+10+1+2+10+26 = 260

Reflection on Delivery

How do I know my model actually works?
My model was trained and tested on the exact same data (100% train = 100% test).
This means my evaluation is not a real measure of generalization.
The model is only proving it can memorize the training data.
When I split 20% for testing, the accuracy drops—not because the model is bad, but because the test set becomes too tiny to represent the true data distribution.

ID	log_emissions	naics...encoded	years_reported	priority
0	12.572828	24	20	CCS Candidate
1	13.961062	19	20	CCS Candidate
2	12.607295	14	20	CCS Candidate
3	13.237733	24	20	CCS Candidate
4	11.729898	10	20	CCS Candidate
5	12.273732	24	20	Potential CU Candidate
6	12.244588	1	8	CCS Candidate
7	15.065295	7	20	CCS Candidate
8	12.556337	1	5	CCS Candidate
9	11.885144	0	20	Potential CU Candidate
11	13.063652	7	20	CCS Candidate
...				
1118	11.561977	14	2	Potential CU Candidate
1119	12.965772	7	2	CCS Candidate
1167	11.682423	0	1	Potential CU Candidate

Assuming naics categories and years_reported are the relevant features (which were not the case), the 150 rows of observations are the signal of data starvation as I have 26 naics categories and 20 years_reported categories there are 26 x 20 combinations = 520 possible combinations while I only have 150 observation.

Was it as accurate as I hoped?
The model appears extremely accurate, but that's because it was evaluated on the same data it was trained on. With such a small dataset and weak features, the model is essentially memorizing the training examples. So while the accuracy looks perfect, it doesn't yet reflect true generalization. To get a realistic measure of accuracy, I would need either more observations or a cross-validation approach that tests the model on unseen data.

GCP Infer JSON predictions

[12.572828,24,20],	CCS Candidate
[13.961062,19,20],	CCS Candidate
[12.607295,14,20],	CCS Candidate
[13.237733,24,20],	CCS Candidate
[11.729898,10,20],	CCS Candidate
[12.273732,24,20],	Potential CU Candidate
[12.244588,1,8],	CCS Candidate
[15.065295,7,20],	CCS Candidate
[12.556337,1,5],	CCS Candidate
[11.885144,0,20],	Potential CU Candidate
[13.063652,7,20],	CCS Candidate
...	
[11.561977,14,2],	Potential CU Candidate
[12.965772,7,2],	CCS Candidate
[11.682423,0,1]	Potential CU Candidate

and if not, what is one thing I would change to make it better?
The accuracy drops when using a 20% test split because the dataset is too small for a stable evaluation. The most impactful improvement would be to increase the dataset size or add more meaningful features. Right now, only log_emission carries strong predictive signal, while naics_encoded and years_recorded contribute very little. More observations or richer features would reduce variance, improve CU detection, and allow the model to learn more reliable decision boundaries.

Scalability

Cloud Engineering, DevOps, and MLOps each strengthen a different layer of a modern platform — infrastructure, software delivery, and the machine-learning lifecycle. Together, they enable systems to grow in users, data volume, and functional complexity without degrading performance.

Cloud Engineering expands infrastructure capacity
Cloud engineering provides elastic, distributed foundation that everything else relies on.

- Cloud architecture supports horizontal and vertical expansion
- Autoscaling groups add compute resources automatically
- Load balancers distribute traffic efficiently
- Multi-region deployments increase global reach
- Infrastructure as a Code (IaaC) ensures reproducible environments
- Cloud engineering delivers the physical capability for systems to grow.

DevOps enhances delivery throughput
DevOps ensures software can be built, tested, and released quickly and reliably as demand increases.

- CI/CD pipelines increase build and deployment throughput
- Automation removes manual bottlenecks
- GitOps keeps environments consistent
- Observability improves incident response
- Container orchestration manages application runtime growth

DevOps provides the operational capability for teams and applications to evolve without slowing down.

MLOps strengthens data and model operations
MLOps ensures machine-learning systems remain reliable as datasets, models, and analytical workloads increase.

Feature stores improve data reuse
Model registries organize versioning
Automated retraining keeps models fresh
Drift monitoring maintains reliability
Distributed training supports compute-intensive workloads

MLOps provides the analytical capability for ML systems to grow without losing accuracy or stability.

How they work together?
Cloud Engineering supplies the infrastructure foundation
DevOps builds software delivery pipelines on that foundation
MLOps builds machine-learning workflows on top of DevOps processes

This layered architecture powers modern platforms such as industrial IoT, predictive maintenance, digital twins, and cloud-native enterprise systems.

Cloud Engineering

At later section, a simple deployment (with buckets, endpoints, and a model registry only) is suitable as the model, data is small, traffic is light, inference load is simple.
I need to move into networking, compute, and larger storage when growth forces me to — more data, more users, more latency pressure, more reliability requirements.

When buckets + endpoints + model registry are enough ?
This setup works perfectly when:

- my dataset is small (from few MBs to low GBs)
- my model is lightweight (e.g., DecisionTreeClassifier)
- only need occasional inference
- don't have strict latency requirements
- don't need distributed compute
- don't need high-throughput APIs

When I must go back to re-setup networking, customize compute, and bigger storage ?
I only escalate when the system's load, data, or performance requirements exceed what a simple endpoint can handle, such as

1. Data volume grows beyond what a bucket + simple model can handle : I need to move to BigQuery or Bigtable when:

   - my dataset becomes tens of GB → TB → PB
   - need SQL analytics at scale
   - need streaming ingestion
   - need low-latency reads for inference
   - need schema enforcement or partitioning

   Buckets are great for files, not for large analytical datasets.

2. As inference traffic increases, Compute Engine, GKE, Cloud Run needed to fulfill:

   - high-throughput inference
   - autoscaling
   - multiple replicas
   - GPU/TPU acceleration
   - low-latency responses
   - batch inference at scale

   Endpoints are good for small workloads. Compute/GKE is needed for heavy workloads.

3. When distributed training exist, larger compute is needed when:

- model grows beyond simple decision trees
- move to XGBoost, RandomForest, LightGBM, or deep learning
- need parallel training
- need GPU/TPU
- need hyperparameter tuning at scale

This is where Vertex AI Training, Google Kubernetes Engine (GKE), or Compute Engine becomes necessary.



Example of parallel jobs

4. When secure, reliable, production-grade networking needed, revisit networking when:

- VPC-SC for data protection is needed
- private endpoints is needed
- load balancing is needed
- traffic splitting is needed
- zero-trust access is needed
- multi-region failover is needed

Simple endpoints don't provide enterprise-grade networking.

5. When real-time/streaming data exists, Pub/Sub, Dataflow, or Dataproc needed due to:

- real-time inference
- streaming feature pipelines
- batch → streaming migration
- ETL/ELT at scale

Buckets cannot handle streaming.

In a simple scenario, emissions data is typically reported and analyzed annually as earlier mentioned thus I don't need real-time decision making. As the complex dataset become available, potential data could be a streaming one. Hoeffding Tree Classifier is a popular choice (Putatunda 2021) for Very Fast Decision Tree (VFDT) or Extremely Fast Decision Tree (EFDT) where synthetic data is crucial for a Hoeffding Tree because streaming classifiers must learn from massive, continuous, non-stationary data, and synthetic streams are the only practical way to generate the volume, variability, and controlled drift patterns needed to evaluate and tune them.

DevOps

Scalability is a core of DevOps, and Kaizen ensures the system keeps improving as it grows.

My journey into DevOps started with my career at Honeywell where I managed kaizen event in a lean re-manufacturing environment of aircraft Auxiliary Power Unit (APU).

In DevOps, scalability is the ability of systems, pipelines, teams to grow without breaking, slowing down, becoming chaotic. It applies to both technical & organizational scalability.

1. Technical scalability
    - Infrastructure scales automatically (Kubernetes, Terraform, cloud autoscaling)
    - CI/CD pipelines handle more builds without bottlenecks
    - Monitoring expands as services multiply
    - Automation replaces manual work so growth doesn't increase workload

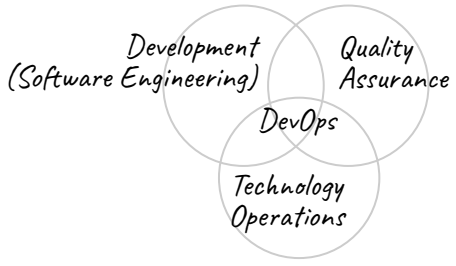
    Scalability ensures as application/platform grows, DevOps processes grow along.

2. Organizational scalability
    - Teams can onboard new developers quickly
    - Releases stay stable even as the codebase grows
    - Quality remains consistent across multiple squads
    - Collaboration stays smooth as the organization expands

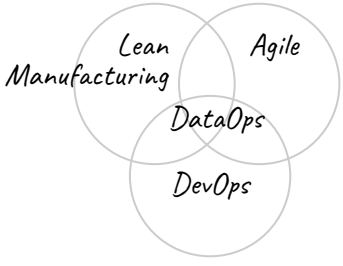
    DevOps makes growth sustainable.

DevOps vs DataOps

DevOps holds the true legacy and spirit of Kaizen. It broke down functional silos and established philosophy of continuous micro-improvements in technology. DataOps simply inherited that strong DevOps DNA and optimized it for the unpredictable world of data.



Source:
Wikipedia



かい　　ぜん
ka-ee　　ze-en
改 善
"change"　　"good"

Kaizen is a philosophy of continuous improvement (Haviv, Gift 2023) – is a natural fit for DevOps because DevOps itself is built on continuous iteration.

Kaizen strengthens DevOps in three key ways

1. Continuous improvement of pipelines
   Kaizen encourages teams to constantly refine:
   - Build times
   - Deployment reliability
   - Test coverage
   - Automation scripts
   - Infrastructure templates

   This prevents pipelines from becoming slow or fragile as the system grows.

2. Eliminating waste (Lean principles)
   Kaizen removes:
   - Manual steps
   - Repetitive tasks
   - Unnecessary approvals
   - Inefficient handoffs
   - Over-processing

This directly improves scalability because waste multiplies as teams grow.

3. Data-driven decision making
   Kaizen uses metrics – defect rates, cycle time, throughput – to guide improvements.
   DevOps uses metrics too – deployment frequency, MTTR, lead time, reliability.

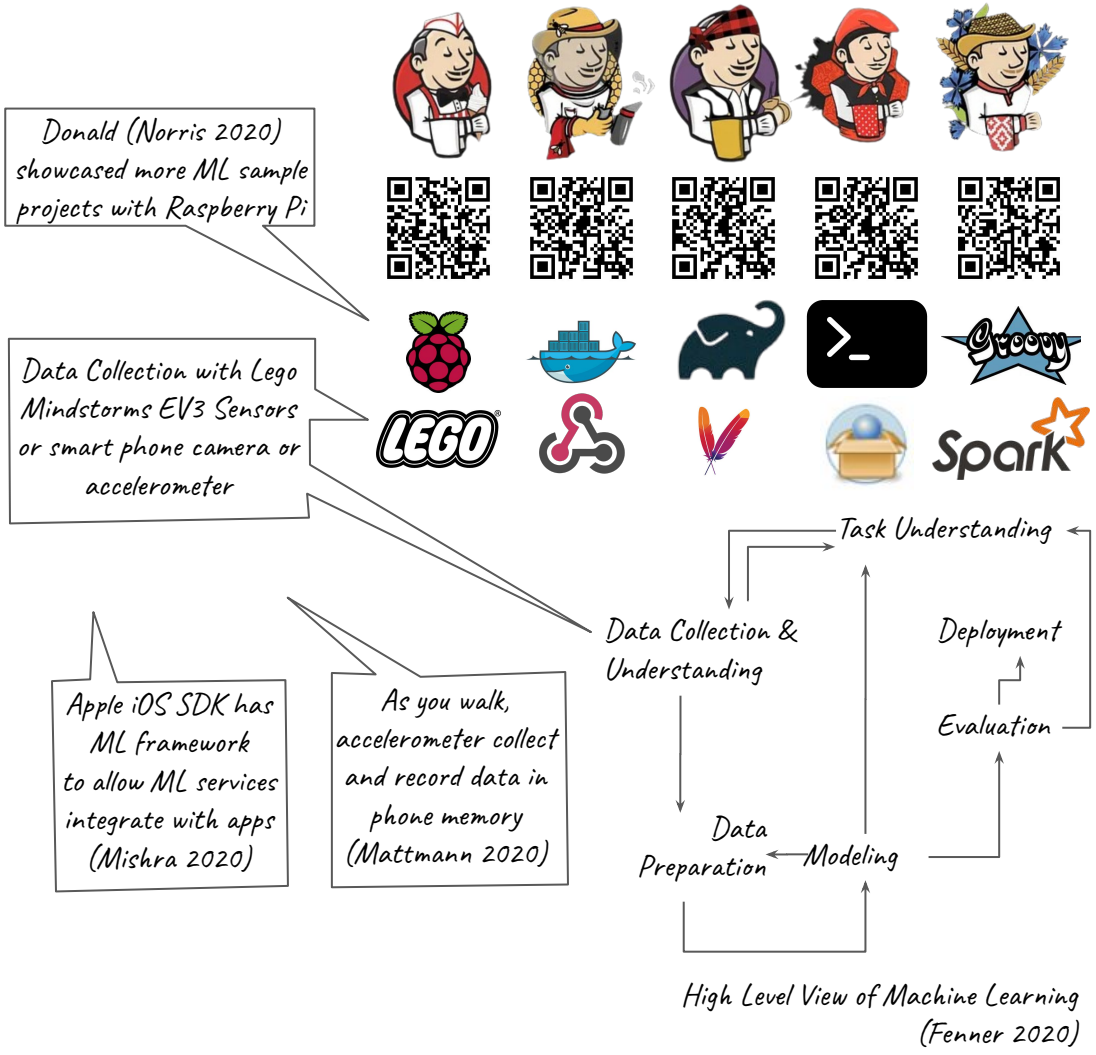
Together, they create a feedback loop that keeps the system scalable.

Companies like Honeywell, GE, Siemens, and Rockwell blend DevOps + Lean + Six Sigma + Kaizen – the combination keeps large industrial systems stable as they scale.

MLOps

MLOps is a practice and cultural approach that unifies machine-learning development with deployment and operations, enabling teams to manage, automate, and standardize the ML lifecycle from model creation to production maintenance. It provides scalability by turning ML workflows into predictable, automated, elastic systems. Operationalizing a model requires balancing time, resources, quality, similar to DevOps traditionally. MLOps applies this mindset to AI, making projects more efficient and reliable. It offers a structured framework for coordinating data, code, infrastructure, and schedules while ensuring quality and meeting business needs through scalable, predictable workflows.

One of the important components in both DevOps and ML Ops: Continuous Integration and Continuous Delivery/Deployment, or CI/CD - a skillset that I started to develop few years ago where I experimented with Lego EV3 + Raspberry Pi for slicing fruit and other Jenkins CI/CD projects with Docker-in-Docker, Mavel / Gradel, CLI & Groovy application.



When is Jenkins the right choice for CI/CD ?

Scenario	Is Jenkins the go-to choice?	Best Alternative
New cloud-native project	❌ No (Overkill/high setup)	GitHub Actions or GitLab CI Example of my GitHub Action project
Kubernetes / GitOps deployment	❌ No (Better specialized tools)	Argo CD / Flux Example of my Flux CD project
Large enterprise with legacy infra	✅ Yes (Unmatched customization)	Jenkins / Azure DevOps
Air-gapped / Self-hosted requirements	✅ Yes (Full infrastructure control)	Jenkins / GitLab (Self-Hosted)

Soroush Razavi from amii mentioned CI/CD is a foundation to DevOps, and CI/CD/CT as the foundation in MLOps. DevOps automate software delivery. MLOps automate both software and model life cycles & continuous training (CT) becomes an essential extension.

Selected to host a technical Lunch-and-Learn at JLL, I demonstrated a full MLOps pipeline for image captioning built with Databricks, Azure DevOps, MLflow, Keras/TensorFlow. Herewith is the similar approach that I used.

Keras original backend was Theano. When Google introduced Tensorflow, programmers began to implement both that one could not have Keras without TensorFlow and vice versa (Silaparasetty 2020). Keras soon made TensorFlow a default backend.

On the last two chapters of Ekaba book (Bisong 2019) he introduced Containers + GKE and Kubeflow + Kubeflow Pipelines. Vertex AI sits between GKE and Kubeflow, not about capability but operational burden vs. flexibility.

Introduction to GCP and other Cloud Platforms

Google Cloud is a suite of cloud computing services offered by Google that provides a series of modular cloud services including computing, data storage, data analytics, and machine learning, alongside a set of management tools.
Beside GCP, the other well known cloud providers include AWS and Azure as well as Virtual Private Server such as Contabo which focuses on Infrastructure-as- a-Service (IaaS), providing low-cost computing resources:

- Cloud VPS (Virtual Private Servers): Scalable, virtual instances for running web applications, databases, or development environments.
- Virtual Dedicated Servers (VDS) & Bare Metal: Dedicated physical hardware for higher performance needs.
- Cloud Object Storage: S3-compatible storage for backups and media.

Feature	Contabo / Linode / DigitalOcean	AWS / Google Cloud / Azure
Pricing Model	Fixed monthly rates with high resource allocations for low cost.	Pay-as-you-go / usage-based, which can scale up unpredictably.
Services Offered	Core compute, storage, basic networking, APIs, and image deployments.	Hundreds of specialized managed services (serverless, AI models, managed databases, global CDNs, etc.).
Best For	Developers, small-to-medium businesses, self-hosters, and budget-conscious web projects.	Large enterprise systems, complex microservices, auto-scaling applications, and serverless architectures.

Herewith a sample classification that run on Contabo to extract images from DBeaver and send back the classification inspired by a fashion computer vision brought by Laurence (Moroney 2020) with Python or Norman (Matloff 2023) with R for example.

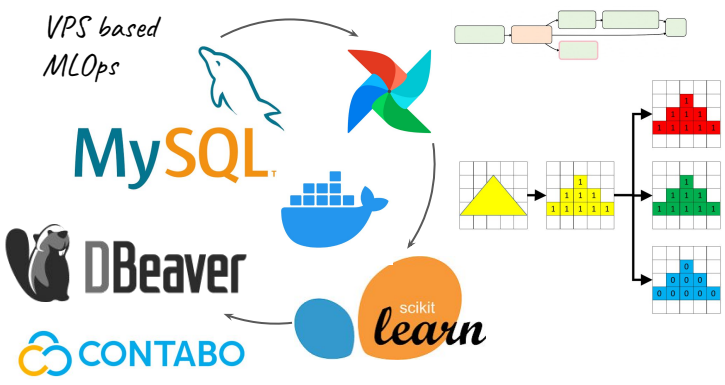


Image Classifier with Airflow DAG (Directed Acyclic Graph) as a scheduler and using python based on Logistic Regression (better Interpretability) or Convolutional Neural Network (better Scalability) which is more commonly used to classify images (Singh 2019)

If a machine learning operation experiences predictable, steady traffic, Contabo's fixed-resource approach provides huge cost savings for large compute or memory needs. If the workload requires rapid, automated spikes in traffic (like sudden viral spikes or flash sales), hyperscalers like AWS/GCP excel because their ecosystem automates resource allocation without human intervention.

Feature / Metric	Contabo / Alternative VPS	AWS / Google Cloud / Azure (Hyperscalers)
Scaling Mechanism	Manual / Vertical Primary: Upgrade RAM/CPU or manually spin up additional nodes. Storage has optional auto-expansion.	Automated Horizontal & Vertical: Native metric-driven auto-scaling groups scale compute up/down automatically.
Speed to Scale	Minutes: Resizing an instance or deploying a new VPS typically requires a manual reboot or user setup.	Seconds: Serverless (e.g., AWS Lambda) and container instances scale instantly with load spikes.
Load Balancing & Orchestration	Self-Managed: Must manually deploy and configure software load balancers (e.g., NGINX, HAProxy) or Kubernetes clusters.	Fully Managed: Plug-and-play Application Load Balancers, global traffic directors, and managed Kubernetes (EKS, GKE).
Global Infrastructure & Availability	Limited Regions: Scaling across regions is manual; fewer data center locations globally.	Global Footprint: Dozens of regions and Availability Zones worldwide for low latency and high availability.
Scaling Cost Predictability	High Predictability: Fixed monthly plans mean costs remain low and transparent even at high static resources.	Low Predictability: Pay-as-you-go dynamic scaling can lead to unpredictable usage spikes and unexpected bills.

Gemini Enterprise Agent Platform

At Google Cloud Next 2026, Google consolidated and rebranded Vertex AI into the Gemini Enterprise Agent Platform (commonly called the Agent Platform).

In the GCP Console, Vertex AI was subsumed under Agent Platform.

API Name: Gemini Enterprise Agent Platform API (or Agent Platform API).
Service Name: aiplatform.googleapis.com

Herewith what Vertex AI really covers



Vertex AI (the platform)= {

AutoML	(no code model builder)
Custom Training	(write your own code)
Vertex Pipelines	(managed engine based on Kubeflow)
Endpoints & Registry	(Hosting and MLOps)

AutoML refers to the methodology of automating the process of building machine learning models, including data preprocessing, feature engineering, model selection, hyperparameter tuning and model deployment. The goal of AutoML is to make machine learning accessible to non-experts and to drastically speed up the iterative workflow for experienced data scientists. As a key property of ML is automation, AutoML can be regarded as automating automation (Song 2022) : the automation of automation.

Custom Training includes writing custom PyTorch/TensorFlow code with custom containers.

Vertex Pipelines is an orchestration service for automating machine learning workflow. Kubeflow is an open source Kubernetes framework fir developing and running portable ML workloads. Kubeflow Pipelines is a Kubeflow service that lets you compose, orchestrate and automate ML systems (Mona). Google didn't "add" Kubeflow to AutoML; rather, Google built Vertex Pipelines on top of Kubeflow Pipelines (KFP). When you write a pipeline to automate data ingestion, AutoML training, evaluation, and deployment in Google Cloud, you typically write it using the Kubeflow SDK, which then runs managed inside Vertex AI.

Model Registry & Endpoints is basically for versioning, hosting, and serving models for inference - which the only piece of Vertex AI we would cover in the next few pages.

In a scaled MLOps project, such as proposed in the page 26, the entire ecosystem would look like these three steps:

Step 1 (Data Prep): BigQuery pulls raw data and prepares feature tables.

Step 2 (Custom Training & MLflow Tracking): A Kubeflow pipeline component launches a training script (running custom PyTorch, LightGBM, XGBoost, etc.).
Inside the training code, MLflow logs training metrics (e.g., loss, accuracy), parameters, and saves the trained model artifact to MLflow Model Registry.

Step 3 (Deployment): The next Kubeflow pipeline component fetches the "Challenger" model from MLflow's Model Registry.
It uploads that artifact to Vertex AI Model Registry and deploys it to a Vertex AI Endpoint for real-time serving.

Aspect	Original Flow (AutoML)	Scaled Flow (MLflow + Custom Code)
Model Creation	Google automatically searches, builds, and selects the model.	Your team writes custom ML code (PyTorch, TensorFlow, XGBoost, etc.).
Experiment Tracking	Native inside Vertex AI Experiments.	Centralized via MLflow Tracking (metrics, parameters, confusion matrices).
Model Registry	Stored natively in Vertex AI Models.	Stored in MLflow Model Registry (and synced/pushed to Vertex AI for endpoint serving).
Control & Flexibility	Low (Black box algorithm selection).	High (Full control over neural architectures, custom loss functions, and feature logic).

Saving and Sharing Machine Learning Model

For a simple deployment covered in this book we would develop a model outside GCP and later exported into Vertex AI such as in joblib format. If the model was developed inside GCP, the final model file format depends entirely on whether AutoML or Custom ML Code, as well as the framework used. Google does not lock a proprietary format, it exports standard, open formats and can run anywhere (on-premises, AWS, Azure, edge devices).

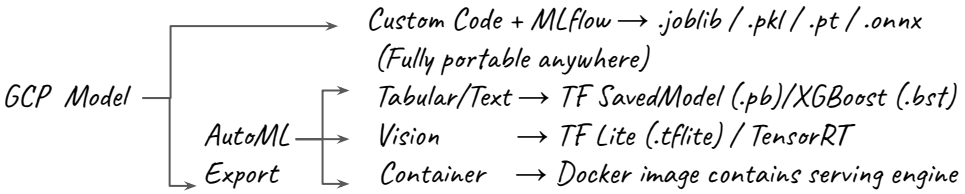
If Custom ML Code + MLflow used
When writing custom model code, you control the format, and MLflow logs it natively in the standard framework format:

- Scikit-Learn / XGBoost / LightGBM: Typically saved as .joblib, .pkl (pickle), or XGBoost .json / .bst files.
- PyTorch: Saved as standard .pt or .pth state dictionaries.
- TensorFlow / Keras: Saved as TensorFlow SavedModel directories (containing saved_model.pb) or .keras / .h5 files.
- Cross-Platform / High-Performance: Many production teams convert custom models into ONNX (.onnx) so they can be served blazing-fast outside of GCP via the ONNX Runtime on any cloud or local server.

If Google AutoML used
If a model using AutoML inside Vertex AI (or BigQuery ML) is trained and wanted to export it outside GCP, Google provides industry-standard export formats like below

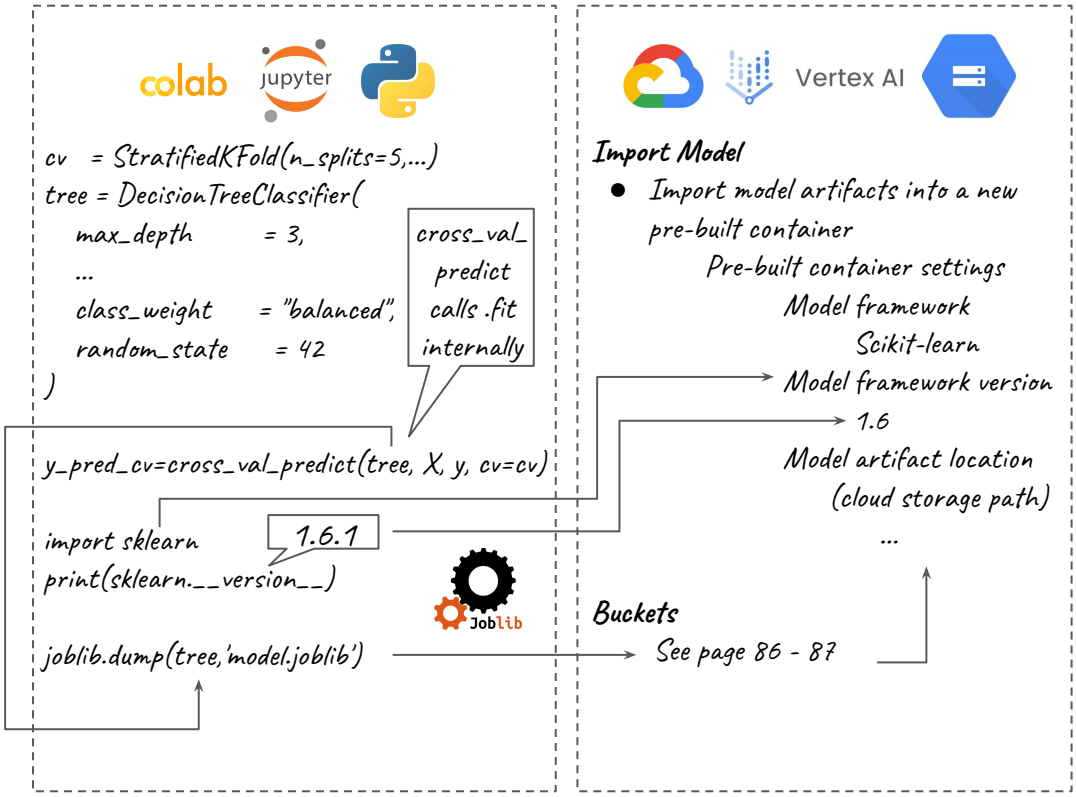
AutoML Task	Exported File Format	How it Works Outside GCP
AutoML Tabular	TensorFlow SavedModel or XGBoost Booster	Google exports a SavedModel directory (saved_model.pb + variables) or XGBoost binary (model.bst) alongside a pre-packaged Docker image. Docker image can run anywhere to serve predictions.
AutoML Vision / Image	TensorFlow Lite (.tflite), SavedModel, or TensorRT	Exports standard .tflite files meant to run on edge devices (like mobile phones or Raspberry Pis), or standard TensorFlow models.
AutoML Text / NLP	TensorFlow SavedModel	Contains the full preprocessing graph and weights ready for TensorFlow Serving.

Exporting Models from GCP



Importing Models into GCP

Below is an example of 80% training / 20% testing split (n_splits=5 → 4 train, 1 test) with DecisionTreeClassifier model, saved into model.joblib, ensured the sklearn version used to export from above model is similar to sklearn version version used to import from GCP.



A common choice for n_splits is typically 5 or 10 with the later offer a good bias and variance trade-off such as for decision trees (Raschka 2024).

Vertex AI accepts PyTorch, Scikit-learn, TensorFlow, XGBoost as model framework. As the data get bigger, Pyspark and spark dataframe may be needed. Abdelazis (Testas 2023) compared how scikit-learn and pyspark performed where Adi (Polak 2023) compared panda dataframe with spark dataframe. In enterprise settings, scikit-learn is preferred due to faster experimentation, simpler deployment, richer model ecosystem and lower cost.

GCP Simple Deployment Procedure

In order to apply a simple deployment, it is needed to create a GCP account. If a Gmail address available, and GCP account not yet created in the past, $300 in free credits is given upon registering for GCP and valid for up to 90 days.

As a GCP account created, Vertex AI can be used to deploy the machine learning model so that other systems or applications can use it.

In cloud environments, this is typically done by exposing the model as a service that can receive input data and return predictions. On Vertex AI (the chosen managed machine learning platform within GCP) this process is simplified into a few clear steps. At a high level, deploying a model involves three steps: registering the model, deploying it to an endpoint, and sending prediction requests.

Deploying the Model to an Endpoint

After a model is registered, it must be deployed to an endpoint so it can receive prediction requests.

An endpoint is a network-accessible service that hosts the model and allows external applications to interact with it. Instead of running the model locally inside a notebook, the model runs on managed cloud infrastructure and listens for incoming requests.

When you deploy a model to an endpoint in Vertex AI:

- Vertex AI allocates compute resources to host the model
- It creates a secure URL where the model can be accessed
- It manages scaling, infrastructure, and availability

At this point, the model is essentially turned into an online prediction service. Other systems such as dashboards, monitoring pipelines, or applications can now send data to this endpoint to obtain predictions.

Registering the Model

The first step is to upload and register the trained model in the model registry.

A model registry is a centralized place where machine learning models are stored and versioned. Instead of keeping models scattered across notebooks or local machines, the registry acts like a structured catalog of models that can later be deployed, updated, or monitored.

In Vertex AI, this typically involves:

- Uploading the model files (for example a .pkl, .joblib, or TensorFlow saved model)
- Storing the model files in Google Cloud Storage, where files are kept in containers called buckets
- Registering the model as an asset (also called an artifact) in the Vertex AI Model Registry

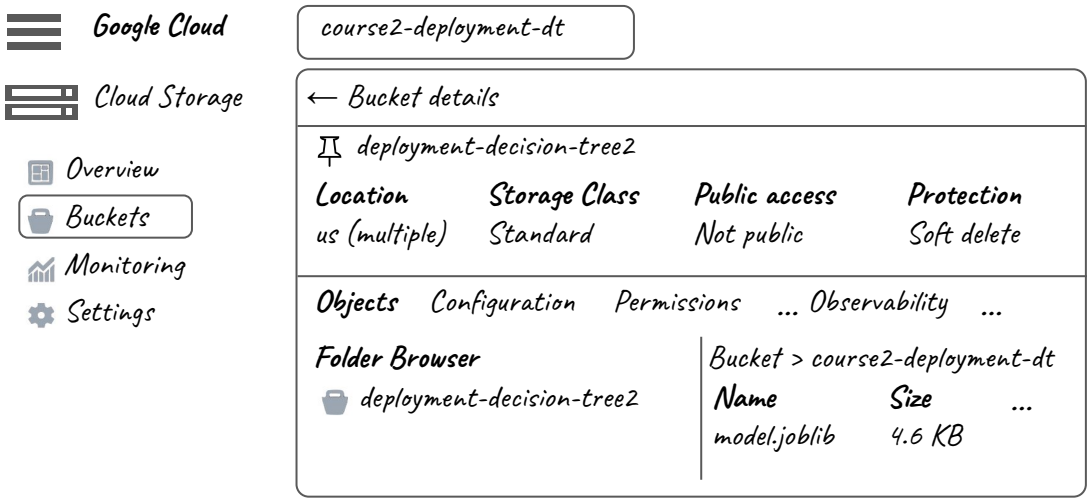
Once registered, the model becomes a managed resource within the platform. This means engineers can track different versions of the model, manage permissions, and deploy it when needed. You can think of the registry as a library of trained models ready to be used in production.

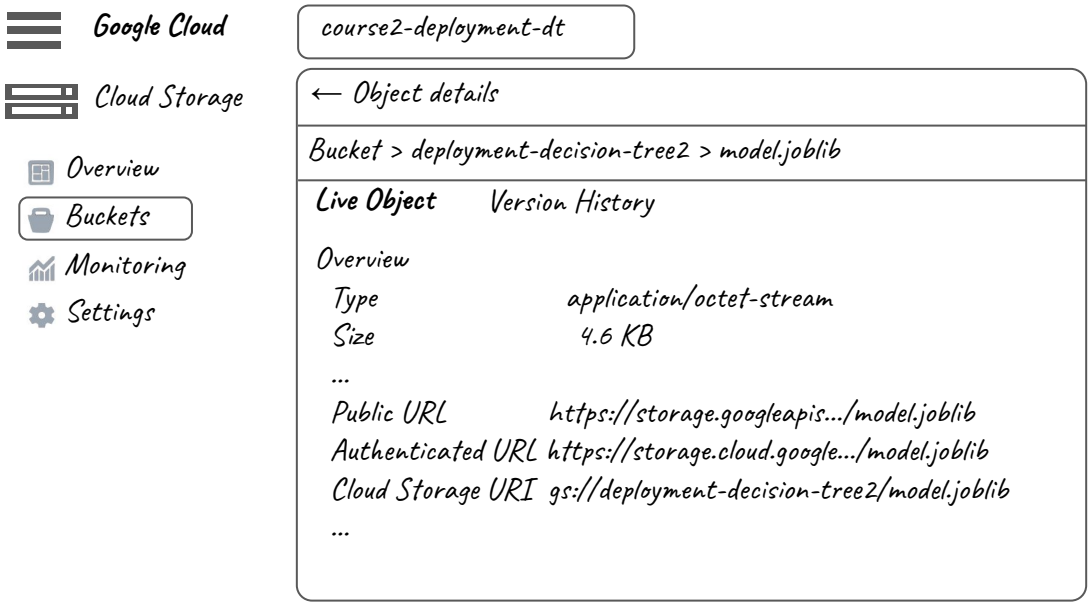
Sending and Receiving Prediction Requests

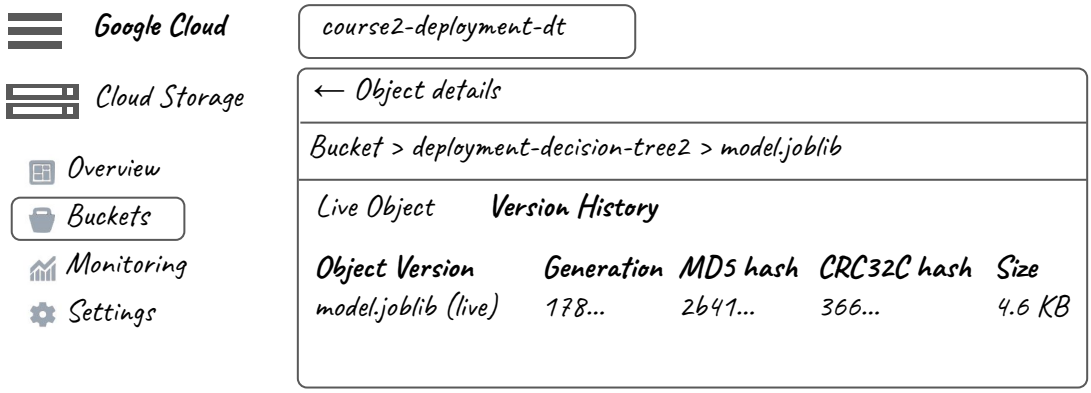
Once the endpoint is active, applications interact with the model using an Application Programming Interface (API). An API is a structured way for two software systems to communicate with each other. Instead of manually running code, a client application sends a request to the model's endpoint through the API.

These requests are typically formatted using JavaScript Object Notation or JSON. JSON is a lightweight text format used to organize and transmit structured data. It represents data as key-value pairs and lists, making it easy for software systems to parse and exchange information. The deployed model processes the input and returns a response, also formatted in JSON.

Buckets


Google Cloud
course2-deployment-dt
Cloud Storage
Overview
Buckets
Monitoring
Settings
← Bucket details
deployment-decision-tree2
Location
us (multiple)
Storage Class
Standard
Public access
Not public
Protection
Soft delete
Objects
Configuration
Permissions
... Observability ...
Folder Browser
deployment-decision-tree2
Bucket > course2-deployment-dt
Name
Size
...
model.joblib
4.6 KB


Google Cloud
course2-deployment-dt
Cloud Storage
Overview
Buckets
Monitoring
Settings
← Object details
Bucket > deployment-decision-tree2 > model.joblib
Live Object
Version History
Overview
Type
application/octet-stream
Size
4.6 KB
...
Public URL
https://storage.googleapis.../model.joblib
Authenticated URL
https://storage.cloud.google.../model.joblib
Cloud Storage URI
gs://deployment-decision-tree2/model.joblib
...


Google Cloud
course2-deployment-dt
Cloud Storage
Overview
Buckets
Monitoring
Settings
← Object details
Bucket > deployment-decision-tree2 > model.joblib
Live Object
Version History
Object Version
Generation
MD5 hash
CRC32C hash
Size
model.joblib (live)
178...
2b41...
366...
4.6 KB

On Google Cloud Storage (GCS), we can find available folders and files and their characteristics such as when the file was created, modified, how to access them from outside GCP and how to use their fingerprint to possibly help to identify duplicates.

On the first screen, this is a typical screen from a specific folder and what files inside it. On the next page where we would like to create endpoint for a model, the folder name is exactly what we need to point out, not the file itself.

When viewing the Object Details of a file in a GCP bucket, the Live Object section displays three different access formats: Public URL, Authenticated URL, and Cloud Storage URI. Each serves a distinct purpose.

In a scaled MLOps project, datasets are often stored in Cloud Storage alongside imported models. If you need to validate data integrity from outside GCP – for example, using Postman – you would use:

- the Public URL when the object is publicly accessible,
- the Authenticated URL when accessing a private object using OAuth2 or a service account token.

The Cloud Storage URI (gs://bucket/object) is not an HTTP endpoint and cannot be accessed through Postman. It is used only by internal GCP services such as BigQuery, Dataflow, Dataproc, and Vertex AI.

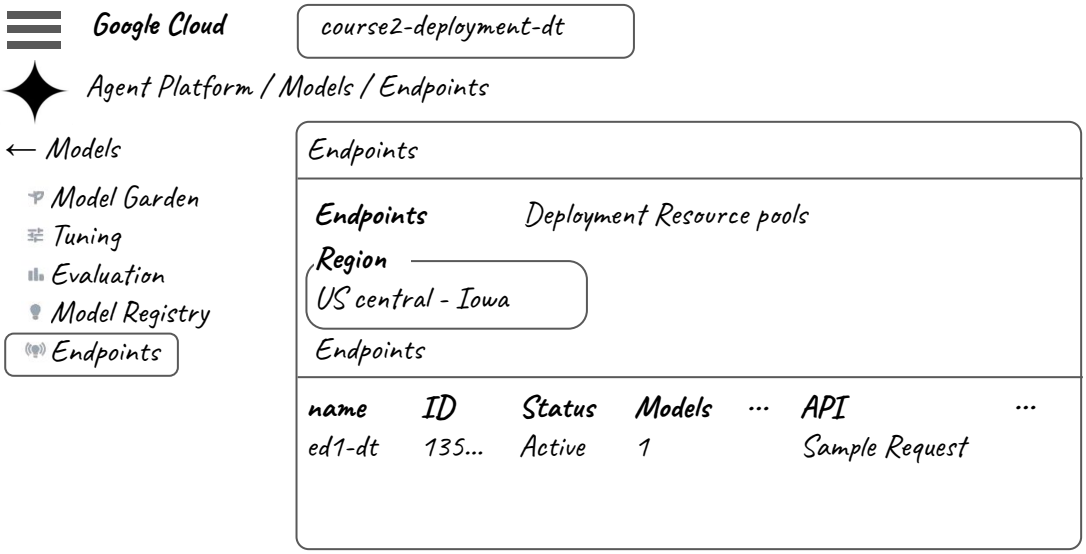
Generation is a 64-bit identifier for each object version in a bucket. Every upload creates a new generation, older versions remain when Object Versioning is enabled, and specific generations can be retrieved or deleted.

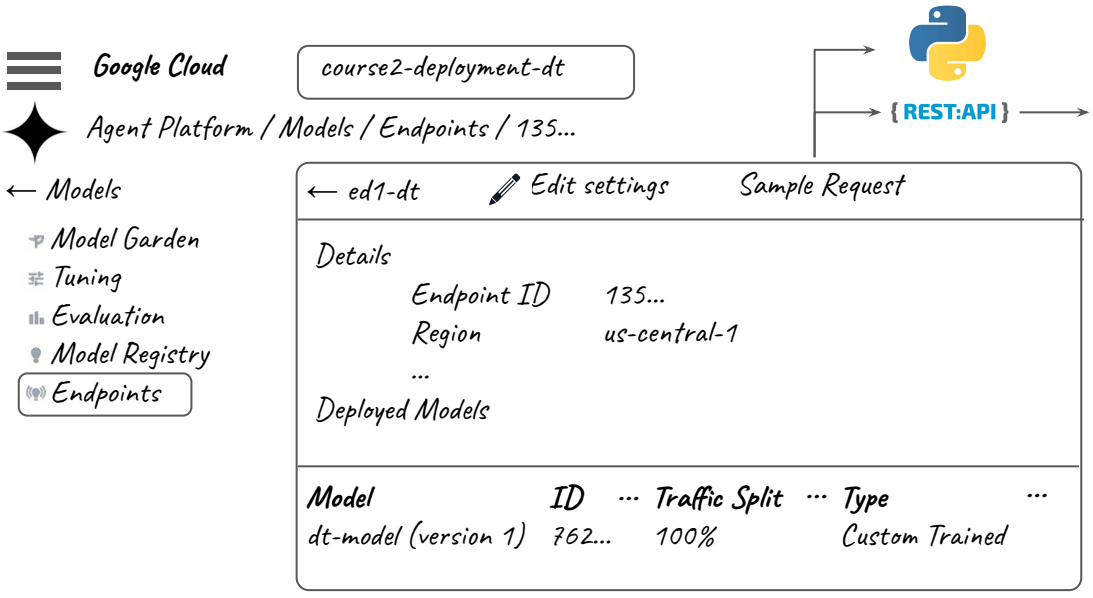
MD5 hash is a cryptographic fingerprint of the object's contents. It verifies that the stored file matches the uploaded file, changes when the content changes, and helps detect corruption or accidental modification.

CRC32C is a fast checksum used internally by Cloud Storage. It supports automatic integrity checks during upload and download, is hardware-accelerated, and ensures the object was transmitted without corruption.

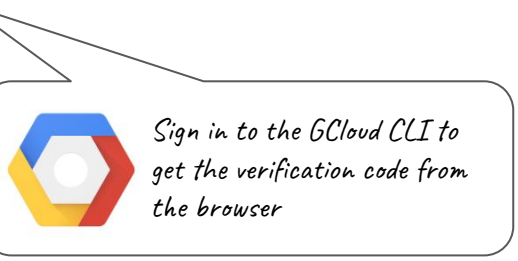
End Points

In order to create an endpoint, we need to define access (standard will allow us for API access), traffic split (100% for now, could be less for further split for testing), Minimum and maximum replicas (1 for no auto-scaling for now), machine type (such as standard n1, 2vCPUs, 7.5 GB memory) , reservation type, availability policies (such as standard VM provisioning model), logging setting, explainability options, monitoring setting such as sampling rate (the more sample, the more is the cost) and monitoring objective (I choose inference drift detection for demo purpose for now). After all setting above done, the creation of this endpoint in my case may take around 40 minutes.





```
$ gcloud auth application-default login
...
Enabling service [cloudresourcemanager.googleapis.com] on project [course2-]...
Operation "operations/acat.p2-... finished successfully.
$ nano test.json
$ cat test.json
{
"instances": [
[12,2,10], [12,4,19],
[10,4,24], [12,4,10],
[14,20,23], [14,10,24],
[13,20,20], [15,10,20]
]
}
$ ENDPOINT_ID="135..."
PROJECT_ID="210..."
$ curl -X POST \
  -H "Authorization: Bearer $(gcloud auth print-access-token)" \
  -H "Content-Type: application/json" \
"https://us-central1-aiplatform.googleapis.com/v1/projects/${PROJECT_ID}/locations/us-ce
ntral1/endpoints/${ENDPOINT_ID}:predict" \
  --data @test.json
{
  "predictions": [
    "Potential CU Candidate",
    "Potential CU Candidate",
    "CCS Candidate",
    "Potential CU Candidate",
    "CCS Candidate",
    "CCS Candidate",
    "CCS Candidate",
    "CCS Candidate"
  ],
  ...
}
```



Classification based on given features log_emissions, naics_ sector_encoded, z and years_reported by using REST

Model Registry

The classifier can also be executed without writing any code by using the Model Registry in Vertex AI. Before doing this, make sure the model already has an endpoint created. In practice, the endpoint you deployed earlier must originate from a model that was first registered in the Model Registry – either by importing it as a new model (when no previous version exists) or by adding it as a new version to an existing model.

When importing the model, I chose a nearby region such as us-central1, which is not only geographically close but also operates as a low-carbon data center. After selecting the region, complete the model configuration steps as described on page 83, and the model will be ready for prediction operations.



Low CO_2 Data Center

In the context of low-carbon strategies, I would like to highlight the topic of electrification, which remains particularly interesting – and challenging – because many regions still rely heavily on diesel generators, as seen recently in the United Kingdom. This underscores that electrification alone does not guarantee low-carbon outcomes unless the supporting infrastructure is also decarbonized.

Google's Iowa region (us-central1) is one of the lowest-CO_2 hyperscale cloud regions in North America, but not because it has no gas turbines. Google's Iowa data centers are backed primarily by wind PPAs, solar, soon nuclear, while the grid itself is one of the most renewable-heavy in the U.S (64% of Iowa's electricity from wind).

The Iowa grid does include natural gas peaker plants, but they are not dedicated to Google's data centers. Iowa utilities have filed for simple-cycle combustion turbines to cover peak demand such as Adair County: 465 MW, Orient Energy Center: 465 MW. These gas units exist on the grid, not "attached" to Google's facilities.

GE Vernova recently feed the grid needs for electricity with gas turbines, such as in
PJM (Virginia) – GE 7F gas turbines
ERCOT (Texas) – GE 7HA, 7F, LM6000 peakers
Brazil – GE LM6000 and LM2500 peakers
India – GE Frame 9E and 6F units
Taiwan – GE Frame 7FA units at Taipower plants



Higher CO_2 Data Center

Indirectly, those turbines help power Google data centers, not dedicated connections. Google buys grid electricity and offsets carbon footprint through PPAs (Power Purchase Agreements), renewable energy credits rather than owning or operating gas turbines.



Deployment



Google Cloud course2-deployment-dt



Agent Platform / Models / Endpoints

← Models

- Model Garden
- Tuning
- Evaluation
- Model Registry
- Endpoints

Model Registry

Region
US central - Iowa

	name	Default version	Deployment Status	Description
◎	dt-model	1	Deployed	decision-tree-scikit



Google Cloud course2-deployment-dt



Agent Platform / Models / Model Registry / Model:565... / Version: 1 / Deploy

← Models

- Model Garden
- Tuning
- Evaluation
- Model Registry
- Endpoints

← dt-model > version 1

Evaluate **Deploy & Test** Batch Infer ...

Deployed Your Model

deploy to endpoint

	name	ID	Status	Models	⋯	API	...
◎	ed1-dt	135...	Active	1		Sample Request	

Test Your Model

JSON request

```
"instances": [
[12,2,10],
[12,4,19],
[10,4,24],
[12,4,10],
[14,20,23],
[14,10,24],
[13,20,20],
[15,10,20]
]
}
```

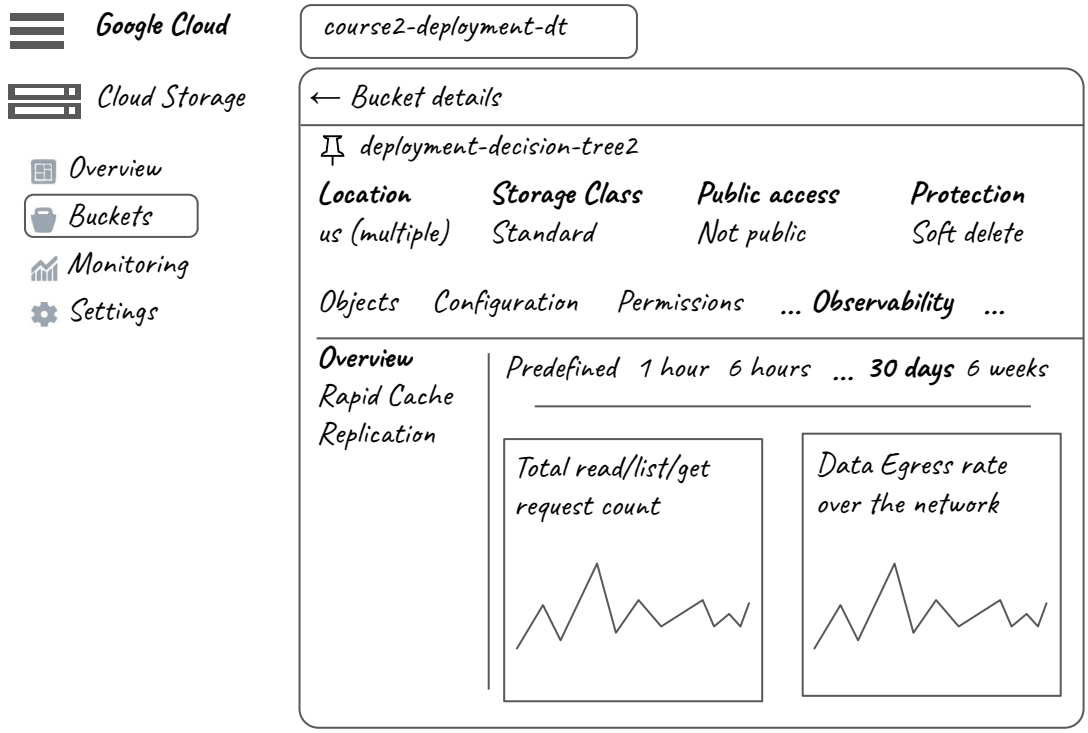
Response

```
{
  "predictions": [
    "Potential CU Candidate",
    "Potential CU Candidate",
    "CCS Candidate",
    "Potential CU Candidate",
    "CCS Candidate",
    "CCS Candidate",
    "CCS Candidate",
    "CCS Candidate"
  ], ... }
```

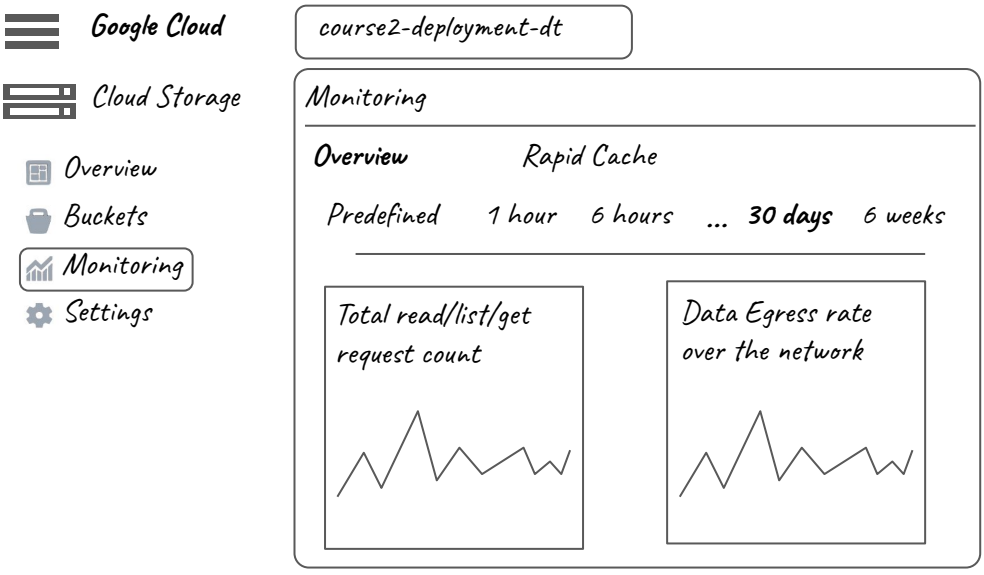
infer

Monitoring and Observability

As we set the observability ON when we setup the endpoint, we can find the Observability tab with few details such as Total read/list/get request count and Data Egress rate over the network from GCS Buckets.



Alternatively the same charts can be found in GCSBuckets Monitoring tab.



I can also monitor my ML model with Prometheus and Grafana instead of GCP's built-in Monitoring. This is a very common pattern in Kubernetes-based ML deployments, especially when you want full control, custom metrics, or vendor-neutral observability.

As a popular observability tool (Luu, Pumperla, Zhang 2024), Prometheus scrapes model's metrics, Grafana visualizes them which all are independent of Google Cloud Monitoring.

When Prometheus/Grafana is better than GCP Monitoring

- run Kubernetes (GKE, EKS, AKS, on-prem).
- want open-source, vendor-neutral observability.
- need custom ML metrics (drift, SHAP, confidence histograms).
- want PromQL for advanced queries.
- want Grafana dashboards shared across multi-cloud.

When GCP Monitoring is better

- use Vertex AI Prediction or Vertex AI Endpoints.
- want automatic dashboards with zero setup.
- want integrated logs + traces + alerts.
- want SLOs and uptime checks built-in.
- don't want to manage Prometheus storage.

Example of Prometheus and Grafana application

Feature	Prometheus + Grafana	GCP Monitoring
Custom ML metrics	✔️ Full control	⚠️ Limited unless using Vertex AI
Kubernetes-native	✔️ Best option	✔️ Good (via Managed Prometheus)
Multi-cloud	✔️ Yes	❌ No
Setup effort	Medium–High	Low
Cost	Free (infra only)	Pay-as-you-go
Alerting	✔️ PromQL	✔️ Cloud Alerts
Logs	❌ Needs Loki	✔️ Built-in
Traces	❌ Needs Tempo	✔️ Built-in

Security and Compliance

From cloud engineering to advanced machine learning, security and compliance aren't just features—they are the foundation of everything we build in MLOps.

As I highlighted earlier, traditional assessments are expensive because they depend on non-public engineering data. But having access to that real-world data is the only way to build models that actually work.

In reality, getting the right data to power a Decision Tree classifier or any other algorithm is often 80% of the work. It's hard because companies guard their operational data fiercely to protect their competitive edge. Securing those key features requires strict legal agreements and airtight infrastructure designed to prevent data breaches from both internal and external threats.

I saw this firsthand during my data engineering role at JLL - a role that dominated cloud during pandemic (Gift, Deza 2021), where our Data Governance team helped us navigate these exact data-sharing hurdles while keeping our training data secure.

Who owns what?

Responsibility	Managed By	Role Summary
Defining Security & Compliance Standards	Data Governance / CISO / Legal	Dictates what rules must be followed and tracks compliance across the enterprise.
Data Classification & Cataloging	Data Governance / Data Stewards	Categorizes datasets (e.g., tagging sensitive user data).
Feature Requirements & Privacy-by-Design	Product Manager	Translates compliance requirements into product requirements (PRDs) so engineers build compliance into user workflows.
Technical Implementation	Engineering / DevOps	Builds identity access management (IAM), data masking, encryption, and K8s RBAC to execute the policies.

How do companies usually delegate sensitive data from spreading out ?

While blocking USB ports stops physical data extraction, preventing exfiltration through personal webmail (e.g., Gmail, Yahoo, Outlook) requires a combination of technical controls, data loss prevention (DLP), and network architecture.

Companies prevent data leakage through personal webmail using four main technical controls:

- Endpoint DLP: Blocks file uploads and copy-pasting into unapproved browser domains.
- CASB / Web Gateways: Inspects HTTPS traffic to block outbound attachments or enforce "read-only" webmail mode.
- Information Rights Management (IRM): Encrypts files so they cannot be opened outside corporate accounts.
- Virtual Desktops (VDI): Seals sensitive data inside cloud environments where local downloading is prohibited.

Is it safe or professionally wise to use my personal GCP account for company proof-of-concept work to avoid appearing costly during layoffs?

Some engineers consider using a personal GCP account for proof-of-concept work because they want to reduce company cloud spending and avoid appearing costly during layoff seasons, assuming HR might flag "expensive employees" based on cloud usage.

However, this approach is professionally inappropriate and counterproductive. Personal accounts introduce unmanaged identities, break IAM governance, weaken MFA enforcement, and create audit and compliance risks. They also obscure cost visibility, fragment billing, and make spend attribution impossible. Importantly, cloud cost KPIs are tied to projects and cost centers—not individuals—so using a personal account offers no protection.

The correct practice is to use company-managed GCP with sandbox projects, budget alerts, proper tagging, and low-cost resources. Personal GCP should be used only for personal learning, never for corporate experimentation.

Give an example of practical use case for MLOps Security and Governance !
Herewith is an example of ClusterRoles, ClusterRoleBindings, and Certificate Signing Requests (CSRs) in Kubernetes environment from my personal project.
https://github.com/FariusGitHub/csr_cr_crb

GCP Account and Billing

In GCP, any projects that are not attached to a billing account won't be able to run an API such as Vertex AI used in our machine learning approach. To see what linked and unlinked projects, we can use IAM or CLI (command line interface) as follow



IAM & Admin → Manage Resources. Let's say there are four projects listed as below. The first one is unlinked to neither billing account, the second one is linked to multiple billing account and the last two to billing account 0138F2- 2D19F2-56DCE1

Name	ID
tije280-org	21869486139
My Project 63401	operating-tiger-503823-m4
My First Project	project-4dfbac02-e93e-4819-bed
No organization	
course2-deployment-dt	course2-deployment-dt-500322
first-project	model-union-500321-r9

This project has **no billing account**

You have **multiple billing accounts**

Linked to 0138F2-2D19F2-56DCE1



The other way to list all projects to above billing account would be using CLI (Command Line Interface) like below

```
$ gcloud billing projects list --billing-account=0138F2-2D19F2-56DCE1
```

to list all projects, we could use below command

```
$ gcloud projects list
```

```
PROJECT_ID: course2-deployment-dt-500322
NAME: course2-deployment-dt
PROJECT_NUMBER: 210354004546
ENVIRONMENT:

PROJECT_ID: model-union-500321-r9
NAME: first-project
PROJECT_NUMBER: 704373922934
ENVIRONMENT:

PROJECT_ID: operating-tiger-503823-m4
NAME: My Project 63401
PROJECT_NUMBER: 658213354826
ENVIRONMENT:

PROJECT_ID: project-4dfbac02-e93e-4819-bed
NAME: My First Project
PROJECT_NUMBER: 476803326754
ENVIRONMENT:
```

Billing
This project has no billing account
This project is not linked to a billing account

Link a Billing Account

Billing Management → Account Management

Billing account ID
0138F2-2D19F2-56DCE1

Manage Billing Account

Projects linked to this billing account

Project name	Project ID	Actions
course2-deployment-dt	course2-deployment-dt-500322	⋮
My First Project	project-4dfbac02-e93e-4819-bed	⋮
first-project	model-union-500321-r9	⋮

disable billing

```
PROJECT_ID: course2-deployment-dt-500322
BILLING_ACCOUNT_ID: 0138F2-2D19F2-56DCE1
BILLING_ENABLED: True

PROJECT_ID: project-4dfbac02-e93e-4819-bed
BILLING_ACCOUNT_ID: 0138F2-2D19F2-56DCE1
BILLING_ENABLED: True

PROJECT_ID: model-union-500321-r9
BILLING_ACCOUNT_ID: 0138F2-2D19F2-56DCE1
BILLING_ENABLED: True
```

operating-tiger
-503823-m4

And to identify only the unlinked project I can use grep command like below
grep -v -f <(gcloud billing projects list --billing-account=0138F2-2D19F2-56DCE1 --format="value(projectId)") <(gcloud projects list --format="value(projectId)")

Other features in Billing section in GCP includes Cost Breakdown, Budget and Alerts which are quite straightforward to ease cost control.

Credit Card Verification

Cloud service platforms such as Google Cloud Platform (GCP) rely on credit cards for automated billing and timely settlement. It works effectively under normal circumstances, but issues can arise when unused compute resources are not regularly reviewed. Many users may overlook this, especially when cloud environments are used briefly or for non-production purposes.

GCP's support model differs from providers such as AWS or Azure. For certain billing matters, GCP requires formal communication through physical mail rather than direct online or phone support. Sales teams can be contacted, but they are not authorized to resolve billing or technical issues. This structure can make dispute resolution slower and more procedural.

In Canada, GCP's regional office is located in Halifax, Nova Scotia. After submitting a formal mailed request, I received confirmation that a one-time waiver would be applied to a 6,000-dollar invoice. However, the waiver was not immediately reflected in Google's central billing system. Because my credit card had already been closed to prevent automatic charges during the dispute, the system generated a standard overdue notice and a debt recovery warning. A second mailed submission to Google's U.S. headquarters, including the waiver approval, resolved the discrepancy.

Credit Card Considerations During a Billing Dispute

Leaving the credit card open

- Allows the provider to automatically charge the disputed amount.
- May result in interest charges if the dispute is not resolved quickly.
- Creates additional financial exposure during the investigation period.

Closing the credit card

- Prevents automatic charges while the dispute is under review.
- Avoids interest accumulation on a potentially incorrect invoice.
- Ensures the provider completes dispute process before any payment is processed.

Closing the card was a practical measure to avoid unintended charges during an unresolved billing review.

Paying the Disputed Amount Upfront

Paying the full amount before the dispute is resolved would have required temporarily using personal savings and relying on the provider to reimburse the funds later. This approach transfers all financial risk to the customer and is not necessary when a formal billing dispute is already in progress.

Example of possible scenario after an automated payment was cancelled

> [URGENT ACTION REQUIRED] Google Cloud account (ID _____)
> at risk of transfer to Debt Recovery agency
>
> From **Google Cloud Platform**
>
> If you have recently paid, then please disregard this notice.
> Thank you for your payment.
>
> Dear Customer,
> The outstanding balance on your Billing Account ID _____ remains unpaid.
>
> To prevent being transferred to a Debt recovery agency, please settle this debt as soon as possible no later within 10 working days via payment on your account.
>
> Transfer to a Debt recovery agency can incur additional fees.

The job market may list lots of buzzwords in job descriptions such as Kubernetes, Terraform, GCP, AWS, Azure, BigQuery, Looker, Dataproc, IAM, Networking, CI/CD, Containers, Serverless, etc and this pressure can lead to job seekers :

- spinning up resources for practice
- forgetting to shut them down
- incurring unexpected charges

Above case may help ML enthusiast to understand as part of responsible cloud education

- cloud learning environments must be monitored
- billing is part of cloud literacy
- disputes require documentation
- payment methods should be managed carefully

Three Months Trial

Although three months trial is given for specific email, many would ask if it is possible to repeatedly open and close GCP account with new emails and the answer would be No.

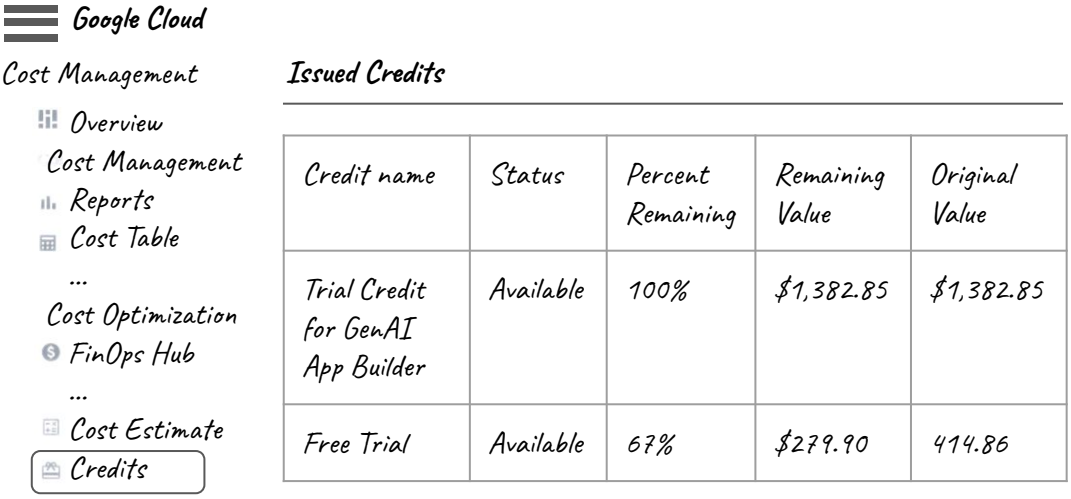
Google verifies multiple identity signals, not just the email address. Here are the key controls:

- Credit card fingerprinting – Google tracks the card number, issuer, and metadata.
- Billing address reuse – Reusing the same address triggers fraud detection.
- Device fingerprinting – Browser, IP, OS, and hardware identifiers are logged.
- IP and geolocation patterns – Repeated sign-ups from the same region or IP block are flagged.
- Identity verification – Even if the email is new, the payment identity is not.

Because of these checks, simply creating a new Gmail account does not reset my eligibility. If the goal is to practice cloud computing without cost, here are safer options:

- Use GCP's always-free tier
- Set strict budget caps
- Use local emulators like Cloud SDK and BigQuery emulator
- Use AWS free tier + Azure free tier for multi-cloud learning

Below is an example of free tier screen, GenAI App Builder was included and it is good for more than 3 months (12 months).



Credit name	Status	Percent Remaining	Remaining Value	Original Value
Trial Credit for GenAI App Builder	Available	100%	$1,382.85	$1,382.85
Free Trial	Available	67%	$279.90	414.86

Why Google Blocks Repeated Free Trials ?

Google Cloud's free trial is intended for:

- new developers
- new businesses
- legitimate evaluation
- onboarding to paid usage

Repeated sign-ups fall under abuse prevention, similar to AWS and Azure.

Google's terms explicitly prohibit:

- creating multiple accounts to obtain repeated promotional credits
- bypassing identity verification
- using false or misleading information to obtain credits

This is standard across all cloud providers.

What happens if I try ?

Google may:

- suspend the new account
- block the credit card from future trials
- flag the identity for promotional abuse
- deny future credits even for legitimate projects
- require manual verification for any future billing

None of these involve legal consequences, but they permanently remove access to free trials.

---

type	CreditID	Usage Scope	Start Date	End Date
one-time	a96...	Certain Usage	June 24, 2026	June 24, 2027
one-time	Free Upgrade	Certain Usage	June 23, 2026	Sept 22, 2026

Example of GCP Cost

GCP is typically the cheapest of the major clouds for compute and data workloads, but AWS and Azure still provide stronger enterprise support, deeper ecosystems, and more mature customer service. This doesn't mean GCP support is "bad" – it means AWS and Azure have far deeper enterprise support infrastructure, especially for regulated industries. I is a pattern shown across multiple independent 2025–2026 benchmarks. GCP is consistently the lowest-cost hyperscaler for compute-heavy and data-heavy workloads. AWS is usually the most expensive. Azure sits in the middle.

GCP is usually cheaper on raw compute
Across equivalent VM configurations, GCP is cheaper than AWS and Azure.
Examples for a 4 vCPU / 16GB instance as per cloudcalcep.com
- AWS: $140.16/mo
- Azure: $140.16/mo
- GCP: $121.36/mo (≈15% cheaper)

General-purpose compute often shows GCP 3–5% lower than AWS/Azure.

GCP is also competitive on databases and some storage
Managed PostgreSQL HA:
- AWS: $835/mo
- Azure: $424/mo
- GCP: $364/mo (lowest)

Object storage for hot tier (tcoiq.com):
- AWS S3: $0.0207/GB
- Azure Hot: $0.018/GB
- GCP Standard: $0.020/GB (similar, slightly cheaper than AWS)

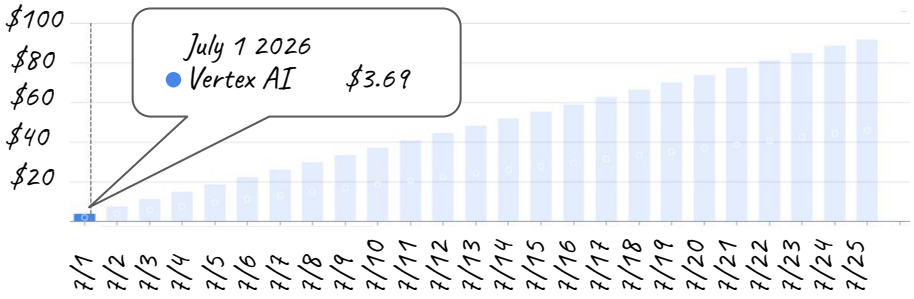
Egress fees: GCP is slightly cheaper than AWS/Azure (quickcloud.co)
- AWS: $0.09/GB
- Azure: $0.087/GB
- GCP: $0.085/GB

Why GCP is cheaper ?
- Aggressive pricing to gain market share (GCP 11%, AWS 31%, Azure 24%).
- Automatic sustained use discounts.
- Strong optimization for data/ML workloads.

Why support feels weaker ?

- Smaller enterprise presence.
- Fewer compliance certifications in some industries.
- Smaller ecosystem and fewer region-specific services.
- Product discontinuation history affects customer confidence.

Example of accumulated Vertex AI daily cost of $3.69 for a simple CCS/CU classifier



Service	Usage Cost	Saving Programs	Other Savings	Subtotal
Vertex AI	$ 91.46	-	-	$91.46
Cloud Storage	$ 0.00	-	-	$ 0

Example of Refund from Disputed Charges from Google in Halifax, NS (see page 99)

Google™

Credit Memo

Google Cloud Canada Corporation
1959 Upper Water Street
Suite 900
Halifax, NS B3J 3N2
Canada

Bill to

Details		Google Cloud	
Credit memo number	----	Total in CAD	- CA$ 6,197.65
Invoice Date	----	Summary for 3 Dec 20XX	
Billing ID	----	Subtotal in CAD	- CA$ 5,405.00
		ON HST (13%)	- CA$ 702.65
		Total in CAD	- CA$ 6,197.65

Reflection on Deployment

The model I just built and uploaded to GCP would appear useful to anyone who wants quick, automated classification of CCS/CCU suitability — but the person who relies on it risks making decisions based on a model that only memorized its training data and cannot generalize. The biggest thing that could go wrong is false confidence: the model will give highly confident predictions that are actually meaningless.

Now my project is ready for others to use, who do I think will benefit from it the most?

- Operational engineers — They often need rapid triage of industrial sites to see which ones might be candidates for CCS/CCU. Even a rough model can help them prioritize which sites to inspect manually.

- Policy analysts — They might use it for early-stage screening to estimate how many facilities in a region could participate in carbon reduction programs.

- Data scientists — Ironically, they benefit the most right now. A weak model is still useful for pipeline testing, endpoint deployment, latency checks, and verifying that GCP model registry + bucket + endpoint integration works.

- Prototype stakeholders — Anyone evaluating feasibility or workflow integration (not accuracy) can benefit from seeing the model in action.

In short:
Right now, the model is most beneficial for workflow validation, infrastructure testing, and early-stage exploration — not real decision-making.

What might go wrong when a real person uses it?

1. False confidence in perfect accuracy
   Because the model memorized the training data, it will output predictions with extremely high confidence. A real user might assume:

   "The model is 100% accurate — I can trust this."

   But in reality, it has zero ability to handle unseen sites.

2. Misclassification of real industrial sites
   When exposed to new NAICS codes, new emission patterns, or different reporting histories, the model may:
   - classify unsuitable sites as suitable
   - classify suitable sites as unsuitable
   - fail entirely on edge cases

   This could lead to costly engineering mistakes, misallocated funding, or incorrect feasibility assessments.

3. Overfitting-induced brittleness
   Because the model memorized the training set:
   - No split data for test like 20% due to data starvation (100% data = 100% test)
   - A decision boundary across 26×20 combinations by having only 150 observations
   - slightly different emission logs

   It produces inconsistent prediction (overfitted models are more dangerous than under fitted ones).

4. Misleading feature importance
   Before I developed this model, feature importance has warned which feature is more / less important. For academic purpose I applied both to bring a discussion. In my case:
   - the importance scores reflect memorization
   - not true causal or predictive relationships
   - A real user might incorrectly believe:
     "Years reported is the most important factor – let's optimize around that."
     Which could lead to wrong operational decisions.

5. Reputational risk
   If someone uses the model in a real CCS/CCU screening workflow and it fails, it reflects poorly on:
   - the model
   - the data pipeline
   - the engineering team
   - the organization's technical credibility

Dealing with Decarbonization

The global push toward decarbonization – driven by rising clean-energy demand – is reshaping the oil and gas sector, as noted by Nicolette Beharie in Shop Metalworking Technology (May 2023). Around the world, escalating threats to health, food production, infrastructure, and livelihoods from global warming continue to dominate headlines. As pressure mounts to significantly reduce greenhouse-gas emissions, oil and gas companies find themselves at the center of a complex challenge: enabling a cleaner energy future while sustaining a prosperous economy. Metal manufacturers that supply this sector are preparing to support that transition.

In early 2023, Imperial Oil announced a $720-million investment to build Canada's largest renewable-diesel facility at its Strathcona refinery near Edmonton. The project is expected to produce more than one billion litres of renewable diesel annually, using locally sourced feedstocks such as canola oil. The facility will rely on low-carbon hydrogen produced through carbon capture and storage (CCS) technologies, supporting Canada's low-emission fuel standards.

Jay Averill, spokesperson for the Canadian Association of Petroleum Producers (CAPP), emphasized that the oil and gas industry is a major investor in clean technologies – including

- carbon capture, utilization and storage (CCUS),
- advanced solvent processes,
- waste-heat recovery, and
- electrification



all of which help producers continue reducing emissions.

Shell Global mentioned expanding renewable-energy supply is only half the battle. Achieving a net-zero future requires transforming demand as well. Customers must actively choose low- and zero-carbon products and have the right equipment and infrastructure to use them. Similar analogy to WWF campaign below "When the buying stop the killing can too". The energy transition is not solely about producing cleaner energy; it is about ensuring the market is ready and willing to adopt it.



I AM NOT A RUG

As few as 3200 wild tigers remain. Poaching for their skins, bones and other parts is greatest immediate threat to their survival

WWF

In a smaller, more personal context, I want to share a story from my early years in Canada. When I first arrived, I worked full-time at GE during weekday while also working nights/weekends at Walmart. At the same time, I was completing an electrical engineering certification and caring for my newborn daughter.

To prepare for weekly flyer launch, I took ownership with agility to unpack seven two-meter-high skids of fresh and frozen meat every Thursday night on my own. Beyond restocking shelves, I actively optimized our storage space—keeping freezer inventory at near-zero and cooler stock to a minimum—which drastically simplified our monthly inventory audits and improved stock turnaround.

What I was practicing—without formally naming it—was just-in-time (JIT). By moving product exactly when needed and avoiding excess stock, I was reducing waste, unnecessary energy use, and avoidable refrigeration load. That same discipline is foundational to decarbonization, where eliminating operational waste directly lowers emissions and improves system efficiency.

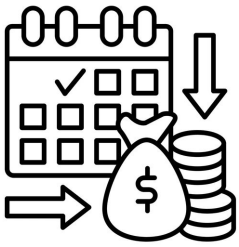
Paul (Greenberg 2021) highlighted one of the simplest ways to reduce a carbon footprint: stay focused on the goal rather than the obstacles. That principle guided this effort.











Mike (Berners-Lee 2022) shows how the world's largest sources of carbon emissions — from wildfires, global IT infrastructure, wars, deforestation, black carbon, annual industrial output, and even the full combustion of remaining fossil fuel reserves — each of them alone add billions of tonnes of CO_2 every year. In contrast, the smaller but meaningful contributions from tree planting, marine restoration, soil carbon sequestration, biochar, bioenergy with carbon capture and storage (BECCS), enhanced rock weathering, and direct air capture with carbon storage (DACCS) offer pathways to offset part of that burden.

He also highlights how everyday choices, including something as ordinary as carrying a mortgage like 270K with 2.5% interest, quietly add a ton of carbon annually. By adopting a lean, keen and green mindset, I managed to finish mine in just 9 years and 9 months.

Final Reflections

I just built a model on real, messy data. Did it behave the way I expected? If not, what did I think was going to happen?

The model didn't behave the way I expected, and that actually taught me a lot about the data. I anticipated some noise and inconsistency, but real-world messiness was stronger than expected: duplicated patterns, missing values, and limited variation caused Decision Tree to memorize instead of generalize. I thought I'd see at least modest structure the tree could learn, but instead it produced perfect accuracy only on the training examples. The surprise wasn't the poor generalization — it was how quickly the model collapsed into overfitting, revealing that the data quality was the real bottleneck, not the algorithm.

The class imbalance was left in on purpose. Did I notice it before, or did my results tip me off? What was my first instinct when I saw the classification report?

As the class imbalance was intentional, the results made it impossible to ignore. I noticed hints of imbalance earlier during EDA, but the classification report made it obvious: the model was overwhelmingly predicting the majority class and showing perfect accuracy only because it memorized those patterns. My first instinct was that the model wasn't learning meaningful structure at all — it was simply exploiting the imbalance. That reaction confirmed the dataset, not the algorithm, was a real limitation. The report didn't just reveal imbalance; it revealed how little signal the model actually had to work with.

Amii picked the Decision Tree because people can follow its logic. Looking at my results, do I think that was the right call for this data, or did we leave performance on the table?

A Decision Tree was a reasonable choice for interpretability, but with this dataset it likely left performance on the table. The data was extremely small, so a shallow tree doesn't have enough capacity to learn meaningful structure and mostly underfits or memorizes. An ensemble like a Random Forest would stabilize predictions and reduce variance, while still allowing interpretability through a single representative tree or surrogate model (n_estimators = 1). So while Decision Trees are easy to follow, this particular setup suggests we sacrificed potential performance for simplicity.

Synthetic data, which is artificially generated to mimic real data and is often used to expand dataset size or balance classes, was an option chosen not to pursue. Knowing how the model performs, if I am designing this project from scratch, would I consider using synthetic data? What changes would I make to improve the model or the dataset?

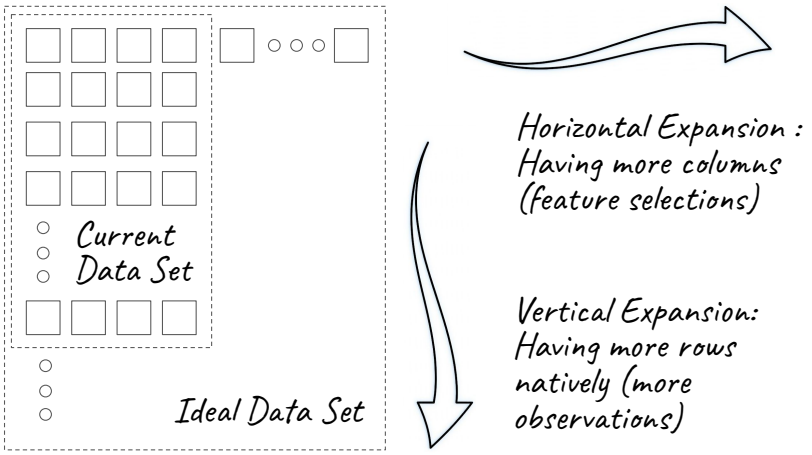
With 20 CU (minority) and 130 CCS (majority), ROS is the more acceptable choice., SMOTE is risky because it generates synthetic points by interpolating between very few real samples. Random Oversampling (ROS) is safer and more stable in this situation.

Technique	Best When	Pros	Cons
SMOTE	Minority class ≥50	Reduces overfitting	Can create noise
ROS	small minority class	Simple, stable	Overfitting risk

What is one thing I would fix if I worked with this data again?

What I would fix: Expand the dataset's dimensions in both directions.

- Vertically (Rows): Collect more native observations to build a more robust, representative sample.
- Horizontally (Columns): Integrate more feature selections to enrich the dataset and provide broader context for analysis.



Imagine handing this model to someone who has to decide where to spend the next hundred million dollars on carbon capture. What would I tell them it can do, and more importantly, what would I warn them it cannot?

The model shows what the workflow could look like, but it cannot guide investment. The real value comes from building the data ecosystem — sensors, incentives, privacy, governance, and site participation. Once that exists, a strong model becomes trivial. And while hundreds of millions sounds large for MLOps, it is small compared to the multi-billion-dollar cost of real CCS deployment (such as SaskPower at Boundary Dam or Imperial Oil Renewable Diesel at Strathcona). The money should be spent first on data collection and later on physical infrastructure, not on the model itself.

Index

ABB 11
Adaboost 43
Agentic platform (Vertex AI) 9, 14, 15, 73, 80, 81, 82, 84, 85, 103
Agile 74
Airflow 78
Alberta machine intelligence institute (AMII) 4, 18, 77, 108
Alberta's industrial decarbonization strategy 4
Alberta's technology innovation and emissions reduction (TIER) 5, 8, 35
Argo CD 77
Artifact 81, 83, 85
Artificial intelligence 44, 76
Automl 80, 81, 82, 83
Autoscaling 15, 70, 72, 74
Avro 26, 27
Azure DevOps 77
Bigquery 72, 81, 82, 87, 99, 100
Billing 17, 95, 96, 97, 98, 99, 100, 101, 103
Blue hydrogen project 40
Box plots 49
Buckets 26, 72, 73, 83, 85, 86, 92, 93
Budget alert 7, 95, 97
Budget-conscious 78
Canadian sovereign AI compute strategy 4
Categorical data 42
CI/CD/CT 77
Classification 8, 16, 25, 26, 27, 41, 43, 48, 52, 57, 60, 64, 65, 77, 78, 89, 94, 104, 108
Cloud composer 27
Cloud data fusion 27
Cloud engineering 70, 71, 72, 94
Cloud functions 27
Cloud literacy 99
Cloud-native enterprise system 71
Clustering 43

CO_2 4,5, 11, 12, 13, 16, 17, 41,
Command line interface (CLI) 76, 89, 96
Consistent, consistency 6, 18, 20, 44,59, 70, 74, 76, 102
Contabo 78, 79
Container orchestration 70
Continuous binning 58
Continuous improvement (kaizen) 9, 74, 75
Continuous integration / continuous delivery (CI/CD) 27, 47, 70, 74, 76, 77, 99
Convolutional neural network (CNN) 78
Cost-complexity pruning 58
CPU, GPU, TPU 10, 11, 72, 73, 79, 88, 102
Credit card 98, 100, 101
DAG (Directed acyclic graph) 78
Data acquisition 6
Data drift 7, 71, 88
Data engineering 27, 94
Data leakage 48
Data loss prevention (DLP) 95
Data preview 21, 22
Data quality 7
Data scientist 7, 80, 104
Data type 21, 24, 43
Data visualization 9, 27, 38, 39, 42, 45, 46, 47, 50,
Dataflow 27, 73, 87
Dataops 74
Dataproc 27, 73, 87, 99
Decision tree classifier 7, 10, 20, 38, 40. 41, 42, 43, 44, 45, 46, 47, 48, 49, 54, 56, 57, 58, 59, 60, 61, 66, 73, 94, 108
Deep learning 73
Devops 15, 27, 70, 71, 74, 75, 76, 77, 94
Digital twins 71
Dimensionality reduction 43
Distributed compute 72

Distributed training 71, 73
Docker 14, 15, 76, 78, 82, 83
Docker-in-docker 76
Duplicated data 21, 24, 60, 87, 108
Elastic Kubernetes Services (AWS EKS) 79
Electrification 106
Endpoints 26, 72, 73, 80, 88, 89, 90, 91
Engie 40
Enhanced oil recovery (EOR) 120
Ensemble (method) 11, 38, 39, 40, 59, 60, 64, 108
Equinor 40
Exploratory data analysis (EDA) 6, 20, 21, 48, 49, 60, 108
Extract-load-transform (ELT) 18, 26, 27, 73
F1-score 48, 49, 53, 65
Feature engineering 6, 21, 38, 58, 80,
Feature importance 38, 39, 40, 105
Feature selection 59, 109
Feedstock 5, 16, 106
Fluid mechanics 11
Flux CD 77
Gaussian process models 43
GenAI 10,100
General Electric (GE), GE Vernova 26, 75, 90, 107
Generalize, generalization 7, 13, 14, 21, 61, 68, 69, 104, 108
Geological formation, suitability, maps, factors 5, 16, 17
Gini impurity or entropy 48, 54, 55, 56, 59, 60
Github 18, 77, 95,
Gitlab 77
Gitops 70, 77
Go 27
GoAnywhere MFT 27
Google Colab 18
Gradient boosting 59
Gradle 76
Greenhouse gas reporting program (GHGRP) 18, 19, 22, 24, 25, 26, 28, 30, 31, 40

Groovy 27, 76
HAProxy 79
Heat map 50, 64
High-cardinality handling 20, 49
Honeywell 74, 75
Horizontal and vertical expansion 70
Human expert 8, 12, 14, 15, 17
Human intervention 79
Human-readable 32, 38, 42
Hyperparameter 7, 73, 80
Hyperscaler (Meta, AWS, Azure, GCP) 41, 79, 102,
Identity access management (IAM) 94, 95, 96, 99
Imbalance (data) 20, 48, 49, 59, 61, 64, 108
Imperial Oil 40, 106, 109
Improvability 7
Inadequate splits 48
Informatica ELT/ETL 27
Infrastructure as a code (IaaC) 70
Infrastructure as a service (IaaS) 78
Internet of thing (IoT) 71
Interoperability 7
Interpretable 4, 8, 38, 39, 40, 41, 44
Jenkins 76
Joblib 27, 82, 83, 85, 86
Jones Lang LaSalle (JLL) 77, 94
JSON 27, 47, 69, 85, 89, 90
Julia 27
Just in time 107
Keras 77, 82
Kernel density estimation 20
Kubeflow 80, 81
Kubernetes 14, 15, 74, 77, 79, 95, 99
Kubernetes engine (GKE) 72, 73, 79
LabelEncoder 37, 39
Lambda 35, 36, 46, 79

Latency 7, 47, 72, 79, 104
Latency requirement 72
Layoff 95
Lean 74, 75
Lego Mindstorms EV3 76
LightGBM 73, 81, 82
Linear support vector machines 43
Load balancer 70, 79
Logistic regression 78
Lotus Notes 21
Machine learning project lifecycle (MLPL) 6
Machine ops 4, 9
Machine-learning workflows 21, 71, 76
Mass-preference data collection sensors 9
Matrix computation 9, 28, 38, 50, 51, 64, 65
Mavel 76
Mechanical engineering 11, 26, 120
Memorizing 7, 38, 60, 61, 68, 69, 105, 108
Mersen 18
Messy data 41, 54, 108
Metric decay 48
Missing values, data, timestamps, fields 6, 20, 21, 41, 108
MLflow 27, 81, 82, 83
MLOps 4, 9, 14, 47, 70, 71, 76, 77, 80, 81, 87, 94, 95, 109
Model inference 26, 27, 69, 72, 73, 80, 88, 90,
Model registry 26, 27, 72, 80, 81, 85, 88, 90, 91, 104
Model selection 7, 80
Monitoring 7, 15, 71, 74, 84, 88, 92, 93
Monotonic constraints 59
Multi-region deployments 70
Multicollinearity 20
MySQL 78
Naive Bayes 43

Nearest neighbors 43
Net-zero by 2050 5
Neural networks 43
NGINX 79
Nodes 44, 45, 46, 54, 57, 78, 79
NOx 41
Null value 21, 24, 25
Numpy (np) 37, 50
Observability, Monitoring 7, 15, 70, 71, 74, 84, 88, 92, 93
ONNX 27, 82, 83
Operational engineers 104
Outliers 6,20, 49
Overfitting 7,21,38,40,49, 58, 60, 61, 105, 108, 109
Parquet 26, 27
Physical AI 9
Pickle (.pkl) 82, 83, 85
Precision 48, 49, 53, 65
Predictive maintenance 71
Predictive performance 9
Pub/Sub 27, 73
Python 9, 17, 18, 27, 39, 78
Quadratic discriminant analysis 43
R 27, 78
Random forest classifier 38, 39, 40, 43, 59, 60, 61, 64,108
Raspberry Pi 76
Support Vector Machines (SVM) 43
Recall 48, 49, 53, 65
Redundant features 58
Regression 20, 43, 78
Renewable diesel 40, 106, 109
Reproducibility 7
Reproducible environments 70
REST (API) 27, 89
Robustness 9
Rockwell 75

Rust 27
S3-compatible storage 78
Sandbox projects 95
Sankey chart 56
SAP ERP 21
SavedModel 26, 27, 82, 83,
Scalability 10, 15, 47, 70, 74, 75, 76
Scalable 4, 8, 15, 17, 27, 41, 46, 75, 76, 78
Scatter plots 20, 49
Schneider Electric 21
Security 10, 94,
Serverless 27, 78, 79, 99
Siemens 75
Six sigma 75
Software engineering 10, 74, 76
SOx 41
Spark 27, 76
Split criterion tuning 59
SQL 47, 72, 78, 102
SQL analytics 72
Statistical process control (SPC) 26
StratifiedKFold 48,50, 83
Streaming ingestion 72
Synthetic minority over-sampling technique(SMOTE) 20, 59, 60, 108, 109
TensorFlow 27, 77, 80, 81, 82, 85
Terraform 74, 99
Thermodynamics 11
Train/test split 39, 48, 61, 64, 83
Training 7,10,11,16, 29, 38, 48, 68, 69, 71,
73, 77, 80, 81, 83, 94, 104, 105, 108
Tree pruning 58, 59, 60
Trias Sentosa 57
Tuning 7, 58, 59, 73, 80
Version control 47
Virtual dedicated servers (VDS) 78

Virtual private servers (VPS) 78
Walmart 107
Weighted distribution, average 44, 46, 52, 65
XGBoost 59, 73, 81, 82, 83

References

Alpaydin 2021, Machine Learning, The MIT Press

Assael 2011, Commonly Asked Questions in Thermodynamics, CRC Press

Berners-Lee 2022, The Carbon Footprint of Everything, Greystone Books

Bisong 2019, Building Machine Learning and Deep Learning Models on Google Cloud Platform: a Comprehensive Guide for Beginners, Apress Media

Bowles 2020, Machine Learning With Spark and Python Essential Techniques for Predictive Analytics, John Wiley & Sons

Brannen 2025, The Story of CO2 Is the Story of Everything: How Carbon Dioxide Made Our World, Ecco Press

Ervine 2018, Carbon, Piloty Press

Esposito 2020, Introducing Machine Learning, Microsoft Press

Fenner 2020, Machine Learning With Python for Everyone, Addison-Wesley

Gift, Deza 2021, Practical MLOps, O'reily Media

Greenberg 2021, The Climate Diet: 50 Simple Ways to Trim Your Carbon Footprint, The Penguin Books

Haviv, Gift 2023, Implementing MLOps in the Enterprise: a Production-first Approach, O'reily Media

Kehrer, Kriser 2024, Machine Learning Upgrade: a Data Scientist's Guide to MLOps, LLMs, and ML Infrastructure, John Wiley & Sons

Lane 2020, Machine Learning for Kids: a Hands-on Guide to the Future of Artificial Intelligence, No Starch Press

Luu, Pumperla, Zhang 2024, MLOps with Ray, Best Practices and Strategies for Adopting Machine Learning Operations, APress Media

Massaron, Mueller 2026, Machine Learning for Dummies, John Wiley and Sons

Matloff 2023, The Art of Machine Learning: a Hands-on Guide to Machine Learning With R, No Starch Press

Mattmann 2020, Machine Learning With TensorFlow, Manning Publications Co.

Mishra 2020, Machine Learning for iOS Developers, John Wiley & Sons

Mona 2024, Official Google Cloud Certified Professional Machine Learning Engineer Study Guide, John Wiley & Sons

Moroney 2020, AI and Machine Learning for Coders: a Programmer's Guide to Artificial Intelligence, O'reily Media

Norris 2020, Machine Learning With the Raspberry Pi: Experiments With Data and Computer Vision, Apress Media

Nwanganga, Chapple 2020, Practical Machine Learning in R, John Wiley & Sons

Paper 2020, Hands-on Scikit-learn for Machine Learning Applications: Data Science Fundamentals With Python, Apress Media

Polak 2023, Scaling Machine Learning With Spark: Distributed ML With MLlib, TensorFlow, and PyTorch, O'reily Media

Pulari, Murugesh, Vasudevan, Akhsay 2025, Machine Learning: a Comprehensive Beginner's Guide, CRC Press

Putatunda 2021, Practical Machine Learning for Streaming Data With Python: Design, Develop, and Validate Online Learning Models, Apress Media LLC

Rabie 2024, Fluid Power Engineering, McGraw Hill

Raschka 2024, Machine Learning Q and AI : 30 Essential Questions and Answers on Machine Learning and AI, No Starch Press

Silaparasetty 2020, Machine Learning Concepts With Python and the Jupyter Notebook Environment Using Tensorflow 2.0, Apress Media

Singh 2019, Practical Machine Learning and Image Processing for Facial Recognition, Object Detection, and Pattern Recognition Using Python, Apress Media

Song 2022, Automated Machine Learning in Action, Manning Publications

Suryadevara 2021, Beginning Machine Learning in the Browser: Quick-start Guide to Gait Analysis With JavaScript and TensorFlow.js, Apress Media

Testas 2023, Distributed Machine Learning With Pyspark: Migrating Effortlessly From Pandas and Scikit-Learn, Apress Media

Wagner 2021, Geoengineering: the Gamble, Piloty Press

About the Writer

With experience across the energy sector, this project aligns naturally with a passion for machine learning (ML) and mechanical engineering—fields that anticipated the early, large-scale adoption of digital models. A background in integrated CAD, CAM, and CNC workflows provided hands-on exposure to sophisticated data management and data-driven technologies long before modern machine learning became mainstream.

Contributions to the early development of Enhanced Oil Recovery (EOR) helped shape a technology that—much like Gernot (Wagner 2021) describes—began as a calculated gamble worth exploring. Today, EOR accounts for 80% of global carbon capture and storage and about 30% of carbon utilization, providing a distinctive vantage point at the intersection of mechanical engineering and modern machine-learning applications.

